第 32 章：Class Odds and Ends（类的杂项收尾）

> 原书：《Learning Python》（6th Edition），作者 Mark Lutz 本章地位：这是全书 OOP（面向对象编程）教学部分（第六部分）的压轴章。前面几章讲完了类的核心，本章把"边角料"集中扫尾：定制内置类型、类与类型的关系、slots（插槽）与 property（属性）、静态方法与类方法、装饰器与元类、以及 super 调用的完整故事。这些主题大多"高级且可选"，应用开发中不常亲手写，但阅读别人代码时又随时可能撞上——所以值得通读。

32.1 开篇引言（Chapter Introduction）

英文原文

> This chapter concludes our look at OOP in Python by presenting a collage of more advanced class topics. We will survey customizing built-in types, the relationship of classes and types, attribute tools like slots and properties, the special-case static and class methods, decorators and metaclasses, and the super call's complete story. Some of these are introduced here but resumed by focused chapters in this book's Part VIII, "Advanced Topics".

As we've seen, Python's OOP model is, at its core, rel atively simple, and some of the topics presented in th is chapter are so advanced and optional that you ma y not encounter them very often in your Python appli cations-programming career. In the interest of compl eteness, though—and because you never know when an "advanced" topic may crop up in code you use—w e'll round out our discussion of classes with a brief lo ok at these advanced tools for OOP work.

As usual, because this is the last chapter in this part o f the book, it ends with a section on class-related "go tchas" and a set of lab exercises for this part to help cement the ideas we've studied here. Beyond these e xercises, studying larger OOP Python projects or start ing some of your own is heartily recommended as a s upplement to this book. As with much in life and co mputing, the benefits of OOP tend to become more apparent with practice.

NOTE（注意）— Blast from the past（往昔的回响）: Py thon 3.X launched with a mandatory "new-style" clas s model that could be enabled in 2.X as an option; 2. X's own model was dubbed "classic." At least in term s of its OOP support, new-style classes transformed P ython into a different language altogether—one that borrows much more from, and is often as complex as , other languages in this domain. The last chapter's M RO and most topics in this chapter were part of this package. Because this book is now focused on 3.X on

ly, the term "new style" is moot and unused here—all its classes qualify.

中文翻译

本章通过拼贴一组更高级的类主题，为我们对 Python 中 OOP（面向对象编程）的探讨画上句号。我们将概览：定制内置类型（customizing built-in types）、类与类型（types）的关系、slots（插槽）与 property（属性）之类的属性工具、特殊的静态方法（static methods）与类方法（class methods）、装饰器（decorators）与元类（metaclasses），以及 super 调用的完整故事。其中一些主题在这里只是引入，会由本书第八部分"高级主题"中的专门章节继续深入。

正如我们所看到的，Python 的 OOP 模型在核心上是相对简单的，本章呈现的一些主题如此高级、如此可选，以至于你在 Python 应用程序开发的职业生涯中可能不常遇到它们。但为了完整起见——因为你永远不知道某个"高级"主题什么时候会在你使用的代码里冒出来——我们将以对这些 OOP 高级工具的简要考察，来收尾我们对类的讨论。

和往常一样，因为这是本书这一部分的最后一章，章末有一节关于类相关的"陷阱"（gotchas）和一组本部分的实验练习（lab exercises），帮助巩固我们在此学到的思想。除了这些练习，认真研读大型 OOP Python 项目、或自己动手开始一些项目，被热忱推荐为本书的补充。与生活中和计算世界中的许多事物一样，OOP 的好处往往在实践之后才愈发明显。

注意——往昔的回响：Python 3.X 一经发布就强制使用"

新式"（new-style）类模型，这一模型在 2.X 中只是可选启用项；2.X 自己的模型被称为"经典式"（classic）。至少在 OOP 支持方面，新式类把 Python 彻底变成了另一门语言——一门从该领域其他语言借鉴更多、复杂度往往也与之相当的语言。上一章的 MRO 和本章的大多数主题都属于这个体系。因为本书现在只针对 3.X，"new style"（新式）这个词在这里已无意义、不再使用——它的所有类都够格。

深度理解

·核心概念：本章是"高级类主题大杂烩"，作者自己承认这些主题"高级且可选"。学习本章的心态应当是：不追求立刻精通，追求"见过、知道存在、知道去哪查"。

·底层视角：注意 "new-style"（新式）这个词的退场。Python 2 时代类分"经典式"和"新式"两种模型，二者在属性查找、MRO、与 type 的关系上行为不同；Python 3 只有新式类，统一了类（class）与类型（type）。本章几乎所有机制（MRO、type/object 体系、super）都建立在这个统一之上。

·设计原因：把高级主题放到部分末尾，是教学上的刻意安排——先让读者扎实掌握"能用的核心"（继承、多态、运算符重载），再展示"锦上添花"的工具，避免初学者被 MRO、元类等概念劝退。

·实际问题：真实项目里，你大概率会在别人写的框架里遇到 slots、super()、@staticmethod、@property、元类这些写法。不认识它们，轻则读不懂代码，重则以为程

序出了 bug。

·初学者误区：以为"会用类"就等于"懂 OOP"。作者反复提醒：本章很多工具是"全有或全无"（all-or-nothing）的，乱用反而会破坏代码——例如 super 在多继承中的"魔法"行为、slots 与 dict 的兼容性问题。

32.2 扩展内置对象类型（Extending Built-in Object Types）

英文原文

> Besides implementing new kinds of objects, classes are sometimes used to extend the functionality of Python's built-in object types to support more exotic data structures. For instance, to add queue insert and delete methods to lists, you can code classes that wrap (embed) a list object and augment it with insert and delete methods that process the list specially, using the delegation technique we studied in Chapter 31. You can also use simple inheritance to customize built-in types for such custom roles. The next two sections show both techniques in action.

中文翻译

> 除了实现新类型的对象之外，类有时还被用来扩展 Python 内置对象类型（built-in object types）的功能，以支持更特别的数据结构。例如，要给 list（列表）添加队列（que

ue）式的插入与删除方法，你可以编写一个类来包装（嵌入）一个列表对象，并用专门处理该列表的插入和删除方法增强它——这正是我们在第 31 章学过的委托（delegation）技术。你也可以用简单的继承来定制内置类型以承担此类角色。接下来的两节将演示这两种技术。

深度理解

·核心概念：扩展内置类型的两条路——嵌入（embedding）与子类化（subclassing）。

·底层实现：嵌入是"组合"思想：类里持有（has-a）一个 list，把请求委托给它；子类化是"继承"思想：类是（is-a）一种 list。Python 3 中 list、str 本身就是类，所以能直接当基类。

·设计原因：有的场景你想给已有类型加行为又不想动它本身；嵌入保留完全控制权（可拦截一切），子类化则自动继承全部原行为（代码最省）。

·实际问题：本章的 Set（集合）类就是活例子——把第 16、18 章用函数实现的集合运算"升级"为类，从而支持多实例、可继承、可用运算符。

·初学者误区：误以为内置类型不能子类化；或反过来，以为子类化内置类型是家常便饭。其实这属于小众但威力强大的技巧，了解即可。

代码分析（嵌入方式）

Example 32-1（文件 setwrapper.py）把第 16/18 章的集合函数"复活"成一个 Python 类：把集合函数变成方法

，并添加一些基本的运算符重载。这类本质上是用额外的集合运算包装（wraps）一个 Python list：

```
class Set:
    def __init__(self, value = []):        # Constructor
        self.data = []                      # Manages a list
        self.concat(value)                  # Removes duplicates…

    def intersect(self, other):             # other is any iterable…
        res = []                            # self is the subjects…
        for x in self.data:
            if x in other:                  # Pick common items…
                res.append(x)
        return Set(res)                     # Return a new Set…

    def union(self, other):                 # other is any iterable
        res = self.data[:]                  # Copy of my list…
        for x in other:                     # Add items in other…
            if not x in res:
                res.append(x)
        return Set(res)

    def concat(self, value):                # value: list, Set...
        for x in value:                     # Removes duplicates
            if not x in self.data:
                self.data.append(x)

    def __len__(self): return len(self.data)               # len(s…
    def __getitem__(self, key): return self.data[key]      # self[…
    def __and__(self, other): return self.intersect(other) # self …
    def __or__(self, other): return self.union(other)      # self …
    def __repr__(self): return f'Set({self.data!r})'       # print…
    def __iter__(self): return iter(self.data)             # for x…
```

像往常一样使用：创建实例、调用方法、运行已定义的运算符：

```
$ python3
>>> from setwrapper import Set
>>> x = Set([1, 3, 5, 7, 3])
>>> x.union(Set([1, 4, 7]))
Set([1, 3, 5, 7, 4])
>>> x | Set([1, 4, 6, 4])
Set([1, 3, 5, 7, 4, 6])
```

解释器如何处理：构造时 Set.init 先把 self.data 置为空列表，再调用 concat 逐项去重——注意 init 的默认参数 value=[] 是共享的可变默认值，但这里没有直接使用它，而是把元素复制进 self.data，所以避开了著名的"可变默认参数陷阱"。and/or 等双下划线方法把运算符表达式翻译成普通方法调用；repr 用 f-string 的 !r 还原列表字面量；iter 返回底层列表的迭代器，使实例可用于 for 循环。因为重载了索引与迭代，Set 实例在许多场合能"冒充"真正的列表。

> Overloading operations such as indexing and iteration also enables instances of our Set class to often masquerade as real lists. Because you will interact with and extend this class in an exercise at the end of this chapter, we'll put this code on the back burner until its solution in Appendix B.

补充中文说明

> 重载索引与迭代等操作，还让 Set 类的实例常常可以"冒充"真正的列表。因为本章末尾的练习会让你与这个类交互并扩展它，我们暂时把这段代码搁置起来，直到附录 B 中的解答。

深度理解

·核心概念：运算符重载是让自定义对象"看起来像内置对象"的关键。只要实现 len、getitem、iter、repr 这几个协议方法，你的对象就能被 len()、索引、for、print 等通用工具接受——这就是鸭子类型（duck typing）在类层面的体现。

·底层实现：Python 的运算符表达式在字节码层面会被转译为对特殊方法（dunder methods）的查找与调用，例如 x | y 实际调用 type(x).or(x, y)。

·设计原因：为什么练习还要"扩展"这个类？因为写代码不是终点——继承、扩展、测试才是真实世界使用类的常态。

·实际问题：嵌入方式下，你要亲手转发每个操作（len、getitem……）；属性一多就会繁琐，这就是下一小节"子类化"存在的理由。

·初学者误区：误以为 repr 与 str 一样。repr 面向开发者，追求"能还原对象的表达式"，所以这里用 !r 显示列表原始形态。

代码分析（子类化方式）

Example 32-2（文件 typesubclass.py）直接子类化内置类型。类型转换函数如 list、str、dict、tuple 其实都是内置类型名，list('text') 本质上就是调用一个类型的构造器（constructor）：

```
"""
Subclass built-in list type/class.
Map 1..N to 0..N-1, call back to built-in version.
 list /
 1..N  0..N-1
"""
class MyList(list):
    def __getitem__(self, offset):
        print(f'<indexing {self} at {offset}>')
        return list.__getitem__(self, offset - 1)

if __name__ == '__main__':
    print(list('abc'))
    x = MyList('abc')          # __init__ inherited from list …
    print(x)                   # __str__/__repr__ inherited from l…
    print(x[1])                # MyList.__getitem__
    print(x[3])                # Customizes list superclass method…
    x.append('hack!'); print(x)  # Attributes from list superclass…
    x.reverse(); print(x)
```

运行结果：

```
$ python3 typesubclass.py
['a', 'b', 'c']
['a', 'b', 'c']
<indexing ['a', 'b', 'c'] at 1>
a
<indexing ['a', 'b', 'c'] at 3>
c
['a', 'b', 'c', 'hack!']
['hack!', 'c', 'b', 'a']
```

解释器如何处理：class MyList(list) 意味着 MyList 实例在 C 层面就是 list，init、append、reverse、str 全部继承。我们只重写了 getitem：把用户给的索引减 1，再回退调用 list.getitem（显式通过超类名调用，绕开 MRO）。当执行 x[1] 时，解释器在 MyList 的 dict 中找到 ge

titem，调用它；内部又显式查找 list 类的原始版本。输出里的 <indexing ...> 就是重写版本打印的追踪文本。

> This output also includes tracing text the class prints on indexing. Of course, whether changing indexing this way is a good idea, in general, is another issue—users of your MyList class may very well be confused by such a core departure from Python sequence behavior. The ability to customize built-in types this way can be a powerful asset, though.

> For instance, this coding pattern gives rise to an alternative way to code a set—as a subclass of the built-in list type rather than a standalone class that manages an embedded list object, as shown in the prior section. As discussed in Chapter 5, Python today comes with a powerful built-in set object, along with literal and comprehension syntax for making new sets. Coding one yourself, though, is still a great way to learn about type subclassing in general.

> The code in Example 32-3, file setsubclass.py, customizes lists to add just methods and operators related to set processing. Because all other behavior is inherited from the built-in list superclass, this makes for a shorter and simpler alternative—everything not defined here is routed to list directly.

补充中文说明

这个输出还包括类在索引时打印的追踪文本。当然，把索引

改成这样到底是不是个好主意，是另一个问题——你的 MyList 类的用户很可能被这种对 Python 序列行为的核心背离搞糊涂。不过，以这种方式定制内置类型的能力，可以是一笔强大的资产。

例如，这种编码模式催生了另一种实现集合的方式——作为内置 list 类型的子类，而不是前一节那种管理内嵌列表对象的独立类。正如第 5 章所讨论的，如今的 Python 自带强大的内置 set（集合）对象，以及创建新集合的字面量与推导式语法。不过，自己写一个集合，仍然是学习类型子类化的绝佳途径。

Example 32-3 中的代码（文件 setsubclass.py）定制列表，只添加与集合处理相关的方法和运算符。因为所有其他行为都继承自内置 list 超类，这是一个更短更简单的替代方案——这里未定义的一切都直接路由回 list。

代码分析

```
class Set(list):
    def __init__(self, value = []):        # Constructor
        list.__init__(self)                 # Customizes list li…
        self.concat(value)                  # Copies mutable defaul…

    def intersect(self, other):             # other is any iterable
        res = []                            # self is the subject
        for x in self:
            if x in other:                  # Pick common items
                res.append(x)
        return Set(res)                     # Return a new Set

    def union(self, other):                 # other is any iterable
        res = Set(self)                     # Copy me and my list…
        res.concat(other)
```

```
        return res

    def concat(self, value):                # value: list, Set, etc.
        for x in value:                     # Removes duplicates
            if not x in self:
                self.append(x)

    def __and__(self, other): return self.intersect(other)
    def __or__(self, other): return self.union(other)
    def __repr__(self): return f'Set({list.__repr__(self)})'

if __name__ == '__main__':
    x = Set([1, 3, 5, 7])
    y = Set([2, 1, 4, 5, 6])
    print(x, y, len(x))
    print(x.intersect(y), y.union(x))
    print(x & y, x | y)
    x.reverse(); print(x)
```

文件末尾自测代码的输出如下。因为子类化核心类型是受众有限的高级特性，这个话题到此为止，不过欢迎你在代码中追踪这些结果来研究其行为：

```
$ python3 setsubclass.py
Set([1, 3, 5, 7]) Set([2, 1, 4, 5, 6]) 4
Set([1, 5]) Set([2, 1, 4, 5, 6, 3, 7])
Set([1, 5]) Set([1, 3, 5, 7, 2, 4, 6])
Set([7, 5, 3, 1])
```

解释器如何处理：class Set(list) 让 Set 实例直接就是 list——len(x)、x.reverse()、迭代等全部继承自 list。与嵌入版的两处关键差异：list.init(self) 显式调用父类构造器，否则实例是一个空列表；union 用 Set(self) 复制（self 本身是 list，可直接转为新 Set）。repr 调用 list.repr(self) 得到 [1, 3, 5, 7]，再包一层 Set(...) 显示。

设计思想：子类化版代码更短，因为"一切未定义行为自动路由回 list"。但也有隐患——书里明确提醒：某些继承来的 list 操作可能给 Set 引入重复项（例如 reverse() 不做去重），而且用嵌套线性扫描实现集合，远不如 Python 内置的 dict 哈希查找快。作者借此再次强调：Python 自带强大的内置 set 对象，自己写 Set 主要为了学习类型子类化。

> Subtleties: some inherited list operations may introduce duplicates to our Set, and there are more efficient ways to implement sets with dictionaries in Python, which replace the nested linear search scans in the set implementations shown here with more direct dictionary index operations (hashing) and so run much quicker. If you're interested in sets, also take another look at the set object type we explored in Chapter 5; this type provides extensive set operations as built-in tools. Set implementations are fun to experiment with but not strictly required in Python today.

> More important here is the question of why we can subclass built-in types like list at all. The next section solves the mystery—at least as much as this chapter can.

补充中文说明

> 微妙之处：某些继承来的 list 操作可能会给我们的 Set 引入重复项；而且用 Python 的字典（dictionary）实现集合有更高效的方式——用更直接的字典索引操作（哈希，has

hing）替换这里展示的集合实现中的嵌套线性扫描，因此运行快得多。如果你对集合感兴趣，再回头看看我们在第 5 章探索过的 set 对象类型；该类型把丰富的集合运算作为内置工具提供。自己实现集合玩玩很有趣，但在今天的 Python 里并非必需。

这里更重要的是一个问题：我们到底为什么能子类化 list 这样的内置类型？下一节将解开这个谜团——至少在本章能解释的范围内。

深度理解

·核心概念：为什么能子类化 list？因为类型（type）与类（class）在很大程度上是同一个东西——这是"新式类"模型带来的统一。

·底层实现：list、str、float 都是类（type 类的实例）。[]、'lp6e'、3.12 这样的字面量与 list()、str()、float() 调用只是创建实例的两种语法，本质相同。

·设计原因：字面量语法是"语法糖"，让常用类型读起来更自然；而类型即类，让用户定义类与内置类型共享同一套继承、实例化机制。

·实际问题：这一统一意味着你可以给 str 加方法、给 dict 定制行为（如 Django、SQLAlchemy 等框架就是这样扩展基础类型的）。

·初学者误区：以为 list('text') 只是"转换函数"。实际上它是构造器调用——这解释了下文 type()、isinstance() 等类型测试工具的准确性为什么重要。

32.3 Python 对象模型（The Python Object Model）

英文原文

> The reason we could subclass built-in types in the prior section is that types and classes are largely one and the same—a unification that came with the "new-style" model alluded to at the start of this chapter. For built-ins, some instances can uniquely be coded with literal syntax like [],'lp6e', and 3.12 instead of class calls like list(), str(), and float(), but they are instances of a class, nonetheless.

> In fact, built-in types and user-defined classes are both classes and are both themselves instances of the built-in type class. The type object generates classes as its instances, classes generate instances of themselves, and classes are really just user-defined types. And on top of all this, the built-in object class provides defaults for every object.

中文翻译

我们能在上一节里子类化内置类型，根本原因是：类型（types）与类（classes）在很大程度上是同一个东西——这一统一，源自本章开头提到的"新式"（new-style）模型。对于内置类型，某些实例可以独特地使用字面量语法（literal syntax）来编码，比如 []、'lp6e' 和 3.12，而不必用 lis

t()、str()、float() 之类的类调用；但即便如此，它们也仍是某个类的实例。

> 事实上，内置类型（built-in types）与用户自定义类（user-defined classes）都既是"类"，又都是内置 type 类的实例。type 对象以类为它的实例、类又以自己为模板生成实例，而类其实只是"用户自定义的类型"。在这之上，内置的 object 类为每一个对象提供默认行为。

深度理解

·核心概念：Python 对象模型的核心是三条链：type 生成"类"，类生成"实例"，object 是所有对象的公共祖先。类型即类、类即类型（type is class, class is type）是本节的一句话总结。

·底层实现：type() 单参数调用会返回任意对象的类型，等价于对象的 class；isinstance() 检查的是"继承关系"而非严格相等。type 本身是 type 的实例（它没有别的类型基于它自己，形成环形顶端），type 也是 object 的子类、同时 object 又是 type 的实例——这是模型里著名的"循环"。

·设计原因：新式类模型来自 Python 2.2（2001 年左右），统一了"类型"与"类"两个原本割裂的概念，让用户类可以使用与内置类型完全一致的机制（继承、子类化、描述符等）。

·实际问题：这套模型让"为何能子类化 list"有了答案；也让类型检查代码必须准确——判断 type(1) 与 isinstance(1, int) 的差别，就要先懂"类型即类、类可继承"。

·初学者误区：以为 type(Hack) 返回的是"Hack 实例的类型"；实际上它返回 <class'type'>，因为类本身是 type 的实例。初学者还容易混淆"实例的类"与"类的类"（元类，metaclass）这两个概念。

代码分析（用户自定义类）

代码里的 type(I) 返回的是"生成 I 的类"；type(Hack) 返回的是"生成 Hack 这个类的东西"——即 type 本身：

```
>>> class Hack: pass                        #
>>> I = Hack()                              #
>>> type(I)                                 #
<class '__main__.Hack'>
>>> type(Hack)                              #  type
<class 'type'>
>>> I.__class__, Hack.__class__             # __class__  type() …
(<class '__main__.Hack'>, <class 'type'>)
>>> isinstance(I, object), isinstance(Hack, object)
(True, True)
```

解释器如何处理：执行 class Hack: pass 时，CPython 会创建一个名为 Hack 的类对象，其 class 指向 type。create() 这个动作在语义上等价于调用 type('Hack', (), {})——于是"类"就是 type 的一个实例。I = Hack() 则是调用 Hack（一个可调用对象），Hack.call 由 type 的机制提供，分配内存并调用 object.new 返回实例，实例的 class 指向 Hack。isinstance 沿继承链（MRO）搜索，I → Hack → object，所以两类判断都是 True。

代码分析（内置类型同理，但多了一种字面量写法）

```
>>> I = 'hack'                              #  str()
>>> type(I)                                 #
<class 'str'>
>>> type(str)                               #  type
<class 'type'>
>>> I.__class__, str.__class__
(<class 'str'>, <class 'type'>)
>>> isinstance(I, object), isinstance(str, object)
(True, True)
>>> type(type)                              # type
<class 'type'>
>>> type(type(type))                        #  type
<class 'type'>
```

解释器如何处理：'hack' 字面量在字节码层面等价于 str('hack') 调用；str 本身是一个"type 实例"。解释器在编译时对字符串字面量直接创建 str 对象，跳过显式调用，但语义完全相同：字面量是创建实例的"语法糖"。type(type) 返回 <class'type'> 说明 type 的元类就是它自己——这是整个对象模型的"终点站"，防止无限递归。

> This model may seem academic (and to some extent is), but it allows us to specialize built-in types with normal user-defined classes and bears on type-testing code: you must know what a type is, to test it accurately.

补充中文说明

这套模型看似学术（某种程度上也确是如此），但它让我们能够用普通的用户自定义类来定制专用内置类型，并且它关系到类型检查（type-testing）代码：要准确地对一个东西

做类型测试，你就必须先搞清楚"类型到底是什么"。

32.4 有些实例比其他实例更"平等"（Some Instances Are More Equal Than Others）

英文原文

It's tempting to simply take away from the foregoing that classes and types are the same, but this story is richer than that may imply: the instances we make from these objects diverge in both functionality and inheritance.

To truly understand how, we have to briefly factor in metaclasses—classes that generate other classes. The type built-in itself is a metaclass and may be customized with user-defined subclasses. These subclasses are coded with normal class statements, selected with special syntax in class headers, and designed to play metaclass roles, but they won't be covered in full until Chapter 40.

In brief, though, the complete relationship between instances, classes, and types is as follows:

Instances are created from classes—both built-in and user-defined. Classes themselves are created from the built-in type class, or one of its subclasses. The object built-in class is a superclass to every object—instance, class, or both.

Although everything is ultimately an "instance" in Pyt
hon, there are two fundamentally different kinds of i
nstances, and conflating these only serves to mask th
e true complexity of the model. It doesn't help to dist
inguish these as instances of user-defined classes or
not: metaclasses may be user-defined classes too. No
r is this about being a subclass of a type: the real fork
in this model is that classes are created from a type c
lass specially.

As you'll learn in full later, classes define their types with optional metaclass syntax that defaults to type if omitted, but other instances define their types by the class calls or literal syntax we've used so far:

```
class C(metaclass=Meta): ...   # Class creation: metaclass defa…
I = C(...)                     # Instance creation: user-defined c…
X = [1, 2]                     # Instance creation: built-in class…
```

While both produce instances in some sense, these di
fferent syntaxes create very different kinds of instanc
es—class and nonclass—with fundamentally different
behaviors:

Nonclass instances do not make instances. Classes ar
e created from type (or another metaclass), similar to
the way instances are created from classes. Once crea
ted, though, the analogy fails: classes create instance
s of their own, but nonclass instances do not.

Classes have an extra inheritance search. There are re
ally two inheritance trees and searches in Python, whi

ch are distinct but not entirely disjoint. The secondary tree is formed by type and its subclasses and is searched only for classes, not nonclass instances.

In other words, nonclass instances seal off the instantiation chain, and inheritance differs for nonclass instances and classes themselves—even though the latter are also instances of type. All of this boils down to different creation syntax that makes different kinds of objects, which are often confusingly lumped together as "instances":

```
>>> class C: pass                          # A type instance…
>>> I = C()                                # A nonclass instance…
>>> isinstance(I, type), isinstance(C, type)   # Only classes are …
(False, True)
```

While types and classes may be synonymous, the instances we create from them vary per creation code.

中文翻译

人们很容易简单地认为"类即类型"而止步于此，但这则故事比这个结论丰富得多：我们从这些对象生成的实例，在功能性与继承性两个维度上都分道扬镳。

而要真正理解其中的机理，我们必须简短地引入 metaclasses（元类）——"生成类的类"。type 内置对象本身就是一个元类，并且可以用用户自定义的子类将其定制。这些子类用普通的 class 语句编写、在 class 头部用特殊语法选定、并被设计去扮演元类的角色，但它们的完整面貌要到第 40 章才会呈现。

> 简而言之，实例（instances）、类（classes）与类型（types）的完整关系如下：

> 实例由类创建——无论内置类还是用户自定义类。类本身由内置的 type 类、或它的某个子类创建。内置的 object 类是所有对象——实例对象、类对象、或两者兼具——的共同超类。

> 尽管 Python 中"一切归根结底都是实例"，但存在两种根本不同的实例，而把它们混为一谈，只会掩盖这套模型的真正复杂度。用"是否用户自定义类的实例"来区分毫无帮助：元类本身也可能是用户自定义类；这也不是"是否为 type 的子类"的问题——这套模型的真正分岔点是：类是专门从某个 type 类创建出来的。

> 稍后你会完整学到：类通过可选的 metaclass 语法定义自己的类型，缺省时默认为 type；而其他实例则通过我们一直使用的类调用或字面量语法定义自己的类型：

```
class C(metaclass=Meta): ...   #  type
I = C(...)                     #
X = [1, 2]                     #
```

> 虽然从某种意义上说两者都"产出实例"，但这两套不同的语法创建的是根本不同的两种实例——class（类）与 nonclass（非类）——它们的行为有着本质差异：

> 非类实例不能再生成实例。类从 type（或其他元类）创建，就像实例从类创建一样；但一旦创建完成，这个类比就失效了：类能再创建属于自己的实例，而非类实例不能。

类还有一棵额外的继承树要搜。Python 里其实有两棵继承树、两套搜索，它们彼此独立但又并非完全不相交。第二棵树由 type 及其子类构成，只对类搜索，不对非类实例搜索

。

> 换句话说，非类实例封死了"实例化链"；继承对非类实例与对"类本身"（尽管后者也是 type 的实例）的运作方式是不同的。这一切最终归结为：不同的创建语法制造出不同种类的对象，而它们往往被笼统地塞进"实例"这一个词里：

```
>>> class C: pass                          #
>>> I = C()                                #
>>> isinstance(I, type), isinstance(C, type)   # "" type
(False, True)
```

> 尽管"类型"与"类"可以同义，但我们从它们创建出来的实例，却因创建代码的不同而各异。

深度理解

·核心概念：虽然"type/class 统一"是主流论调，但模型在"谁能创建谁"上分岔：类本身也是实例（type 的实例），但它是"能继续生成实例的实例"；普通对象则"到此为止、只做被操作的主体"。

·底层实现：type 是 type 的实例（自指封顶）；object 是 type 的实例；type 又是 object 的子类。这段循环关系诚然让人眩晕，但如果你把它想成类型系统的一个"不动点"，就豁然开朗了。

·设计原因：统一类/类型（一元模型）为 Python 带来了强一致性，同时"付出"了复杂度：必须在"实例"这个词之下区分"类实例"与"普通对象"，否则 isinstance、继承搜索都会乱套。

·实际问题：理解不了这个分岔，你会在 isinstance(I, type) 及 bases 是否可被实例继承的问题上吃亏——本小节

末尾的代码会证明：C.bases 合法、I.bases 抛错。

·初学者误区：认为"实例的 class 与元类是一回事"。I.class 是"实例的类（class）"，而 C.class 则是"类的类（type/metaclass）"——这是两棵不同的树。

代码分析

```
>>> class C: pass                  # a type instance
>>> I = C()                       # a nonclass instance
>>> isinstance(I, type), isinstance(C, type)    # "" type
(False, True)
```

解释器如何处理：isinstance(I, type) 为 False，因为 I 的 MRO 是 C -> object，其中没有 type；而 C 的 MRO 是 type -> object——注意这里的 MRO 走的是"元类链"（secondary tree）。于是"类"与"普通实例"在类型空间里彻底分开。也正因如此，后面我们看到 C.bases 成立、I.bases 报 AttributeError。

继承的两叉（The Inheritance Bifurcation）

英文原文

> While we can't get into full details here, the inheritance search used for classes and types differs from what we've seen so far. In short, inheritance runs the MRO (method resolution order) we studied before, but varies as follows, for these cases:
>
> Nonclass inheritance—As we've seen, inheritance on a nonclass instance searches the dict attributes of instance, class, and superclasses, per the MRO order we

studied in the prior chapter. This works by first checking the instance, then following the instance's class to its class, and finally following each class's bases to superclasses. Class inheritance—Inheritance run on a class directly, though, first searches the dict of the class and all its superclasses, then notices the separate class tree formed by the type class and its metaclass subclasses. The second part of this works by following the class's own class to its type class tree, using bases and MROs there—but only as a last resort.

中文翻译

虽然我们不能在此详述，但用于"类（types）"的继承查找与至今所见（普通实例的继承查找）是不同的。简言之，继承始终基于我们在前面学过的 MRO（method resolution order，方法解析顺序），但分两种情况：

非类实例（Nonclass instance）的继承——如我们所见的，对非类实例执行的继承，先查实例自身的 dict，然后按照我们上章学到的 MRO 顺序，一路查实例的类及其各超类。它的实现方式是：先检查实例，接着沿实例的 class 到它的类，再沿各类的 bases 到达超类。

类（Class）的继承——直接对"类"本身执行的继承，同样先按通常方式在类的 dict 及所有超类中搜索，但它还会额外搜索由 type 类及它的元类子类构成的另一棵树。第二部分通过在 class 之后沿类自己的 class 进入 type 类树、再用 bases 与 MRO 来完成——但这仅作为最后手段、且只对类的继承生效。

深度理解

·核心概念：Python 有两棵继承树：主继承树（class → superclasses，供普通实例查属）与"type/metaclass 树"（供类对象自身查属）。非类实例把树一的搜索彻底封住；类则把两棵树都扫一遍。

·底层实现：每个类的 class 就是它的"type"，type 树里的对象也会构成 mro。所以对"类"的属性查找，最终候选集是 [x.dict for x in C.mro] + [x.dict for x in C.class.mro] 两个扁平树的依次叠加。

·设计原因：两棵树让"类模型自身的元属性"（如 bases）能放在 type 树中，避免污染普通实例。

·实际问题：涉及其实现细节——C.bases 可用而 I.bases 不可用，恰好是因为 bases 躺在"第二棵树"里，只有对类搜索时才经过。

·初学者误区：以为 dir() 就是个"所有属性一览"，而不了解名字可能来自两条独立的继承链。

代码分析

```
>>> isinstance(I, C), type(I)
(True, <class '__main__.C'>)
>>> I, I.__class__, I.__class__.__bases__
(<__main__.C object at 0x101265d60>, <class '__main__.C'>, (<class…
>>> C, C.__bases__, C.__class__, C.__class__.__bases__
(<class '__main__.C'>, (<class 'object'>,), <class 'type'>, (<clas…
```

再看两个更精确的 MRO 视图——每个 mro 都是一棵"压扁的树"：

```
>>> I, I.__class__.__mro__
(<__main__.C object at 0x101265d60>, (<class '__main__.C'>, <class…
>>> C.__mro__, C.__class__.__mro__
((<class '__main__.C'>, <class 'object'>), (<class 'type'>, <class…
```

于是非实例与"类"各自属性查找的有序候选集，分别就是这个样子（两棵树对普通实例只扫描头一段、对"类"则扫两段）：

```
[I.__dict__] + [x.__dict__ for x in I.__class__.__mro__]
[x.__dict__ for x in C.__mro__] + [x.__dict__ for x in C.__class__…
```

解释器如何处理：I.class 是实例的类，I.class.bases 是该类的直接超类（单个 object）；而 C.class 是元类 type，type.bases 正好也是 (object,)。所以 C.mro 与 C.class.mro 结构平行但隶属两棵树。之前提到"bases 不能被实例继承"，用两棵树立刻解释得通：

```
>>> I.__bases__
AttributeError: 'C' object has no attribute '__bases__'. Did you m…
>>> C.__bases__
(<class 'object'>,)
```

元类/类的二分（The Metaclass/Class Dichotomy）

英文原文

> So, where does this odd tale of type/class unity leave us? Types do behave as classes, and yes, we can use plain class syntax to extend them in both the primary and metaclass trees. But it also comes with noticeable seams, including special-case syntax for class instan

tiation, an extra type-tree search for classes, two very different kinds of instances, and unique semantics for metaclasses that customize types.

> In fact, a reasonable argument can be made that the type/class dichotomy of earlier Pythons may simply have morphed into one of metaclass/class—which trades a straightforward distinction for all the seams just enumerated, and muddles inheritance and the very meaning of names in Python, to support what in the end is a very rare use case. As usual, the net merit of the morph is yours to weigh.
>
> To be fair, some of the widespread confusion this model has spawned may stem from type itself: it's overloaded to either return the type of a sole argument, or generate a new instance of itself for multiple arguments—just like other constructors, and like what a class statement does to make a class object:

```
type(object)              # Fetch object's type
type(classname, superclasses, attributedict)   # Make a class/type…
```

中文翻译

> 那么，这则关于 type/class 统一的奇特故事，把我们带到了何处？类型（type）确实表现得像类，这也允许我们用普通的类语法，在"主树"与"元类树"两处都扩展它们。但这也带来了显而易见的"接缝"（seams）：类实例化有特殊的语法；类独享一棵额外的 type 树搜索；存在两种根本不同的实例；以及定制类（即元类）独特的语义——这些我们稍后会揭开。

> 事实上我们完全可以争辩说：早期 Python 的"type/class"二分，不过是演变成了"metaclass/class"的二分罢了——把一个本来干脆利落的区分，换取上面罗列的种种"接缝"，并把继承与 Python 中名字（names，指属性名）的根本含义搅浑。这笔烂账，为的是支持一个极其罕见的使用场景。而这套变形究竟净值几何，依旧留给你自己去掂量。

> 再举一例：这套模型引发的许多困惑，可能都源于 type 本身——它被"重载"了：用一个参数调用时返回该参数的类型；用多个参数调用时则像其他构造器一样生成自己的一个新实例，正如 class 语句生成一个类对象那样：

> 第一个角色也许命名为 typeof 更贴切；而第二个角色，则要留待本章稍后的元类预览和第 40 章扩展的元类专题来展开。

```
type(object)                          #
type(classname, superclasses, attributedict)   # /type
```

深度理解

·核心概念："type/class"二分变形为"metaclass/class"二分——这是本章作者的主观评价，也为第 40 章讲元类做伏笔。

·底层实现：type(name, bases, dict) 等价于一次 class 语句；而 class C(metaclass=Meta) 则改走 Meta(...) 生成类。

·设计原因：向一个机制喂进一个动态参数就能复用 type 自己的实现，是"类即可调用对象"设计哲学的延伸。

·实际问题：这解释了 Python 中反复出现的同名工具"一

式两用"（如 type() 既可查形又可造型、int() 既可转换又可创建实例）为什么让新手困惑——官方文档也专门标注过这一坑。

·初学者误区：以为 type(x) 与 type("C", (object,), {}) 是同一个东西。前者是"取类型"，后者是"造类型"，同名不同功。

且凭一"object"号令天下（And One "object" to Rule Them All）

英文原文

> To round out this topic, keep in mind that because to pmost classes inherit from the built-in class object, every object derives (i.e., inherits) from it, whether directly or through a superclass—and whether you code object or not:

```
> >> class C: pass >> class D(object): pass >> dir(C) == dir(D)

True
> >> C.__bases__, D.__bases__

((<class 'object'>,), (<class 'object'>,))
```

> In fact, the type class inherits from the object class, and object inherits from type, even though the two are different objects—a circular relationship that crowns the object model and may make your cranium catch fire (to avoid combustion, keep in mind that isinstance is true for either a subclass relationship or creati

on source, and that this is based on inheritance throu gh the secondary type-class tree for classes):

```
> >> type is object

False
> >> type(type), type(object)

(<class 'type'>, <class 'type'>)
> >> isinstance(type, object), isinstance(object, type)

(True, True)
> >> type.__bases__, object.__bases__

((<class 'object'>,), ())
```

中文翻译

为了给这个话题收个尾，请记住：因为最顶层的类都继承自内置类 object，所以每一个对象——无论通过直接继承还是经由某个超类——都派生自（即继承自）它，无论你是否显式写上 object：

事实上，type 类继承自 object 类，而 object 又继承自 type——尽管它们是两个不同的对象；这个循环关系给对象模型戴上了皇冠，也可能让你的脑瓜烧起来（为避免"燃烧"，请记住：isinstance 对"子类关系"或"创建来源"任一成立即返回真，而对类来说这与经由第二棵 type-class 树的继承有关）：

代码分析（环形的实际效果）

```
>>> class C: pass
>>> class D(object): pass
>>> dir(C) == dir(D)              #  object
True
>>> type is object                #
False
>>> type(object)                  # object  type
<class 'type'>
>>> isinstance(object, type)      #  object  type /
True
```

Strange though it may seem, this has a number of pr actical consequences. For one thing, it means that we sometimes must be aware of the method defaults tha t come with the implicit (or explicit) object root class. As we noted in earlier chapters, for instance, the obje ct class comes with a repr for display:

```
>>> class C: pass
>>> X = C()
>>> X.__repr__
<method-wrapper '__repr__' of C object at 0x1091a7920>
```

For another, this also allows code to be written that c an safely assume and use an object superclass. As an example, we can rely on it to be a call-chain "anchor" in some super built-in roles described ahead and can reroute method calls to it from attribute-interception methods to invoke higher default behavior. Per Chap ter 30:

```
object.__setattr__(self, attr, value)
```

We'll code examples of such rerouting later in the bo ok; for now, let's move on to something a bit more ta

ngible and our next topic in this OOP jamboree.

补充中文说明

另一方面，这也允许我们写出那种"放心假设存在一个 object 超类"的代码。举例来说，我们可以依赖它充当某些 super 内建角色里所谓"调用链之锚"（call-chain anchor，详见后文），并可以让属性拦截方法（attribute-interception methods）把方法调用重路由到它那里，从而触发更高层的默认行为。如第 30 章所述：

```
object.__setattr__(self, attr, value)
```

本书后面会给这类重路由写出示例；眼下，让我们转向更实在的东西——这场 OOP 狂欢的下一个主题。

深度理解

·核心概念：object 是所有对象的默认祖先：type.bases == (object,)，而 object 又是由 type "创建"的实例——二者互为循环，构成对象模型的顶点。

·底层实现：type.bases 是 (object,)（type 通过继承链排在 object 之下）；而 object.bases 是 ()（类型树里最底层的根，没有祖先）。

·设计原因：一个统一的根 object 提供全 Python 共用的默认方法（repr、eq、setattr、reduce 等），相当于给所有对象配齐了"出厂设置"。

·实际问题：super() 在构造器调用链一端找不到"锚"（anchor）时，常常要借 object.init / object.setattr 作为兜底；例如属性拦截方法里要绕开递归，就得显式调用 obje

ct.setattr(self, ...)。

·初学者误区：以为 type 和 object 是同一个东西。其实一个负责"生成类"（type），一个是"最顶层基类"（object），分别站在继承链的两端——type is object 返回 False 正说明了这一点。

32.5 高级属性工具（Advanced Attribute Tools）

英文原文

> Along with the normal class and instance attributes we've been using so far, Python's OOP support includes attribute tools of narrower scope—slots, properties, descriptors, and more. Slots, for example, are an optimization option, and properties and descriptors allow classes to augment access. None of these tools are required, but as for most topics in this chapter, all are fair game in Python code you may someday use. Most of these tools get extended coverage in Chapter 38, but slots get full coverage here, and others are presented in abbreviated form.

中文翻译

除了我们迄今一直在用的普通类属性与实例属性之外，Python 的 OOP 支持还包括一批适用范围更窄的属性工具——slots（插槽）、properties（属性）、descriptors（描

述符）等等。例如，slots 是一个优化选项；properties 与 descriptors 则允许类对属性访问进行增强。这些工具都不是必需的，但正如本章多数主题一样，它们都可能出现在你将来要读的 Python 代码里。这些工具大部分会在第 38 章得到扩展讲解，但 slots 在本章获得完整覆盖，其余工具只做简略介绍。

深度理解

·核心概念：普通属性是"存进 dict 字典"；而本节工具都试图在"存储"与"访问"上做文章——slots 换存储方式，properties/descriptors 拦截访问。

·底层实现：slots 与 properties 都建立在描述符协议（get/set）之上，只是描述符本身要到第 38 章才展开。

·设计原因：Python 尊重"显式优于隐式"，于是提供多个粒度不同的属性机制，让使用者按需选择。

·实际问题：读框架代码时经常碰到 slots 和 @property；不知道它们，就会误以为"属性丢了"或"有魔法"。

·初学者误区：以为 slots 能"提升性能"就是默认推荐；作者反复强调它们只适合内存敏感、实例数量极大的特殊场景。

Slots: 属性声明（Slots: Attribute Declarations）

英文原文

First off, we've noted the implications of slots several times in this part of the book. In short, by assigning an iterable of attribute name strings to a special slots

class attribute, we can enable a class to both limit the set of legal attributes that instances of the class will have and optimize memory usage and possibly program speed. As you'll find, though, slots should be used only in applications that clearly warrant the added complexity. They will complicate your code, may complicate or break code you may use, and rigidly require universal deployment to be effective.

中文翻译

先说开场白：本书这一部分已经多次提过 slots 的影响。简言之，把一个"属性名字符串组成的可迭代对象"赋给特殊的 slots 类属性，就能让一个类做到两件事：限制该类的实例所能拥有的合法属性集合，并优化内存使用（或许还有程序速度）。但你也会发现，slots 只应在"确实值得承受这份额外复杂度"的应用里使用——它们会让你的代码变复杂、可能搞乱甚至搞坏你使用的代码，并且严格要求全树普及（universal deployment）才有效。

深度理解

·核心概念：slots 是"白名单"式的属性声明——不在名单里的实例属性名一律不允许创建。

·底层实现：解释器不再为每个实例分配 dict 字典，而是让类为每个 slot 名创建描述符，并在实例中预留固定内存块（slot 数组）存放值。

·设计原因：大量实例 + 少量属性时，每实例一个字典的内存开销可观；slots 用"类级描述符 + 实例级定长数组"

换取更紧凑的布局（类似 C 语言 struct 的固定字段）。

·实际问题：因为"不用 dict"与 Python 动态本性冲突，slots 会波及所有"通用属性访问"的代码——如 vars()、序列化、调试工具。

·初学者误区：以为 slots 能加速所有程序。作者给出的实测：CPython 3.12 下速度提升微乎其微（见后面的 slots-test.py），主要收益只在内存。

代码分析（Slot 基础）

```
>>> class Limiter(object):
...     __slots__ = ['age', 'name', 'job']     # ""
>>> I = Limiter()
>>> I.age                                     #
AttributeError: 'Limiter' object has no attribute 'age'
>>> I.age = 40                                #
>>> I.age
40
>>> I.ape = 1000                              #  __slots__
AttributeError: 'Limiter' object has no attribute 'ape'
```

解释器如何处理：class Limiter 执行时，解释器见到 slots，就不再为实例准备 dict，而是把 age/name/job 变成类上的槽描述符（member descriptor）。I.age = 40 被描述符的 set 捕获，直接写进实例内部分配的槽位。I.ape = 1000 时，setattr 在类及其 MRO 上找不到名为 ape 的描述符、也没有 dict 可扩展，于是抛出 AttributeError——"拼写错误被立刻抓住"，这正是 slots 号称能"捉虫"的原因。

> This feature is advertised as both a way to catch typo errors like this (assignments to illegal attribute name

s not in slots are detected instead of silently assigned), as well as an optimization mechanism that saves m emory. Allocating a namespace dictionary for every i nstance object can be expensive in terms of memory if many instances are created and only a few attribut es are required. To save space, instead of allocating a dictionary for each instance, Python reserves just eno ugh space in each instance to hold a value for each sl ot attribute, along with inherited attributes in the co mmon class to manage slot access. This might additi onally speed execution, though this benefit may vary per program, platform, and Python version (spoiler: t he speedup is trivial today, as we'll prove ahead).

补充中文说明

这个特性号称兼具两个用途：一是像这样抓拼写错误（对不在 slots 里的非法属性名的赋值会被检测出来，而不是静默地创建）；二是一种省内存的优化机制。如果创建了大量实例而每个实例只需少数几个属性，那么为每个实例对象都分配一个命名空间字典，内存开销可观。为节省空间，Python 不为每个实例分配字典，而是只给每个实例预留"恰好能放下每个 slot 属性值"的空间，同时把管理 slot 访问所需的东西放在所有实例共享的类上。这或许还能加快执行速度，不过该收益因程序、平台、Python 版本而异（剧透：在今天的 CPython 上提速微乎其微，后面我们会有实测数据）。

你通常不该用 slots（You shouldn't normally use slots)

英文原文

> Slots are a fairly major break with Python's core dynamic nature, which dictates that any name may be created by assignment (and frankly, tend to appeal most to people with backgrounds in draconian languages). In fact, they partly imitate C++ for efficiency at the expense of flexibility and even have the potential to break some programs. As you'll see, slots also come with a plethora of special-case usage rules. Per Python's own manual, they should not be used except in clearly warranted cases—they are difficult to deploy correctly, complicate your code badly, and are best limited to very rare memory-critical programs that produce an extremely large numbers of instances. In other words, this is yet another feature that should be used only if clearly justified. Unfortunately, slots seem to be showing up in Python code much more often than they should; their obscurity seems to be a draw in itself. Slots are actually used by Python, unlike type hinting, their declaration cousin of Chapter 6, but they are similarly paradoxical and restrictive. As usual, knowledge is your best ally in such things, so let's take a deeper look here.

中文翻译

> slots 对 Python 的核心动态本性是一次相当大的背离——Python 的本性是"任何名字都可以通过赋值随时创建"（坦白讲，slots 最吸引那些出身于"严苛语言"背景的人）。事实上，它部分地是在牺牲灵活性换取类似 C++ 的效率，甚至有可能弄坏一些程序。如你所见，slots 还附带一大堆"特例用法规则"。按照 Python 官方手册的说法，除非有明确的正当理由，否则不应该使用它——slots 难以正确部署、会把你的代码弄得很复杂，最好仅限于极少数"内存至关重要、实例数量极其庞大"的程序。换句话说，这又是一个"只有被明确证明合理才使用"的特性。遗憾的是，slots 出现在 Python 代码里的频率似乎远超应有的程度——它的"神秘感"本身就是一种吸引力。与第 6 章的"声明同类"类型标注（type hinting）不同，slots 是真的会被 Python 使用到的，但两者同样自相矛盾、同样束缚人。照例，知识是你在这种事情上最好的盟友，所以让我们深入看看。

深度理解

·核心概念：作者对 slots 的态度很明确——非必要不用；"神秘感"不是使用理由。

·底层实现：slots 让类在 C 层面采用固定布局（类似于 PyPy 或 C 语言的 struct），但代价是牺牲 dict 的任意扩展性。

·设计原因：Python 手册都劝退，原因在于"全有或全无"：一棵继承树上任何一处不用 slots，就立刻退回字典存储。

·实际问题：作者观察到 slots 在真实代码中被过度使用，提醒读者保持警惕、以手册为准。

·初学者误区：把 slots 当成"性能最佳实践"；实则它更像"内存危机的最后手段"，且伴随一堆限制规则。

Slots 与命名空间字典（Slots and namespace dictionaries)

英文原文

Potential benefits aside, slots can complicate a class model—and code that relies on it—substantially. In fact, some instances with slots may not have a dict attribute namespace dictionary at all, and others will have data attributes that this dictionary does not include. To be clear, this is a major incompatibility with the traditional class model—one that can impact any code that accesses attributes generically and may even cause some to fail altogether. For instance, programs that list or access instance attributes by name string may need to use more storage-neutral interfaces than dict if slots may be used. Because an instance's data may include class-level names such as slots—either in addition to or instead of namespace dictionary storage—both attribute sources may need to be queried for completeness, and some roles may be rendered impossible. Let's see what this means in terms of code and explore more about slots along the way. First off, when slots are used, instances do not normally hav

e an attribute dictionary—instead, Python uses the cl ass descriptors feature introduced ahead to allocate and manage space reserved for slot attributes in the i nstance:

```
>>> class C:
...     __slots__ = ['a', 'b']        # __slots__  __dict__
>>> I = C()
>>> I.a = 1
>>> I.a
1
>>> I.__dict__                       #  __dict__
AttributeError: 'C' object has no attribute '__dict__'. Did you me…
```

However, we can still fetch and set slot-based attribu tes by name string using storage-neutral tools such a s getattr and setattr (which look beyond the instance dict and thus include class-level names like slots) and list them with dir (which collects all inherited names of any kind throughout a class tree):

```
>>> getattr(I, 'a')
1
>>> setattr(I, 'b', 2)               #  getattr()/setattr()
>>> I.b
2
>>> 'a' in dir(I)                    # dir()  slot
True
>>> 'b' in dir(I)                    #  __dict__
True
>>> I.__dict__
AttributeError: 'C' object has no attribute '__dict__'. Did you me…
```

Also keep in mind that without an attribute namespa ce dictionary, it's not possible to assign new names t

o instances that are not names in the slots list:

```
>>> class D:
...     __slots__ = ['a', 'b']
...     def __init__(self):
...         self.d = 4                #  __dict__
>>> I = D()
AttributeError: 'D' object has no attribute 'd'
```

We can still accommodate extra attributes, though, b y including dict explicitly in slots in order to create a n attribute namespace dictionary in addition to slots:

```
>>> class D:
...     __slots__ = ['a', 'b', '__dict__']    #  __dict__ …
...     c = 3                                  #
...     def __init__(self):
...         self.d = 4                         # d  __dict__a …
>>> I = D()
>>> I.d
4
>>> I.c
3
>>> I.a                                       #
AttributeError: 'D' object has no attribute 'a'
>>> I.a = 1
>>> I.b = 2
```

In this case, both storage mechanisms are used. This renders dict too limited for code that wishes to treat slots as instance data, but generic tools such as getat tr still allow us to process both storage forms as a sin gle set of attributes:

```
>>> I.__dict__                          #  __dict__  sl…
{'d': 4}
>>> I.__slots__
['a', 'b', '__dict__']
>>> getattr(I, 'a'), getattr(I, 'c'), getattr(I, 'd')   #
(1, 3, 4)
```

> Because dir also returns all inherited attributes, though, it might be too broad in some contexts; it also includes class-level methods and even all object defaults. Code that wishes to list just instance attributes may, in principle, still need to allow for both storage forms explicitly. We might at first naively code this as follows:

```
>>> for attr in list(I.__dict__) + I.__slots__:
...     print(attr, '=>', getattr(I, attr))    # ……
```

> Since either can be omitted, we may more correctly code this as follows, using getattr to allow for defaults—a noble but nonetheless inaccurate approach, as the next section will explain:

```
>>> for attr in list(getattr(I, '__dict__', [])) + getattr(I, '…
...     print(attr, '=>', getattr(I, attr))
d => 4
a => 1                          # ……
b => 2
__dict__ => {'d': 4}
```

中文翻译

好处先放一边，slots 可能把类模型——以及依赖它的代码——大大复杂化。事实上，某些带 slots 的实例可能根本没有 dict 属性命名空间字典，另一些实例则拥有这个字典装

不下的数据属性。说清楚些：这是与传统类模型的重大不兼容——凡是"通用地"访问属性的代码都会受影响，有些甚至直接崩溃。例如，凡是以名字字符串列出或访问实例属性的程序，如果 slots 可能被使用，就需要改用比 dict 更"存储无关"（storage-neutral）的接口。因为一个实例的数据可能既包括像 slots 这样的类级名字（这些名字可能与命名空间字典存储并存、也可能取而代之），两处来源都得查询才算完整，有些功能甚至完全做不到了。让我们用代码看看这意味着什么，顺便再探探 slots 的细节。首先，一旦使用 slots，实例通常不再有属性字典——取而代之的是，Python 利用稍后介绍的类描述符（descriptors）特性，在实例中分配并管理预留给 slot 属性的空间：

不过，我们仍可以借助 getattr、setattr 这类"存储无关"的工具，用名字字符串读取、写入基于 slot 的属性（它们会越过实例 dict 去找类级名字，因此 slots 也在其中）；也可以用 dir 把它们列出来（dir 会收集整棵类树里各种继承来的名字）：

还要记住：没有属性命名空间字典时，就不可能给实例赋值"不在 slots 名单里"的新名字：

不过我们仍能容纳额外属性——只要把 dict 显式写进 slots，就能在 slots 之外再创建一个属性命名空间字典：

在这个例子里，两种存储机制并存。于是，对于那些想"把 slots 当实例数据对待"的代码来说，单看 dict 已经不够全面；但 getattr 这类通用工具仍能把两种存储形式当作同一组属性来统一处理：

不过，dir 还会返回所有继承来的属性，在某些场合可能过宽——它连类级方法乃至 object 的全部默认属性都包含进

来了。凡是想"只列实例属性"的代码，原则上仍须显式地同时考虑两种存储形式。我们起初可能天真地这样写：

由于两者都可能缺失，我们或许会用 getattr 提供默认值、写出下面这种"更正确"的版本——这是种体面却不精确的做法，下一节会解释原因：

深度理解

·核心概念：slots 与 dict 是互斥又互补的两套存储：默认互斥（有 slots 无字典），显式声明'dict' 后并存。

·底层实现：slot 存储是类描述符 + 实例内定长缓冲（C 层面）；getattr/setattr 按"描述符优先"的完整属性协议查找，所以天然兼容两种存储。

·设计原因：dir 的"广撒网"与 dict 的"单点存储"都无法单独覆盖 slots 场景，是工具代码（如属性列示器）必须直面的事实。

·实际问题：任何"按名字串枚举属性"的通用工具（序列化、日志、测试框架）遇到 slots 都可能漏数据或抛异常。

·初学者误区：以为 vars(I) 或 I.dict 能看全实例数据；在有 slots 的实例上它们要么报错、要么只显示字典部分。

超类中多个 slots 列表（Multiple slot lists in superclasses)

英文原文

The preceding code works in this specific case, but in general, it's not entirely accurate. Specifically, this code addresses only slot names in the lowest slots attri

bute inherited by an instance, but slot lists may appe ar more than once in a class tree. That is, a name's ab sence in the lowest slots list does not preclude its exi stence in a higher slots. Because slot names become class-level attributes, instances acquire the union of a ll slot names anywhere in the tree by the normal inhe ritance rule:

```
>>> class E:
...     __slots__ = ['c', 'd']          #  slots
>>> class D(E):
...     __slots__ = ['a', '__dict__']   #  slots
>>> I = D()                             #
>>> dir(I)
[…names omitted…, 'a', 'c', 'd']
>>> I.a = 1; I.b = 2; I.c = 3           # slots: ac__dict__: b
>>> I.a, I.c
(1, 3)
```

But inspecting just the inherited slots list won't pick up slots defined higher in a class tree:

```
>>> E.__slots__                         # __slots__
['c', 'd']
>>> D.__slots__
['a', '__dict__']
>>> I.__slots__                         # "" __slots__
['a', '__dict__']
>>> I.__dict__                          #
{'b': 2}
>>> for attr in list(getattr(I, '__dict__', [])) + getattr(I, '__s…
...     print(attr, '=>', getattr(I, attr))
b => 2                    #  slots
a => 1
__dict__ => {'b': 2}
>>> dir(I)                              #  dir()  slot
```

```
[…names omitted…, 'a', 'b', 'c', 'd']
```

> In other words, in terms of listing instance attributes generically, one slots isn't always enough—they are potentially subject to the full inheritance search procedure. If multiple classes in a class tree may have their own slots attributes, tools must develop other policies for listing attributes—as the next section explains.

中文翻译

> 前面的代码在这个具体例子里是能用的，但一般来说，它并不完全准确。具体来说，那段代码只处理了实例继承到的最低层 slots 里的名字；然而在类树里，slots 列表可能出现不止一次。也就是说，"某个名字不在最低层 slots 里"并不能排除"它在更高层 slots 里"的可能。因为 slot 名字会变成类级属性，实例按普通继承规则获得整棵树里所有 slot 名字的并集：

> 但如果只检查继承来的 slots 列表，就会漏掉定义在类树更高处的 slots：

> 换句话说，就"通用地枚举实例属性"而言，一个 slots 常常不够——它们本身也可能要经历完整的继承搜索过程。如果类树里可能有多个类各自定义 slots，工具就必须另立"列出属性"的规则——这正是下一节的内容。

深度理解

·核心概念：slots 不拼接、只继承最低层的；但生效的 slot 集合是整棵树并集。这两件事的分裂正是坑之所在。

·底层实现：每个类的 slots 在类创建时被转换成各自的槽

描述符；实例的空间布局则综合了整条 MRO 上的所有描述符（CPython 在类型创建时用 typenew 合并）。

·设计原因：让超类与子类可以各自独立声明插槽（类似 C++ 派生类加字段），但代价是"查名单"必须沿整条继承链。

·实际问题：写"属性列示器"时，只查 I.slots 会漏掉超类的 slots——这就是第 31 章 mapattrs.py 要沿 MRO 逐个类扫描的原因。

·初学者误区：以为 I.slots 等于"我所有能用的属性名单"；它只是"最低层类声明的那份"。

通用地处理 slots 与其他"虚拟"属性（Handling slots and other "virtual" attributes generically)

英文原文

The prior chapter concluded with a brief summary of the slots policies of its attribute lister tools—a prime example of why generic programs may need to care about slots. Such tools that attempt to list instance data attributes generically must account for slots and perhaps other such "virtual" instance attributes like properties and descriptors introduced ahead—names that similarly reside in classes but may provide attribute values for instances on request. Slots are the most data-centric of these but are representative of a larger category. Such attributes require inclusive approaches, special handling, or general avoidance—the latter

of which becomes unsatisfactory as soon as any programmer uses slots in subject code. Really, class-level instance attributes like slots probably necessitate a redefinition of the term instance data—as locally stored attributes, the union of all inherited attributes, or some subset thereof. For example, some programs might classify slot names as attributes of classes instead of instances; these attributes do not exist in instance namespace dictionaries, after all. Alternatively, as shown earlier, programs can be more inclusive by relying on dir to fetch all inherited attribute names and getattr to fetch their corresponding values—without regard to their physical location or implementation. If you must support slots as instance data, this may be the most robust way to proceed:

```
>>> class Slotful:
...     __slots__ = ['a', 'b', '__dict__']
...     def __init__(self, data):
...         self.c = data
>>> I = Slotful(3)
>>> I.a, I.b = 1, 2
>>> I.a, I.b, I.c                 #
(1, 2, 3)
>>> I.__dict__                    # __dict__  slots
{'c': 3}
>>> [x for x in dir(I) if not x.startswith('__')]
['a', 'b', 'c']
>>> I.__dict__['c']               # __dict__
3
>>> getattr(I, 'c'), getattr(I, 'a')   # dir+getattr  __dict__ …
(3, 1)
>>> for a in (x for x in dir(I) if not x.startswith('__')):
```

```
...     print(a, '=>', getattr(I, a))
a 1
b 2
c 3
```

> Under this dir/getattr model, you can still map attributes to their inheritance sources and filter them more selectively by source or type, if needed, by scanning the MRO—as we did in the prior chapter's mapattrs.py (Example 31-14). As a bonus, such tools and policies for handling slots will potentially apply automatically to properties and descriptors too, though these attributes are more explicitly computed values, and less obviously instance-related data than slots. Also keep in mind that this is not just a tools issue. Class-based instance attributes like slots also impact the traditional coding of the setattr operator-overloading method we met in Chapter 30. Because slots and some other attributes are not stored in the instance dict, and may even imply its absence, classes must instead generally run attribute assignments by rerouting them to the object superclass.

中文翻译

上一章在结尾简要总结过它的属性列示工具对 slots 的处理策略——这正是"通用程序为什么必须关心 slots"的绝佳例证。凡是想通用地列出实例数据属性的工具，都必须把 slots 以及也许其他类似的"虚拟"实例属性（如后文将介绍的 properties 与 descriptors——它们同样住在类里，却能在被请求时为实例提供属性值）考虑进来。slots 是这类属性

中最"数据化"的，但它代表的是更大的一类。这类属性要求"包容式"的处理策略、专门的照顾、或者干脆敬而远之——而最后一种选择，只要主体代码里有人用了 slots，立刻就变得不能令人满意了。说真的，slots 这类"类级实例属性"的存在，可能迫使我们对"实例数据"（instance data）这个词重新定义——是"本地存储的属性"？还是"所有继承属性的并集"？抑或其中某个子集？例如，有些程序或许会把 slot 名字归类为"类的属性"而非"实例的属性"——毕竟它们确实不存在于实例命名空间字典里。又或者，如前面所示，程序可以更有包容性：用 dir 抓取所有继承来的属性名、用 getattr 抓取对应的值——完全不关心它们的物理位置或实现方式。如果你必须把 slots 当实例数据来支持，这或许是最稳妥的路子：在这个 dir/getattr 模型下，你依然可以像上一章 mapattrs.py（Example 31-14）那样扫描 MRO，把属性映射回它们的继承来源，并按来源或类型做更精细的过滤。附带的好处是：这套处理 slots 的策略与工具，将自动适用于 properties 与 descriptors——虽说这些属性更多是"显式计算出来的值"，不如 slots 那么明显是"实例相关的数据"。还要记住：这不只是工具的问题。像 slots 这样的"基于类的实例属性"，同样会影响第 30 章认识的 setattr 运算符重载方法的传统写法。由于 slots 等属性不存于实例 dict（甚至意味着没有 dict），类一般必须把属性赋值重路由到 object 超类去执行。

深度理解

·核心概念：处理 slots 的三种策略——包容（dir+getattr）、特殊处理（逐个类扫描）、回避；作者推荐前者。

·底层实现：getattr 走完整属性解析（类型描述符优先、

实例字典次之、类树兜底），所以对"存储无关"名字（slots、property、descriptor）天然通用。

·设计原因：与其为每种"虚拟属性"定制逻辑，不如用协议统一的 dir+getattr 组合，一次兼容所有未来新增的属性种类。

·实际问题：setattr 里写 self.dict[name] = value 在 slots 实例上直接炸掉——必须改用 object.setattr(self, ...)，这为第 39 章 Private 装饰器案例埋下伏笔。

·初学者误区：以为 dict 是"所有属性的家"；在 slots/property 面前它只是"属性来源之一"。

Slot 使用规则（Slot usage rules）

英文原文

> Slot declarations can appear in multiple classes in a class tree, but when they do, they are subject to a number of constraints that are somewhat difficult to rationalize unless you understand the implementation of slots as class-level descriptors for each slot name that are inherited by the instances in which the managed space is reserved (again, you'll meet descriptors briefly ahead). Here are the main constraints that slots impose: Slots in subs are pointless when absent in supers. If a subclass inherits from a superclass without a slots, the instance dict attribute created for the superclass will always be accessible, making a slots in the subclass largely pointless. The subclass still manages

its slots but doesn't compute their values in any way and doesn't avoid a dictionary—the main reason to u se slots. Slots in supers are pointless when absent in subs. Similarly, because the meaning of a slots declar ation is limited to the class in which it appears, subcl asses will produce an instance dict if they do not defi ne a slots, rendering a slots in a superclass largely po intless. Redefinition renders super slots pointless. If a class defines the same slot name as a superclass, its r edefinition hides the slot in the superclass per norma l inheritance. You can access the version of the name defined by the superclass slot only by fetching its des criptor directly from the superclass. Slots prevent clas s-level defaults. Because slots are implemented as cla ss-level descriptors (along with per-instance space), y ou cannot use class attributes of the same name to p rovide defaults as you can for normal instance attribu tes: assigning the same name in the class overwrites t he slot descriptor. Slots cannot be combined in multi ple inheritance. Multiple inheritance cannot be used i f more than one of the classes mixed together have n onempty slots lists—even if their slots define the sam e names. You'll get an error when running the class t hat does the mixing. Empty slots lists allow the mixer to define slots or not, as desired. Slots can impact dic t. As shown earlier, slots preclude both an instance di ct and assigning names not listed, unless dict is listed explicitly too. Slots similarly preclude a weakref attrib

ute used to support instance "weak references" cover ed briefly in Chapter 6, but these are rare enough to soft-pedal here. We've already seen the last of these in action. It's easy to demonstrate how the new rules here translate to actual code—most crucially, a name space dictionary is created when any class in a tree o mits slots, thereby negating the memory optimizatio n benefit but also supporting classes that require a di ct when mixed in with others:

```
>>> class C: pass                # 1 slots
>>> class D(C): __slots__ = ['a']
>>> I = D()                      #
>>> I.a = 1; I.b = 2
>>> I.__dict__
{'b': 2}
>>> D.__dict__.keys()
dict_keys([… '__slots__', 'a', …])
>>> class C: __slots__ = ['a']   # 2 slots
>>> class D(C): pass
>>> I = D()
>>> I.a = 1; I.b = 2
>>> I.__dict__
{'b': 2}
>>> C.__dict__.keys()
dict_keys([… '__slots__', 'a', …])
>>> class C: __slots__ = ['a']   # 3 slot
>>> class D(C): __slots__ = ['a']
>>> class C: __slots__ = ['a']; a = 99   # 4
ValueError: 'a' in __slots__ conflicts with class variable
>>> class C: __slots__ = ['a']   # 5
>>> class D: __slots__ = ['a']
>>> class E(C, D): pass
TypeError: multiple bases have instance lay-out conflict
```

> In other words, besides their program-breaking potential, slots essentially require both universal and careful deployment to be effective—because slots do not compute values dynamically like properties (coming up in the next section), they are largely pointless unless each class in a tree uses them and is cautious to define only new slot names not defined by other classes. It's an all-or-nothing feature—an unfortunate property shared by the super call discussed ahead:

```
>>> class C: __slots__ = ['a']   #
>>> class D(C): __slots__ = ['b']
>>> I = D()                      #
>>> I.a = 1; I.b = 2
>>> I.__dict__
AttributeError: 'D' object has no attribute '__dict__'. Did you me…
>>> C.__dict__.keys(), D.__dict__.keys()
(dict_keys([… '__slots__', 'a', …]), dict_keys([… '__slots__', 'b'…
```

> Such rules—and others omitted here for space—are part of the reason slots are not widely used and are not generally recommended except in pathological cases where their space reduction is significant. Even then, their potential to complicate or break code should be ample cause to carefully consider the trade-offs. Not only must they be spread almost neurotically throughout a framework, but they may also break tools you rely on.

中文翻译

slots 声明可以出现在类树中的多个类里，但一旦如此，就要受制于若干约束——除非你理解"slots 对每个 slot 名来

说就是类级描述符、且会被预留给管理空间的那些实例所继承"这一实现本质，否则这些约束很难讲得通（描述符稍后你就会见到）。以下是 slots 施加的主要约束：超类没有 slots 时，子类里的 slots 毫无意义。如果子类继承自一个没有 slots 的超类，超类创建出的实例 dict 总是可用的，这让子类里的 slots 基本形同虚设——子类虽仍管理着自己的 slots，但既不省任何值、也躲不开字典——而"躲开字典"恰恰是使用 slots 的主要理由。子类没有 slots 时，超类里的 slots 也毫无意义。同理，因为 slots 声明的含义只局限于出现它的那个类，子类若未定义 slots，就会产生实例 dict，使超类的 slots 基本形同虚设。重定义会使超类的 slots 失效。如果一个类定义了与超类同名的 slot，按普通继承规则，这个重定义会隐藏超类中的同名 slot。想访问超类 slot 定义的那个名字版本，只能直接从超类身上取它的描述符。slots 不允许类级默认值。因为 slots 实现为类级描述符（外加每实例的空间），你不能像普通实例属性那样用同名类属性提供默认值：在类里给同名赋值会覆盖掉 slot 描述符。slots 不能与多重继承组合。被混合的多个类中，只要不止一个类有非空 slots 列表，就不能用多重继承——即使它们的 slots 定义的是相同的名字也不行。执行那个做混合的类时就会报错。空的 slots 列表则允许混合者按需决定要不要定义 slots。slots 会波及 dict。如前所示，slots 既排除了实例 dict，也排除了未列入名单的名字的赋值——除非把 dict 也显式列入。slots 同理还会排除用于支持实例"弱引用"（weak references，第 6 章简单提过）的 weakref 属性，不过弱引用相当罕见，这里就轻轻带过了。上面最后一条我们已经见过了。这里的新规则如何落实为代码，很容易演示——最关键的一点是：类树中只要有任何一个

类省略了 slots，命名空间字典就会被创建，从而抵消内存优化收益（但反过来，这也支持了某些"混入时必须有 dict"的类）：换句话说，除了可能弄坏程序的潜力之外，slots 实际上要求全树普及且小心翼翼地部署才有效——因为 slots 不像 properties（下一节就要登场）那样动态计算值，除非树上每个类都用它、而且小心地只定义别人没定义过的新 slot 名，否则基本白搭。这是一个"全有或全无"（all-or-nothing）的特性——一个不幸的属性，它还与后文讨论的 super 调用共享：这些规则（以及为节省篇幅略去的其他规则），正是 slots 没有被广泛使用、除"省空间收益显著"的病态场景外一般不被推荐的部分原因。即便如此，它"搞复杂甚至弄坏代码"的潜力，也足以让你三思而后行。不仅几乎要神经质地把它铺满整个框架，它还可能弄坏你依赖的工具。

深度理解

·核心概念：六条规则背后一句话：slots 只有在"全树普及 + 命名不撞车 + 不使用混合"时才有意义。

·底层实现：多条规则的根源在"实例内存布局"：CPython 按 MRO 上所有类的 slot 描述符合并出最终布局；任何一级产生 dict（因缺 slots）都会破坏紧凑布局；两个非空 slots 基类合并会产生"布局冲突"（lay-out conflict）错误。

·设计原因：内存布局必须在类创建时敲定（类似 C 的 struct），这决定了它无法与"随时可加的字典"共存。

·实际问题：与 super 一样，这是作者反复点名的"all-or-nothing"特性——用一半等于没用，还可能弄坏工具。

·初学者误区：以为"子类加 slots、超类不加"能省内存；实际上超类一侧的字典照样创建，优化归零。

slots 影响的实例：ListTree 与 mapattrs（Example impacts of slots: ListTree and mapattrs）

英文原文

> As a more realistic example of slots' effects, due to the first bullet in the prior section, Chapter 31's ListTree class (Example 31-13) does not fail when mixed into a class that defines slots, even though it scans instance namespace dictionaries without verifying their presence. This lister class's own lack of slots is enough to ensure that the instance will still have a dict and hence not trigger an exception when fetched or indexed. For example, both of the following single-inheritance trees display without error—the second also allows names not in the slots list to be assigned as instance attributes, including any required by the superclass:

```
class C(ListTree): pass
I = C()                       #  __slots__
print(I)
class C(ListTree): __slots__ = ['a', 'b']   #  __dict__
I = C()
I.c = 3
print(I)                      #  I  c C  a  b
```

> The following multiple-inheritance classes display correctly as well—any nonslot class like ListTree generates an instance dict and can thus safely assume its pre

sence. Although it renders subclass slots pointless, th
is is a positive side effect for tool classes like ListTree
and its Chapter 28 predecessor:

```
class A: __slots__ = ['a']        # 1
class B(A, ListTree): pass
print(B())
class A: __slots__ = ['a']
class B(A, ListTree): __slots__ = ['b']    #  B  b A  a
print(B())
```

In general, though, tools may need to catch exceptio
ns when dict is absent or use a hasattr or getattr to t
est or provide defaults if slot usage may preclude an
instance namespace dictionary. For instance, Chapter
31's mapattrs.py module (Example 31-14) must check
for dict presence explicitly because it is not a class mi
xed into others, and so cannot assume this attribute.
Like ListTree, this example also associates slots with t
heir classes. Run these examples on your own for mo
re info. Slots' impacts may be onerous, but knowledg
e is your best defense.

中文翻译

作为 slots 影响的一个更现实的例子：由于上一节的第一条规则，第 31 章的 ListTree 类（Example 31-13）即使被混入一个定义了 slots 的类，也不会崩——尽管它会扫描实例命名空间字典而不验证其存在。这个列示器类自己不定义 slots 这一事实，就足以保证实例仍有 dict，因此取它、索引它都不会触发异常。例如，下面两个单继承树都能无错显示——第二个还允许把 slots 名单之外的名字（包括超类可

能需要的任何名字）赋成实例属性：下面这些多重继承的类也能正确显示——任何像 ListTree 这样的非 slot 类都会生成实例 dict，从而可以安全地假定它的存在。虽然这使子类的 slots 形同虚设，但对 ListTree 及其第 28 章前辈这样的工具类来说，这是个"正面副作用"：不过一般来说，工具可能需要：在 dict 缺失时捕获异常；或者用 hasattr/getattr 测试、提供默认值——以防 slot 的使用使实例失去命名空间字典。例如，第 31 章的 mapattrs.py（Example 31-14）必须显式检查 dict 是否存在，因为它不是"混入别的类"的类，无法假定这个属性存在。和 ListTree 一样，这个例子也把 slots 与它们的类关联起来。请自己运行这些例子以获得更多信息。slots 的影响可能令人头痛，但知识是你最好的防御。

深度理解

·核心概念：工具类（lister/mapper）与 slots 的共存策略：自己不写 slots → 保证 dict 存在 → 安全；或显式 hasattr 检查。

·底层实现：print(l) 触发 ListTree.str，它用 getattr(l,'dict', None) 或直接 l.dict 枚举；只要树里有一个非 slot 类，dict 就在。

·设计原因：作者以此说明"slots 的规则也可以反过来成为工具的护身符"——非 slot 类的存在是"宽容的锚点"。

·实际问题：mapattrs.py 这类独立模块无法依赖树内存在非 slot 类，只能显式探测——同一问题的两种应对。

·初学者误区：以为工具只要"小心一点"就能兼容 slots；实际上设计时就必须定策略（包容/探测/回避），运行期

再补救已迟。

slots 的速度到底如何？ (What about slots speed?)

英文原文

> Finally, while slots primarily optimize memory use, th eir speed impact is less clear-cut. Example 32-4 code s a simple test script using the timeit techniques we s tudied in Chapter 21. For both the slots and nonslots (instance dictionary) storage models, it makes 1,000 i nstances, assigns and fetches 4 attributes on each, an d repeats 1,000 times—for both models taking the b est of 5 runs that each exercise a total of 8M attribut e operations. Example 32-4. slots-test.py

```
import timeit
base = """
Is = []
for i in range(1000):
    I = C()
    I.a = 1; I.b = 2; I.c = 3; I.d = 4
    t = I.a + I.b + I.c + I.d
    Is.append(I)
"""
stmt = """
class C:
    __slots__ = ['a', 'b', 'c', 'd']
""" + base
print('Slots =>', end=' ')
print(min(timeit.repeat(stmt, number=1000, repeat=5)))
stmt = """
class C:
    pass
""" + base
```

```
print('Nonslots=>', end=' ')
print(min(timeit.repeat(stmt, number=1000, repeat=5)))
```

> At least for this code, on the macOS test host, and using CPython 3.12, the best times imply that slots are only slightly quicker, though this says little about memory space and is prone to change arbitrarily in the future (PyPy 7.3 struggled on this test with times 10x slower than CPython, presumably due to dynamic class creation, but relatively similar):

```
$ python3 slots-test.py
Slots => 0.17895982996560633
Nonslots=> 0.18887511501088738
```

> For more on slots in general, see the Python standard manual set. Also, watch for the Private decorator case study of Chapter 39—an example that naturally allows for attributes based on both slots and dict storage, by using delegation and storage-neutral accessor tools like getattr.

中文翻译

最后，slots 主要优化的是内存，而它对速度的影响则没那么明确。Example 32-4 用我们在第 21 章学过的 timeit 技巧写了一个简单的测试脚本：对 slots 与非 slots（实例字典）两种存储模型，都创建 1,000 个实例、给每个实例赋值并读取 4 个属性、重复 1,000 次——两个模型都取 5 次运行的最好成绩，每次运行总共做了 800 万次属性操作。Example 32-4. slots-test.py（文件开头 import timeit 导入计时模块；base 是两套存储共用的"创建实例→赋值→

求和→收集"脚本；stmt 先后拼上 slots 版与普通版类定义；最后用 min(timeit.repeat(...)) 取 5 轮中最快的成绩）：

```
import timeit
base = """
Is = []
for i in range(1000):
    I = C()
    I.a = 1; I.b = 2; I.c = 3; I.d = 4
    t = I.a + I.b + I.c + I.d
    Is.append(I)
"""
stmt = """
class C:
    __slots__ = ['a', 'b', 'c', 'd']
""" + base
print('Slots =>', end=' ')
print(min(timeit.repeat(stmt, number=1000, repeat=5)))
stmt = """
class C:
    pass
""" + base
print('Nonslots=>', end=' ')
print(min(timeit.repeat(stmt, number=1000, repeat=5)))
```

至少就这段代码而言，在 macOS 测试主机、CPython 3.12 上，最优成绩显示 slots 只略微快一点——不过这并不能说明内存空间上的差异，而且这个结果将来随时可能变化（PyPy 7.3 在这个测试里表现挣扎，用时比 CPython 慢 10 倍，大概是动态创建类所致，但相对差距类似）：

```
$ python3 slots-test.py
Slots => 0.17895982996560633
Nonslots=> 0.18887511501088738
```

关于 slots 的更多一般信息，参见 Python 标准手册。另外，敬请留意第 39 章的 Private 装饰器案例研究——那个例

子通过委托与 getattr 这类存储无关的访问器工具，天然同时支持基于 slots 与 dict 两种存储的属性。

深度理解

·核心概念：实测数据说话：CPython 3.12 下 slots 与字典存储的速度几乎无差（0.179s vs 0.189s）；slots 的真正价值只在内存。

·底层实现：为什么没快多少？CPython 的属性访问本来就经过缓存优化（inline caching、函数调用少），描述符查找与字典查找的差距在现代解释器里已被抹平。

·设计原因：作者特意给"slots 更快"的都市传说泼冷水——把记忆点锚定在"内存"上，避免读者为伪优化付出真实复杂度。

·实际问题：性能决策要用数据而非直觉；timeit 取 min()（最稳的一次）而非平均值，正是计时方法论的正确姿势。

·初学者误区：以为"slots 提升性能"成立；实际上它最可靠的价值是"阻止拼写错误"与"省内存"，速度收益可忽略。

32.6 属性：访问器（Properties: Attribute Accessors）

英文原文

Our next attribute-related topic is properties—a mec

hanism that provides another way for classes to defin e methods called automatically for access or assignm ent to instance attributes. This feature is similar to "g etters" and "setters" in languages like Java and C#, b ut in Python is generally best used sparingly as a way to add accessors to attributes after the fact as needs evolve and warrant. Where needed, though, properti es allow attribute values to be computed dynamically without requiring method calls at the point of access. Though properties cannot support generic attribute r outing goals, at least for specific attributes, they are an alternative to some traditional uses of the getattr and setattr overloading methods we first studied in C hapter 30. Properties can have a similar effect to thes e two methods but, by contrast, incur an extra metho d call only for accesses to names that require dynami c computation—other nonproperty names are access ed normally with no extra calls. Although getattr is in voked only for undefined names, the setattr method i s instead called for assignment to every attribute. Pro perties and slots are related, too, but serve different goals. Both implement instance attributes that are no t physically stored in instance namespace dictionaries —a sort of "virtual" attribute—and both are based on the notion of class-level attribute descriptors. In cont rast, slots manage instance storage, while properties intercept access and compute values arbitrarily. Beca use their underlying descriptor implementation tool i

s too advanced for us to cover here, properties and descriptors both get full treatment in Chapter 38.

中文翻译

我们下一个与属性相关的主题是 properties（属性）——它为类提供了又一种"对实例属性的访问或赋值自动调用方法"的定义机制。这个特性与 Java、C# 等语言里的"getters / setters"（取值器/设值器）类似，但在 Python 中通常最好少用，把它当作"事后的、随着需求演化而按需添加的"给属性加访问层的手段。当然，在需要处，properties 允许属性值动态计算，而无需在访问点显式调用方法。properties 无法支持"通用属性路由"的目标，但至少针对具体属性，它可以替代 getattr 与 setattr 这两个重载方法（我们早在第 30 章见过）的某些传统用法。properties 的效果与这两个方法类似，但区别在于：只有访问那些需要动态计算的属性名时才多一次方法调用——其他非 property 名字照常访问、零额外调用。反观 getattr 只对未定义的名字触发，而 setattr 则对每一次属性赋值都被调用。properties 与 slots 也有亲缘关系，但分工不同。两者都实现了"不物理存放于实例字典里"的实例属性——一种"虚拟"（virtual）属性——并且都建立在"类级属性描述符"（descriptors）这同一机制之上。区别在于：slots 管理实例存储，而 properties 拦截访问并可任意计算值。因为它们背后的描述符实现机制对我们来说还太复杂，properties 与 descriptors 都将在第 38 章获得完整的讲解。

深度理解

·核心概念：property 把"读"、"写"、"删"三种操作分别绑

定到三个可选方法上；只传 getter 时它就是只读属性。

·底层实现：property(fget, fset, fdel, doc) 返回的对象本身就是一个描述符；挂到类属性名上后，obj.name 的读取/赋值/删除自动路由到相应方法。

·设计原因：相比 setattr 的"全拦截"，property 是"精确制导"——只拦截你指定的名字，其余名字零开销。

·实际问题：改造旧代码时（如把内部 age 升级为带校验的 age 属性），property 能以不破坏调用方语法的方式"事后打补丁"。

·初学者误区：以为 property 的 getter 只在"第一次"计算、之后缓存；其实每次都调用（要缓存得自己加 functools.cachedproperty）。

代码分析（Property 基础）

```
>>> class WithOperators:
...     def __getattr__(self, name):        # ""
...         if name == 'age':
...             return 40
...         else:
...             raise AttributeError(name)
>>> x = WithOperators()
>>> x.age                                   #  __getattr__
40
>>> x.name                                  #  __getattr__
AttributeError: name
```

换成 properties 实现同样效果：

```
>>> class WithProperties:
...     def getage(self):
...         return 40
...     age = property(getage)             # (get?, set?, del?, do…
>>> x = WithProperties()
>>> x.age                                  #  getage
40
>>> x.name                                 #
AttributeError: 'WithProperties' object has no attribute 'name'
```

解释器如何处理：x.age 做属性查找，解释器在 type(x) 的 MRO 中发现 age 是一个 property（描述符），于是调用它的 get(x, type(x)) → 即 getage(x)。而 x.name 的 name 不在任何地方定义，普通查找失败——与 getattr 版不同，property 版不会生成"动态属性"。注意：property 版的 x.name 抛出的错误消息更自然，这正是"专用工具干专用事"的好处。

进一步，加上赋值（setter）支持后 property 的优势更明显——代码更少，且对"不想管理的属性"的赋值不再有额外方法调用：

```
>>> class WithProperties:
...     def getage(self):
...         print('get age')
...         return 40
...     def setage(self, value):
...         print('set age:', value)
...         self._age = value
...     age = property(getage, setage)
>>> x = WithProperties()
>>> x.age                                  #  getage
get age
40
>>> x.age = 42                             #  setage
```

```
set age: 42
>>> x._age                                  #  getage
42
>>> x.job = 'hacker'                        #  setage
>>> x.job                                   #  getage
'hacker'
```

而等价的运算符重载版，对"未管理的属性"赋值也要付出额外方法调用，并且必须把属性赋值路由进属性字典以避免死循环（或路由到 object 超类的 setattr 以更好地支持其他类里定义的 slots、properties 这类"虚拟"属性）：

```
>>> class WithOperators:
...     def __getattr__(self, name):       #
...         if name == 'age':
...             print('get age')
...             return 40
...         else:
...             raise AttributeError(name)
...     def __setattr__(self, name, value):  #
...         print('set:', name, value)
...         if name == 'age':
...             self.__dict__['_age'] = value    #  object.__set…
...         else:
...             self.__dict__[name] = value
>>> x = WithOperators()
>>> x.age                                   #  __getattr__
get age
40
>>> x.age = 41                              #  __setattr__
set: age 41
>>> x._age                                  #  __getattr__
41
>>> x.job = 'coder'                         #  __setattr__
set: job coder
>>> x.job                                   #  __getattr__
'coder'
```

解释器如何处理（关键差异）：运算符重载版为每一次属性赋值付出调用开销——x.job ='coder' 也要走一遍 setattr 并打印日志；property 版则完全跳过非托管属性。而且运算符版还需在 setattr 内部绕过递归（self.dict[...] 或 object.setattr），否则 self.age = value 会再次触发 setattr 形成无限递归。这就是"专用工具"（property）胜出之处。

> Properties seem like a win for this simple example. However, some applications of getattr and setattr still require more dynamic or generic interfaces than properties directly provide. For example, the set of attributes to be managed might be unknown when a class is coded and may not even exist in a tangible form (e.g., when delegating arbitrary attribute references to a wrapped and embedded object generically). In such contexts, a generic attribute handler like getattr with a passed-in attribute name may be preferable. Because such generic handlers can also support simpler cases, properties may be a redundant extension—albeit one that may avoid extra calls on assignments and one that some programmers may prefer when applicable. For more details on both options, tune in to Chapter 38. As you'll see there, it's also possible to code properties using the @ symbol function decorator syntax—a topic introduced in brief later in this chapter and hence it's equivalently an automatic alternative to manual assignment in class scope:

```
class WithProperties:
    @property            #  property
    def age(self):       #  instance.age
        ...
    @age.setter
    def age(self, value):   #  instance.age = value
        ...
```

> To make sense of this decorator syntax, though, we must move ahead.

深度理解

·核心概念：property 胜在"精准拦截 + 零额外开销"，运算符重载版是"全量拦截 + 处处付费"。

·底层实现：普通属性赋值先走 setattr（object 的 C 实现）→ 描述符协议 → 实例字典；property 直接 hook 进描述符层，天然避开字典。

·设计原因：getattr 只拦"未定义"、setattr 拦"一切"——中间地带（只拦某几个名字）恰好就是 property 的生态位。

·实际问题：委托（delegation）场景（把任意属性转发给内嵌对象、名字集合不可预知）仍要去语法 getattr；property 面对的是"名字已知的固定按钮"。

·初学者误区：把 property 当性能优化工具；实际上它通常更慢（每个访问多一层方法调用），价值是"代码清晰 + 可校验 + 后补兼容"。

32.7 getattribute 与描述符：属性的另一种实现（getattribute and Descriptors: Attribute Implementations)

英文原文

> To complete our attribute-tools collection, the getattribute operator-overloading method intercepts all attribute references, not just undefined references. This makes it more potent than its getattr cousin we used in the prior section, but also trickier to use—it's prone to loops much like setattr, but in different ways.

> For more specialized attribute interception goals, in addition to properties and operator-overloading methods, Python provides attribute descriptors—classes with get and set methods, assigned to class attributes and inherited by instances, that intercept read and write accesses to specific attributes. As a preview, here's one of the simplest descriptors:

```
> >> class AgeDesc:

...     def __get__(self, instance, owner): return 40
...     def __set__(self, instance, value): instance._age = value
> >> class WithDescriptors:

...     age = AgeDesc()                 #
> >> x = WithDescriptors() >> x.age #  AgeDesc.__get__

40
```

```
> >> x.age = 42 #  AgeDesc.__set__ >> x._age #  AgeDesc

42
```

> Descriptors have access to state-information attributes in instances of themselves as well as their client class and are, in a sense, a more general form of properties. In fact, properties are a simplified way to define a specific type of descriptor—one that runs functions on access. Descriptors are also used to implement the slots feature we met earlier, among other Python tools, and are afforded special cases in attribute inheritance alluded to earlier in this chapter.

> Because getattribute and descriptors are too substantial to present here, we'll defer the rest of their coverage to Chapter 38. We'll also employ them in Python 39's examples and study how they factor into inheritance in Chapter 40. Here, the topics tour is moving on.

中文翻译

> 为了凑齐属性工具全家福，先看 getattribute 运算符重载方法：它拦截所有属性引用，不只拦截未定义的引用。这比上一节的表亲 getattr 更"威力强大"，但也更考验使用技巧——它和 setattr 一样容易陷入无限循环，只是陷入的方式不同。此外，为了更专门的属性拦截目标，除 properties 与运算符重载方法外，Python 还提供属性描述符（descriptors）——带 get 与 set 方法的类，被赋给类属性并被实例继承，用于拦截对特定属性的读写访问。作为预览，下面是一个你可能遇到的最简单的描述符：

```
>>> class AgeDesc:
...     def __get__(self, instance, owner): return 40
...     def __set__(self, instance, value): instance._age = value
>>> class WithDescriptors:
...     age = AgeDesc()                 #
>>> x = WithDescriptors()
>>> x.age                               #  AgeDesc.__get__
40
>>> x.age = 42                          #  AgeDesc.__set__
>>> x._age                              #  AgeDesc
42
```

> 描述符既能访问自己实例上的状态信息属性，也能访问其客户端类的状态，因此在某种意义上，它们是一种更通用的属性形态。事实上，properties 正是"某种特定描述符"的简化写发——那种在访问时运行函数的描述符。描述符还被用来实现前面提到的 slots 特性（以及其他 Python 工具），并且在本章早些时候提到的"属性继承"中享有特殊地位。由于对 getattribute 与描述符展开讲内容太庞大，我们将其余覆盖推迟到第 38 章；第 39 章的示例也会用到它们，第 40 章则会研究它们如何参与继承。就主题巡游而言，这里要往前走一步了。

深度理解

·核心概念：getattribute（拦截一切）、getattr（拦截"未定义"）、descriptor（拦截"特定名字"）是三种粒度递增的拦截器。

·底层实现：getattribute 是属性协议的"总入口"（对象模型规范），描述符在它内部被查识；obj.age 若 getattribute 被重写，细节全部由你重排，极易递归。

·设计原因：描述符把"拦截逻辑"装进可复用对象（一个 AgeDesc 可贴到多个类），实现"一次定义、多处复用"。

·实际问题：slots、property、绑定方法、staticmethod/classmethod 全部都是基于描述符实现的——理解了它，等于理解了半本 OOP。

·初学者误区：以为 getitem 一类的运算符协议与属性查找同路；其实运算符的隐式查找从类出发，常绕过 getattr/getattribute——这就是"委托内置失效"的根源。

32.8 静态方法与类方法（Static and Class Methods）

英文原文

Beyond the usual methods we've been using so far, classes can define two kinds of methods called without an instance: static methods work roughly like simple instance-less functions inside a class no matter how they're called, and class methods are passed a class instead of an instance. Both are similar to tools in other languages (e.g., C++ static methods). The prior chapter's bound method coverage noted these briefly, but we'll finish the story here. To enable these special method modes, you call built-in functions named staticmethod and classmethod within the class or invoke them with the special @name decoration syntax you'll meet later in this chapter. The classmethod call is

required to enable its mode; staticmethod is not required for instance-less methods called only through a class name but is required if such methods are called through instances.

中文翻译

在我们迄今用过的普通方法之外，类还能定义两种不需要实例就能调用（called without an instance）的方法：static（静态）方法无论怎么被调用，都大致像类里一个普通的无实例函数；class（类）方法则收到一个"类对象"而非"实例"作为参数。两者都与别的语言里的工具相似（例如 C++ 的静态方法）。上一章讲绑定方法时已经简略提过它们，这里我们把故事讲完。要启用这些特殊的方法模式，你在类内调用名为 staticmethod 与 classmethod 的内建函数，或用稍后本章要介绍的特殊 @name 装饰语法来调用它们。classmethod 调用是启用其模式必需的；staticmethod 对"只通过类名调用、且无实例"的方法并非必需，但如果要从实例调用这类方法，则必须有它。

深度理解

·核心概念：普通方法（instance methods）自动收 self；静态方法收不到任何东西；类方法自动收类对象（不叫 self 就叫 cls）。

·底层实现：staticmethod/classmethod 返回的描述符类在访问阶段改变"绑定"行为——instance.getattr 时的绑定把函数变成"原始函数"（静态）或"绑定到类"（类方法）。

·设计原因：处理"类级数据"（如实例计数、类级配置）时，无实例可用的方法要有安身之地；把它们放类里又能享受继承链的多态。

·实际问题：@staticmethod 与 @classmethod 是阅读框架代码（ORM、配置类）的高频词。

·初学者误区：以为静态方法=类方法。区别在于：静态方法不接收任何主体参数；类方法接收"当前最低层的类"（这就是继承时类方法的表现行为）。

为什么需要这些特殊方法？（Why the Special Methods?）

英文原文

As we've seen, a class's a method normally passed an instance object in its first argument to serve as the implied subject of the method call—that's the "object" in "object-oriented programming." Though it's normal to expect, there are two formal ways to temper this model. Before we get to the syntax, let's clarify why this might matter to you. Sometimes, programs need to process data associated with classes instead of instances. Consider keeping track of the number of instances created from a class or maintaining a list of all of a class's instances that are currently in use. This type of information and its processing are associated with the class rather than its instances. That is, the information is usually stored on the class itself and proc

essed apart from any instance. For such tasks, simple functions coded outside a class might already suffice—because they can access class attributes through the class name, they have access to class data, and never require access to an instance. However, to better associate such code with a class and to allow such processing to be customized with inheritance as usual, it would be better to code these types of functions inside the class itself. To make this work, we need, that monsters in a class are not passed, and do not expect, a self instance argument. Per the prior chapter, methods accessed through the class are plain functions that meet some of this need but fail if accessed through an instance. Recall: the resulting bound method passes an instance in calls, even if the plain function doesn't expect one. To address, Python provides static methods—plain functions that are nested in a class and never expect nor receive an automatic self argument, regardless of how they are called. They're optional for methods only ever accessed through classes but needed for access through instances. Although less commonly used, Python also supports class methods—methods of a class that are passed a class object in their first argument instead of an instance, regardless of whether they are called through an instance or a class. Such methods can access class data through their class argument—what we've called self thus far—even if called through an instance. Normal methods, so

metimes called instance methods, still receive a subject instance when called; static and class methods do not. Below is an example-based tour.

中文翻译

如我们所见的，类的方法通常在其第一个参数接收一个实例对象，作为方法调用时隐含的"主体"——这正是"面向对象编程"中"对象"一词的由来。虽然接收实例是常态，但也有两种不那么常见的模式需要放宽这一模型。在讲语法前，先弄清楚："这对你来说为何重要？"有些时候，程序需要处理与类相关而非与实例相关的数据。想想看：统计一个类创建了多少实例、维护"当前使用中的该类的全部实例"列表。这类信息及其处理逻辑，关联的是"类"而不是"实例"。也就是说，这类信息通常存储在类自身上，并在任何实例之外被处理。对于这类任务，写在类外面的普通函数也许已经够用——因为它们能通过类名访问类属性，因此拿得到类数据，且从不要求访问实例。但为了把这类代码更好地与类关联起来、并让这类处理能够照常通过继承被定制，把这类函数写进类内部会更好。要让它成立，我们需要的是"类内不接收、也不期望 self 实例参数的方法"。按上一章所述，"通过类访问"的方法是一等普通函数（plain functions），能满足部分需求，但"通过实例访问"就会失败：产生的绑定方法（bound method）即使这不期望实例，也会把实例塞进调用参数。针对这一点，Python 提供了静态方法——嵌套在类里的普通函数，无论怎样被调用，都永远不期望也不接收自动传入的 self 参数。对"只经由类访问"的方法它是可选的，但对实路而来的调用却是必须的。尽管较少被用到，Python 还支持类方法：其第一个参数收的是一类对象而非一

> 个实例，无论它是通过实例还是通过类来调用的。这样，即使通过实例调用，这类方法也能经由它的"类参数"访问类数据——也就是此前我们一直叫"self"的东西。普通方法（有时称实例方法）被调用时仍接收一个"主体实例"；静态方法和类方法则不然。天生小编我来演示一下——请看下面几个例子。

深度理解

·核心概念："类数据（class data）+ 无实例上下文"的两难：函数放外部 → 失去关联与继承；绑进类 → 会被自动塞入 self。静态/类方法就是解。

·底层实现：类属性近似地讲只是命名空间；经 C.meth 取到的是一等普通函数，经 I.meth 取到的是绑定方法（闭包捕获 I）。staticmethod/classmethod 这两个描述符改写的是"绑定结果"。

·设计原因：把"与外部函数同构"的逻辑也搬进类内，享受作用域隔离、命名不冲突、可被继承定制——这三点给了静态方法存在的正当性。

·实际问题：实例计数器、类级缓存、批量创建的"工厂方法"（@classmethod 返回 cls(...)）都是典型应用。

·初学者误区：遇"类数据 + 无实例"就乱用类变量+类名引用；本地化优化解决方案是老老实实写清楚 self。

纯函数方法（Plain-Function Methods）

英文原文

> To demo the preceding ideas, let's suppose that we want to use class attributes to count how many instances are generated from a class. Example 32-5, hack1.py, makes a first attempt—its class has a counter stored as a class attribute, a constructor that bumps the counter up by one each time a new instance is created, and a method that displays the counter's value. Remember, class attributes are stored just once on a class and shared by all instances; storing the counter this way ensures that it effectively spans all instances.

> Example 32-5. hack1.py

```
class Hack:
    numInstances = 0
    def __init__(self):
        Hack.numInstances += 1
    def printNumInstances():
        print('Number of instances created:', Hack.numInstances)
```

> The printNumInstances method is designed to process class data, instance data—it's about all instances, not any one in particular. Because of that, we want to be able to call it without having to pass an instance. Indeed, we do not want to create an instance just to fetch the number of instances, because this would change the number of instances we're trying to fetch?! In other words, we want a self-less "static" method.

Whether this code's printNumInstances works or not, though, depends on which way you call the method

—through the class or through an instance. Calls to self-less methods made through classes work because they produce plain functions, but calls from instances produce bound methods and fail:

```
$ python3
> >> from hack1 import Hack >> a, b, c = Hack(), Hack(), Hack() # …

Number of instances created: 3
> >> a.printNumInstances() #

TypeError: Hack.printNumInstances() takes 0 positional arguments b…
```

Calls to instance-less methods made through the class work, but calls made through an instance fail because an instance is automatically passed to a method that does not have an argument to receive it. To allow self-less methods to be called through instances, you'd need to either adopt other designs or mark such methods. Let's look at both options in turn.

中文翻译

为了演示上面这些想法，假设我们想用类属性统计一个类总共生成了多少个实例。Example 32-5（hack1.py）做了第一次尝试——它的类有一个存成类属性的计数器、一个每次创建新实例就令计数器 +1 的构造器、以及一个显示计数器值的方法。记住：类属性只在类上存一份、被所有实例共享；用这种方式存计数器，保证它的计数能跨所有实例生效。

Example 32-5. hack1.py

```
class Hack:
    numInstances = 0
    def __init__(self):
        Hack.numInstances += 1
    def printNumInstances():
        print('Number of instances created:', Hack.numInstances)
```

> printNumInstances 设计来处理的是类数据而非实例数据——它讲的是"所有的实例"，不属于某一个。正因如此，我们希望无需传入实例就能调用它。事实上，我们不想为了取"实例总数"而新建一个实例，因为那会改变我们正要查询的实例总数！换言之，我们想要一个"无 self 的静态方法"。这段代码的 printNumInstances 能不能成功，取决于你怎么调用它——通过类还是通过实例。通过类调用无 self 的方法是成立的，因为它产出的是普通函数；但通过实例调用则产出绑定方法，就会失败：

```
$ python3
>>> from hack1 import Hack
>>> a, b, c = Hack(), Hack(), Hack()
>>> Hack.printNumInstances()             #
Number of instances created: 3
>>> a.printNumInstances()                #
TypeError: Hack.printNumInstances() takes 0 positional arguments b…
```

> 通过"类"调用无实例方法成功；通过"实例"调用失败，是因为实例被自动塞进了一个"没有参数接收它"的方法。若想让无 self 方法也能经实例调用，你要么另作设计、要么把这些方法标记为特殊。让我们依次看看两个选项。

深度理解

·核心概念：绑定（bound）vs 非绑定（unbound）机制直接决定了方法能不能省掉 self。

·底层实现：Hack.printNumInstances 是个普通函数对象（无绑定）；a.printNumInstances 则是"绑定方法"对象，调用时把 a 作为第一个实参——于是少一个参数的方法瞬间多了一个实参而报 TypeError。

·设计原因：正是这种"类与实例取函数行为不同"的机制，催生了"给函数打标记"的静态方法设计——否则无 self 方法只能限制在"类调用"一种用法。

·实际问题：实例计数（工厂统计、监控埋点）是这一节的动机场景：为了查询计数而新建实例会污染数据。

·初学者误区：写 def printNumInstances(): 顺手期望实例不能调用；须知"实例也能调用"只是习惯，Python 从不打算屏蔽这种经实例调用的错误。

静态方法的替代方案（Static Method Alternatives）

英文原文

> Short of marking a self-less method as special, you can sometimes achieve similar results with different coding structures. Perhaps the simplest idea is to use normal functions outside the class, not class methods. This way, an instance isn't expected in the call. The mutation in Example 32-6 illustrates.

> Example 32-6. hack2.py

```
def printNumInstances():
    print('Number of instances created:', Hack.numInstances)
class Hack:
    numInstances = 0
    def __init__(self):
        Hack.numInstances += 1
```

> Because the class name is accessible to the simple fu nction as a global variable, this works fine. The functi on name becomes global but only to this single mod ule; it will not clash with names in other files:

```
> >> import hack2 as hack >> a = hack.Hack() >> b = hack.Hack()…

Number of instances created: 3     # ……
> >> hack.Hack.numInstances

3
```

> Prior to static methods in Python, this structure was t he general prescription. Because Python already prov ides modules as a namespace-partitioning tool, one c ould argue that there's not typically any need to pack age functions in classes unless they implement object behavior. Simple functions within modules do much of what instance-less class methods could and are alr eady associated with the class because they live in th e same module.

This approach, though, may be subpar. For one thing, it adds to this file's scope a global name used only fo r processing a single class. For another, the function i s not directly associated with the class by structure; it s def could be hundreds of lines away. Worse, simple

functions like this cannot be customized by inheritance since they live outside a class's namespace: subclasses cannot directly replace such a function by redefining it.

> We might also try making this example work by always calling through an instance, as usual. Unfortunately, such an approach is completely unworkable if we don't have an instance available, and making an instance changes the class data, as noted earlier. A better solution would be to somehow mark a method inside a class as never requiring an instance. The next section shows how.

中文翻译

> 如果不去给 self-less 方法打特殊标记，有时你还能用别的结构达到类似效果。最简单的想法：用类外面的普通函数，而不是类方法。这样调用时就不会期待实例了。Example 32-6 的改法说明了这一点。
>
> Example 32-6. hack2.py：把 printNumInstances 搬到类外，成为模块级函数；它通过全局变量可以访问 Hack 类名：

```
def printNumInstances():
    print('Number of instances created:', Hack.numInstances)
class Hack:
    numInstances = 0
    def __init__(self):
        Hack.numInstances += 1
```

> 因为类名对这个普通函数而言是可见的全局变量，这能正常

工作。函数名虽然是全局的，但只属于这一个模块，不会与其它文件的名字冲突：

```
>>> import hack2 as hack
>>> a = hack.Hack()
>>> b = hack.Hack()
>>> c = hack.Hack()
>>> hack.printNumInstances()      #
Number of instances created: 3
>>> hack.Hack.numInstances        #
3
```

在 Python 推出静态方法之前，这种结构是一般的官方指导做法。因为 Python 已经提供了"模块"作为命名空间分区工具，可以争辩说：除非函数要表现对象行为，否则通常没必要把它打包成类。模块里的普通函数，就已能完成无实例类方法的大部分工作，又因它们和类住在同一模块，已经"关联着类"。不过这个方案可能欠佳。其一，它给这个文件的全局作用域塞进一个只为处理某个类服务的名字；其二，函数与类没有结构上的直接关联——它的 def 可能离类几百行远。更要命的是，像这样的普通函数不能被继承定制：它们生活在类命名空间之外，子类没法通过重定义来替换或扩展它。我们也可以尝试"始终通过实例调用普通方法"来让例子成立。可惜若没有可用实例，这方案完全不成立；而制造一个实例又会改变类数据（前面说过了）。更好的答案是把类内某个方法"标记为永远不需要实例"。下一节就讲怎么做。

深度理解

·核心概念：三种"无实例函数"的演进：模块顶层函数 → （缺陷：无继承、占全局名）→ 静态方法（类内、可继承、

无冲突）。

·底层实现：模块顶层名字被放进模块字典，其它文件 import 时只能通过模块名取；因而不会全局碰撞属性于 Python 的"命名空间金字塔"之下。

·设计原因：作者用弃方案来说明"类内嵌函数"的三个红利：作用域隔离、物理就近、可继承重。这也是为何静态方法值得存在。

·实际问题：框架常暴露模块.函数的顶层 API；若它依赖特定类，改用 @staticmethod 更内聚、更强健。

·初学者误区：以为随处放顶层函数没关系；当它依赖某个类的内部数据时，就破坏了"类即自足单元"的封装边界。

使用静态与类方法（Using Static and Class Methods)

英文原文

> To designate a self-less method that may be called through either the class or its instances, classes can simply call the built-in functions staticmethod and classmethod. Both mark a function object as special—requiring no instance for the former and requiring a class argument for the latter. Example 32-7 shows how.

> Example 32-7. allmethods.py

```
class Methods:
    def imeth(self, x):        #  self
        print([self, x])       #  self
    def smeth(x):              #
        print([x])          #
    def cmeth(cls, x):         #
        print([cls, x])      #
    smeth = staticmethod(smeth)   #  smeth  @
    cmeth = classmethod(cmeth)    #  cmeth  @
```

Notice how the last two assignments in this code sim ply reassign (a.k.a. rebind) the method names smeth and cmeth. Attributes are created and changed by an y assignment in a class statement, so these final assig nments simply overwrite the assignments made earli er by the def statements. As you'll see in a few mome nts, the @ decorator syntax works as an alternative t o this, but it's essentially teaching you the assignmen t form here that it automates.

Technically, Python supports three kinds of class-rela ted methods with differing argument protocols: • Inst ance methods: passed a self instance object — the d efault. • Static methods: passed no extra instance obj ect (via staticmethod). • Class methods: passed a clas s object (via classmethod, inherent in metaclass).

Moreover, plain functions in a class also serve as stati c methods, without any extra protocol, when called t hrough a class only. The allmethods.py module illustr ates all three method types, so let's expand on these in turn.

Instance methods are the normal and default case. A n instance method must always be called with an inst ance object. When you call it through an instance, Py thon passes the instance to the first (leftmost) argum ent automatically; when you call it through a class, yo u must pass the instance manually:

```
> >> from allmethods import Methods >> obj = Methods() # …

[<allmethods.Methods object at 0x1015a79b0>, 1]
> >> Methods.imeth(obj, 2) #

[<allmethods.Methods object at 0x1015a79b0>, 2]
```

Static methods. By contrast, are called with no instan ce argument at all. Their names are local to the class scopes in which they're defined, and they may be loo ked up by inheritance. Instance-less functions can be called through a class normally, but using staticmeth od allows the same function to be called through an i nstance too:

```
> >> Methods.smeth(3) #

[3]                                #
> >> obj.smeth(4) #

[4]                                # —— staticmethod
```

Class methods. Are similar, but Python automatically passes the class (not an instance) to a class method's first (leftmost) argument, whether it is called through a class or an instance:

```
> >> Methods.cmeth(5) #

[<class 'allmethods.Methods'>, 5]   #  cmeth(Methods, 5)
> >> obj.cmeth(6) #

[<class 'allmethods.Methods'>, 6]   #  cmeth(Methods, 6)
```

> In Chapter 40, you'll also find that metaclass methods—an advanced and technically distinct method type used in the secondary class trees of types—behave similarly to the explicitly declared class methods we're exploring here.

中文翻译

> 为了把一个"既想通过类、也想通过实例调用"的无 self 方法指定清楚，类只需调用内建函数 staticmethod 与 classmethod 即可。前者把函数对象标记为"不需要 self"，后者标记为"需要一个类参数"。Example 32-7 演示了做法。

> Example 32-7. allmethods.py：

```
class Methods:
    def imeth(self, x):        #  self
        print([self, x])       #  self
    def smeth(x):              #
        print([x])             #
    def cmeth(cls, x):         #
        print([cls, x])        #
    smeth = staticmethod(smeth)  #  smeth  @
    cmeth = classmethod(cmeth)   #  cmeth  @
```

注意代码末尾两次赋值只是把 smeth、cmeth 这两个名字重新绑定（rebind）。一个 class 语句里任何一次赋值都会创建、改变属性，所以最后这两行只是把前面 def 建立的

属性盖过去而已。很快你会看到 @ 装饰语法就是这个过程的自动化替代形式——但要是你不先理解这里它自动化的"赋值形式"，那个语法就没法真正理解。

技术上说，Python 支持三种"与类相关的方法"，参数协议各不相同：- 实例方法（Instance methods）：第一个参数收一个 self 实例对象——这是默认。- 静态方法（Static methods）：不收取额外的实例对象（借 staticmethod）。- 类方法（Class methods）：第一个参数收一个类对象（借 classmethod；在元类里它则是固有的）。

此外，"类里的普通函数"在只通过类调时、不需任何额外协议即可扮演静态方法。allmethods.py 把三种方法都演示了一遍，现在我们逐个展开。实例方法是正常默认情形，必须用一个实例对象来调用。经由实例调用时，Python 自动把这个实例传给第一个（最左）参数；经由类调用时，你得手动把实例塞进去：静态方法则不需要实例参数。它们的名字只存在于各自类的类作用域里，可以继承查找。无实例函数经类调用本就不需要修饰，而 staticmethod 让同一函数也能经实例调用：类方法与静态方法类似，但 Python 会自动把"类"（而不是实例）传给它的第一个参数——无论它经由类还是实例被调用：在第 40 章你还会发现：元类方法（metaclass methods）——一种用在类型的专门类树中的高级、技术上独立的方法类型——在这里和我们显式声明的类方法行为非常相似。

深度理解

·核心对比：

方法类型	第一个参数收到什么	经实例调用	经类调用
实例方法	self（实例）	自动传入	需手动传入
静态方法	无	不传	不传
类方法	cls（类对象）	自动传类	自动传类

·底层实现：staticmethod 返回一个 staticmethod 描述符，只有 get 返回你给的那个函数（不绑定）；classmethod 返回 classmethod 描述符，get 返回 types.MethodType(cls, ...) 绑定到类。

·设计原因：三个协议各司其职——普通方法（面向实例）、静态方法（面向类内工具函数）、类方法（面向"以类为主体"的工厂与计数器）。

·实际问题：元类方法与类方法行为相似但不需 classmethod 声明——这是第 40 章元类的伏笔。

·初学者误区：把 staticmethod 和 classmethod 混为一谈。静态方法是"我以为它在类里，但它其实谁也不传"；类方法是"给我派來类的化身"。

用静态方法统计实例数（Counting Instances with Static Methods）

英文原文

> Now, given these built-ins, Example 32-8 codes the static method equivalent of this section's instance-counting example—it marks the method as special, so it never gets an instance passed to it automatically.

> Example 32-8. hackstatic.py

```
class Hack:
    numInstances = 0             #
    def __init__(self):
        Hack.numInstances += 1
    def printNumInstances():
        print('Number of instances:', Hack.numInstances)
    printNumInstances = staticmethod(printNumInstances)
```

Using the static method built-in, our code now allows the self-less method to be called through the class or any instance of it:

```
> >> from hack_static import Hack >> a, b, c = Hack(), Hack(), …

Number of instances: 3
> >> a.printNumInstances() #

Number of instances: 3
```

Compared to simply moving printNumInstances outside the class, as prescribed earlier, this version requires an extra staticmethod call (or an @ line). However, it also localizes the function name in the class scope; moves the function code; and allows subclasses to customize the static method with inheritance. The following subclass illustrates (this continues the prior session, so the count is already 3 at the start):

```
> >> class Sub(Hack):

...     def printNumInstances():            #
...         print('Extra stuff...')
...         Hack.printNumInstances()        #
...     printNumInstances = staticmethod(printNumInstances)
> >> a, b = Sub(), Sub() >> a.printNumInstances() #

Extra stuff...
Number of instances: 5
> >> Sub.printNumInstances()

Extra stuff...
Number of instances: 5
> >> Hack.printNumInstances() #

Number of instances: 5
```

> Moreover, classes can inherit the static method without redefining it—it's run without an instance, regardless of where it is defined:

```
> >> class Other(Hack): pass #  >> c = Other() >> c.pri…

Number of instances: 6
```

> Notice how this also bumps up the superclass' instance counter because its constructor is inherited and run—a behavior that begins to encroach on the next section's subject.

中文翻译

> 现在，借助这两个内建函数，Example 32-8 把本节"实例计数"例子改写为等价的静态方法版本——它把方法标记为静态，所以永远不会被自动传实例。

Example 32-8. hackstatic.py

```
class Hack:
    numInstances = 0             #
    def __init__(self):
        Hack.numInstances += 1
    def printNumInstances():
        print('Number of instances:', Hack.numInstances)
    printNumInstances = staticmethod(printNumInstances)
```

借由 staticmethod 内建函数，我们为无 self 方法打通了一条路：它既可通过类调用，也可通过实例调用：

```
>>> from hack_static import Hack
>>> a, b, c = Hack(), Hack(), Hack()
>>> Hack.printNumInstances()     #
Number of instances: 3
>>> a.printNumInstances()        # —
Number of instances: 3
```

和前面"把 printNumInstances 搬到类外"的处方相比，此版本多了一次 staticmethod 调用（或多一行 @ 语法）。但它把函数名字局部化于类作用域（不与模块中其他名字冲突）；把函数代码挪到使用处近旁（就在类内）；并且允许子类经由继承去定制它。下面的子类演示了这点（承接上一会话，所以起始计数已是 3）：

```
>>> class Sub(Hack):
...     def printNumInstances():            #
...         print('Extra stuff...')
...         Hack.printNumInstances()        #
...     printNumInstances = staticmethod(printNumInstances)
>>> a, b = Sub(), Sub()
>>> a.printNumInstances()                   #
Extra stuff...
Number of instances: 5
>>> Sub.printNumInstances()                 #
```

```
Extra stuff...
Number of instances: 5
>>> Hack.printNumInstances()                #
Number of instances: 5
```

> 此外，类可以不重定义就继承静态方法——无论定义在树中哪一位，它都不带实例运行：

```
>>> class Other(Hack): pass                 #
>>> c = Other()
>>> c.printNumInstances()
Number of instances: 6
```

> 注意：这同时抬高了超类的实例计数器——因为 Sub()/Other() 会调用被继承的 init 并让计数 +1——这个行为已经开始侵入下一节的话题（类方法）了。

深度理解

·核心概念：静态方法让"统计总体"的函数既可以放在类内、经类调用，也能经实例调用；子类覆盖后仍可回调超类版本。

·底层实现：Sub 的类创建过程中，printNumInstances = staticmethod(...) 再次绑定函数；子类的同名描述符按 MRO 遮蔽超类版本即可。

·设计原因：将"与类的数据相关"的加工以及时与继承链盘活——这就是静态方法对比模块层函数的最大价值。

·实际问题：计数干净的"工厂式"用法会随继承联动（构造子类实例也让超类计数+1），需要明确预期。

·初学者误区：以为覆盖静态方法要写 staticmethod；忘记写它，经实例调用子类版本就会 TypeError。

用类方法统计实例数（Counting Instances with Class Methods)

英文原文

> Interestingly, a class method can do similar work here—Example 32-9 has the same behavior as the static method version listed earlier, but it uses a class method that receives the instance's class in its first argument. Rather than hardcoding the class name, the class method uses the automatically passed class object generically.

> Example 32-9. hackclass.py

```
class Hack:
    numInstances = 0            #
    def __init__(self):
        Hack.numInstances += 1
    def printNumInstances(cls):
        print('Number of instances:', cls.numInstances)
    printNumInstances = classmethod(printNumInstances)
```

> This class is used in the same way as the prior versions, but its printNumInstances method receives the Hack class, not the instance, when called from either the class or an instance:

```
> >> from hack_class import Hack >> a, b = Hack(), Hack() >> a.…

Number of instances: 2
> >> Hack.printNumInstances() #

Number of instances: 2
```

When using class methods, keep in mind that they re ceive the most specific (i.e., lowest) class of the call's subject. That has some subtle implications when tryin g to update class data through the passed-in class. T o demo, Example 32-10 subclasses to customize the same way we did for static methods, and augments H ack.printNumInstances to also trace its class argume nt.

Example 32-10. hackclass2.py

```
class Hack:
    numInstances = 0                #
    def __init__(self):
        Hack.numInstances += 1
    def printNumInstances(cls):
        print('Number of instances:', cls.numInstances, cls)
    printNumInstances = classmethod(printNumInstances)
class Sub(Hack):
    def printNumInstances(cls):             #
        print('Extra stuff...', cls)
        Hack.printNumInstances()            #
    printNumInstances = classmethod(printNumInstances)
class Other(Hack): pass                     #
```

Running this in a REPL reveals that the lowest class is passed in whenever a class method is run—even for s ubclasses that have no class methods of their own:

```
> >> from hack_class2 import Hack, Sub, Other >> x = Sub() >> y…

Extra stuff... <class 'hack_class2.Sub'>
Number of instances: 2 <class 'hack_class2.Hack'>
> >> Sub.printNumInstances() #

Extra stuff... <class 'hack_class2.Sub'>
Number of instances: 2 <class 'hack_class2.Hack'>
> >> y.printNumInstances() #

Number of instances: 2 <class 'hack_class2.Hack'>
```

In the first call here, a class method call is made thro ugh an instance of the Sub subclass, and Python pass es the lowest class, Sub, to the class method. All is w ell in this case — since Sub redefinition calls the Hack superclass's version explicitly, so it receives its own cl ass. But watch what happens for an object that inheri ts the class method verbatim:

```
> >> z = Other() #  >> z.printNumInstances()

Number of instances: 3 <class 'hack_class2.Other'>
```

This last call here passes Other to Hack's class metho d. This works in this example because fetching the co unter finds it in Hack by class inheritance. But if this method tried to assign through the passed class, it w ould update Other, not Hack! In this specific case, Ha ck is probably better off hardcoding its own class na me to update its data — if it means to count instance s of all its subclasses too — rather than relying on th e passed-in class argument.

中文翻译

> 有意思的是，这里也能用类方法完成同样的工作——Example 32-9 与前面静态方法版行为相同，但它用的是"在第一个参数接收实例所属类"的类方法。它不硬编码类名，而是通用地使用那个自动传入的类对象。

> Example 32-9. hackclass.py

```
class Hack:
    numInstances = 0             #
    def __init__(self):
        Hack.numInstances += 1
    def printNumInstances(cls):
        print('Number of instances:', cls.numInstances)
    printNumInstances = classmethod(printNumInstances)
```

> 这个类与之前的版本用法一致，但它的 printNumInstances 无论经由类还是实例被调用，首次参数收到的都是 Hack 类，而不是实例：

> 使用类方法时请记住：它收到的是调用主体里最具体（最低层）的类。这会给"通过传入的类更新类数据"带来微妙影响。为了演示，Example 32-10 按上一节静态方法的做法子类化定制，并给 Hack.printNumInstances 增加类参数追踪。

> Example 32-10. hackclass2.py：

```
class Hack:
    numInstances = 0             #
    def __init__(self):
        Hack.numInstances += 1
    def printNumInstances(cls):
        print('Number of instances:', cls.numInstances, cls)
    printNumInstances = classmethod(printNumInstances)

class Sub(Hack):
    def printNumInstances(cls):              #
        print('Extra stuff...', cls)
        Hack.printNumInstances()             #
    printNumInstances = classmethod(printNumInstances)

class Other(Hack): pass                      #
```

在 REPL 里运行会发现：无论谁运行类方法，传入的都是最低层的那个类——哪怕子类自己没有定义任何类方法：

上面第一次调用经由 Sub 子类的实例触发，Python 把最低层的 Sub 传给了类方法。这里一切正常——因为 Sub 的重定义显式回调了 Hack 超类版本，超类版自然拿到自己的类。但请看"原样继承类方法"的对象会发生什么：

```
>>> z = Other()                 #
>>> z.printNumInstances()
Number of instances: 3 <class 'hack_class2.Other'>
```

最后一次调用把 Other 传给了 Hack 的类方法。它之所以还正确，是因为计数器经类继承能在 Hack 里被读到；但如果这方法想通过传入的类做赋值，就会更新 Other 而不是 Hack！在这个具体例子里，若 Hack 想连所有子类的实例都算进去，最好硬编码自己的类名来更新自己的数据，而不是依赖传入的类参数。

深度理解

·核心概念：类方法收到的是最低层类（lowest class），而非"定义这个方法的那一类"——这是它与"硬编码类名"最本质的分歧。

·底层实现：x.printNumInstances(cls) 绑定机制等价于 type(x).printNumInstances.get(None, type(x))，type(x) 即最具体类；因此任何继承者调用都拿到自己。

·设计原因：object.class 机制天然决定"谁调的、就给谁"，这让类方法适合"每个类各自维护自己状态"的场景。

·实际问题："读"还好（继承查找兜底），"写"就危险了——写入 cls.numInstances 会落在最具体类上，可能不是你以为的类。

·初学者误区：以为类方法的 cls 就是"定义它的那个类"。在继承/混入场景下，它常常是调用方的类。

用类方法实现"每类各计各的账"（Counting instances per class with class methods）

英文原文

> In fact, because class methods always receive the lowest class of an instance's tree: - Static methods and explicit class names may be a better solution for processing data local to a class. - Class methods may be better suited to processing data that may differ for each class in a hierarchy.

Code that needs to manage per-class instance counters, for example, might be best off leveraging class methods. To illustrate, the top-level superclass manages state that varies per class, similar in spirit to how instance methods manage state per instance.

Example 32-11. hackclass3.py

```
class Hack:
    numInstances = 0
    def count(cls):                      #
        cls.numInstances += 1            # cls
    def __init__(self):
        self.count()                     #  self.__class__  cou…
    count = classmethod(count)
class Sub(Hack):
    numInstances = 0
    def __init__(self):                  #  __init__
        Hack.__init__(self)
class Other(Hack):                       #  __init__
    numInstances = 0
```

When run, each subclass gets its own counter:

```
> >> from hack_class3 import Hack, Sub, Other >> x = Hack() >> …


(1, 2, 3)
> >> Hack.numInstances, Sub.numInstances, Other.numInstances


(1, 2, 3)
```

Static and class methods have additional advanced roles, which we will skip here. Later Pythons eventually added simpler designations via function decoration syntax, which we'll examine next. These can also augment classes, the topic of the next section.

中文翻译

> 事实上，正因为类方法总收到一棵实例树里的最低层类：- 要处理"归某类自己所有"的数据，静态方法 + 显式类名通常是更好的答案。- 要处理"树中每个类各不相同"的数据，类方法可能更合适。例如，"按类维护的实例计数器"这类代码，用类方法往往最佳。为了说明，顶层超类用它管理"每个类各自存储、各自不同"的状态信息——在精神上，就像实例方法管理"每个实例各不相同"的状态一样。

> Example 32-11. hackclass3.py：

```
class Hack:
    numInstances = 0
    def count(cls):                         #
        cls.numInstances += 1               # cls
    def __init__(self):
        self.count()                        #  self.__class__  cou…
    count = classmethod(count)

class Sub(Hack):
    numInstances = 0
    def __init__(self):                     #  __init__
        Hack.__init__(self)

class Other(Hack):                          #  __init__
    numInstances = 0
```

> 运行后，每个类拥有自己的计数器：

```
>>> from hack_class3 import Hack, Sub, Other
>>> x = Hack()
>>> y1, y2 = Sub(), Sub()
>>> z1, z2, z3 = Other(), Other(), Other()
>>> x.numInstances, y1.numInstances, z1.numInstances   #
(1, 2, 3)
>>> Hack.numInstances, Sub.numInstances, Other.numInstances
(1, 2, 3)
```

> 静态方法与类方法还有更多高级用法，这里略过。较新的 Python 版本后来通过函数装饰语法让这些声明更简单，同时提供了更多能力——而这正是下一节的主题。

深度理解

·核心概念："每类一账本"：self.count() 天然把"最具体类"传入 count，从而把计数落在正确的类上。

·底层实现：self.class 就是最具体类；cls.numInstances += 1 在有同名类属性时直接修改当前类的数据（分配），若该类没有则沿继承链找。

·设计原因：类方法让"类级多态"成为可能——同一套 init 代码针对不同子类做不同记账。

·实际问题：框架的"模型注册表""按类缓存"都用这个模式幂等每个子类的专属数据。

·初学者误区：把 count 和 init 拆开放或都命名 count；忘记子类需初始化自己的计数器会导致"子类实例增的是超类计数"。

方法与函数的一个结语（The methods finale）

英文原文

> NOTE — The methods finale: For a postscript on Python's method types, be sure to watch for coverage of metaclass methods in Chapter 40 — because these are designed to process a class that is an instance of a metaclass, they turn out to be very similar to the class methods defined here but require no classmethod declaration, and apply only to the shadowy metaclass realm previewed next.

中文翻译

> 注意——方法的"终场哨"：关于 Python 的方法类型，作一个"写在扉页后的话"：第 40 章将覆盖"元类方法"（metaclass methods）——因为它们被设计为处理"作为元类实例的类"，所以它们与我们这里定义的类方法极为相似，却无需 classmethod 声明，并且只适用于下一节将要预览的那个暗影中的元类领域。

深度理解

·核心概念：类方法（classmethod）与元类方法殊途同归，仅差"要不要显式声明"。

·底层实现：元类的方法天然以"类（作为元类的实例）"为第一参数，等于 Python 层面的 classmethod。

·设计原因：元类是"面向类的类"」，其方法签名自然面向类。

·实际问题：让读者把"现在的内容"与"第 40 章"串起来，

建立连续的学习地图。

·初学者误区：在还不懂元类时硬记"元类方法不需要 classmethod"——等学完元类自然就通了。

32.9 装饰器与元类（Decorators and Metaclasses）

英文原文

Because the staticmethod and classmethod call technique described in the prior section initially seemed obscure to some observers, a device was eventually added to make the operation simpler. Python decorators — similar to the notion and syntax of annotations in Java — both address this specific need and provide a general tool for adding logic that augments functions and classes or later calls to them. This is called a "decoration," but in more concrete terms is really just a way to run extra processing steps at function and class definition time with explicit syntax. It comes in two flavors: Function decorators: the initial entry, augment function definitions at def statements. They specify operation modes for both simple functions and classes' methods by wrapping them in an extra layer of logic implemented as another function, often called a metafunction (a "function of functions"). They were introduced in 2.4/2.5. Class decorators: a later extension, augment class definitions at class statements. The

y wrap classes in a similar way, adding support for m anagement of whole objects and their interfaces. We met decorators very briefly in Chapter 19 in relation t o simple functions, but they are more general than e arlier implied: they can also be used for class method s and whole classes, and can add arbitrarily little logi c to functions and classes that go well beyond the st atic- and class-method roles used as a segue here. Fu nction decorators may be used to plug in tracing logi c, argument validation, timing, and so on, and can be used to manage either functions themselves or later calls to them. In the latter mode, function decorators are similar to the delegation pattern we explored in C hapter 31, but they are designed to augment a specif ic function or method call. Python provides a few buil t-in function decorators for such operations—markin g static and class methods, defining properties (as sk etched earlier, the property built-in works as a decor ator automatically)—but programmers can also code arbitrary decorators of their own. Although not strictl y, user-defined function decorators are often coded a s classes to save the original functions and other dat a. This proved such a useful hook that it was extende d: class decorators bring the same augmentation to c lasses and are more directly tied to the class model. L ike their function counterparts, class decorators may manage classes themselves or later instance-creation calls, and often employ delegation of entire interface

s in the latter mode. Their roles also often overlap with metaclasses but are a more lightweight way to achieve some goals.

中文翻译

上一节 staticmethod/classmethod 的调用手法，对某些读者来说起初显得有些晦涩，于是后来加入了一个让操作更简单的装置。Python 的 decorators（装饰器）——与 Java 中"注解"（annotations）的概念和语法类似——既补上了这一具体需求，又提供了一个通用工具：为函数、类、或对它们后续的调用添加管理逻辑。它名叫"装饰"，但说得更实在些，它其实是一种"在函数/类定义时刻以显式语法运行额外处理步骤"的方式。它分为两派：- 函数装饰器（Function decorators）：最初的功能，在 def 语句处增强函数定义。它们通过把函数包进"由另一个函数实现的额外逻辑层"来为简单函数与类方法指定运行模式——那个函数常被称为"元函数（metafunction）"，其实只是个称谓问题。（这份历史说明原文如此——重要是掌握用法。）- 类装饰器（Class decorators）：后来的扩展，在 class 语句处增强类定义。它们以类似方式包裹类，为的是管理整个对象及其接口，而非单个函数。我们在第 19 章就简单场合见过装饰器，但它比当时暗示的更通用：也能用于类方法和整个类，还能给函数与类加上远超"静态/类方法角色"的任意逻辑。例如，函数装饰器可以为函数添加调用日志、参数类型检查、计时等；可采用"管理函数本身"或"管理后续对它的调用"两种模式。后者与第 19 章学过的委托（delegation）模式相似，只是它增强的是某个具体函数/方法调用，而非整个对象接口。Python 自带少量内建函数装饰器用于操

作：标记静态方法、类方法，定义 property（如前面草图的，property 内建函数自动可作装饰器）。程序员当然也能自编任意装饰器。虽然严格说不是非得如此，但用户自定义装饰器常被写成"类"，以便把原函数存起来供日后分发、并附带其他状态信息。这个钩子太有用了，于是最终又扩展出——类装饰器：把同样的增强带到"类"身上，并且与类模型耦连得更直接。和函数装饰器一样，类装饰器既能管理类自身，也能管理"后续创建实例的调用"，后者模式下常对整个接口实施委托。它们的角色常与元类重叠，但实现一些目标时类装饰器是更轻量的手段。

深度理解

·核心概念：装饰器 = "定义时刻挂钩子"，函数装饰器管函数，类装饰器管类（及其实例创建）。

·底层实现：@deco 等价于 name = deco(name)；装饰器返回的任何对象（原函数、代理对象）替换掉原名字。

·设计原因：把"横切逻辑"（日志、校验、注册）从函数体内剥离，让业务代码只关注自身。

·实际问题：Python 内置 @staticmethod、@classmethod、@property；框架里到处是自定义装饰器（路由、鉴权、缓存）。

·初学者误区：以为装饰器"必须返回原函数"。实际上返回任意可调用对象即可——返回代理对象正是跟踪、计次类装饰器的常见做法。

函数装饰器基础（Function Decorator Basics）

英文原文

> Syntactically, a function decorator is a sort of runtime declaration about the function that follows it. A funct ion decorator is coded on a line by itself just before t he def statement that defines a function or method. I t consists of the @ symbol, followed by a metafuncti on — a plain function (or other callable object) of on e argument, which is passed and manages that functi on. The code following the @ is usually the name of a metafunction with an optional argument list, but as of Python 3.9 it can be any expression returning a on e-argument function. Listing multiple decorators on c onsecutive lines allows them to nest. For example, th e prior section's methods may be coded with decorat or syntax like this:

```
class C:
    @staticmethod            #
    def meth():
        ...
```

> Internally, this syntax has the same effect as the follo wing — passing the function through the decorator a nd reassigning the result back to the same name:

```
class C:
    def meth():
        ...
    meth = staticmethod(meth)    #
```

> Decoration rebinds the method name to the decorator's result. The net effect is that calling the method's name later triggers the decorator's result first. Because a decorator can return any sort of object, this allows the decorator to insert a layer of logic to be run on every later call. The decorator function is free to return the original function or a new proxy object that saves the original to be invoked after the extra logic runs.

中文翻译

> 从语法上讲，函数装饰器是关于它后面那个函数的"运行时声明"。它单独占据一行、写在定义函数/方法的 def 语句正前方。它由一个 @ 符号加一个元函数（一个接收一个参数的普通函数或其他可调用对象，负责接收并管理被装饰函数）组成。@ 后面通常是一个带可选参数列表的元函数名；自 Python 3.9 起，也可以是"返回单参数函数"的任意表达式。多行排列多个装饰器时它们会层层嵌套。例如，上一节的方法可以用这样的装饰器语法来写:

```
class C:
    @staticmethod            #
    def meth():
        ...
```

> 内部来说，这行语法与下面这句同效——把函数经过装饰器处理后，把结果重新绑定回同名：

```
class C:
    def meth():
        ...
    meth = staticmethod(meth)    #
```

> 装饰这个动作把方法名重绑定到装饰器的结果。净效果：你之后调用方法名，其实先触发了它被装饰后的结果。因为装饰器能返回任意对象，它便得以在每次后续调用里插入一层逻辑——既可以原样返回原函数，也可以返回一个保存原函数的代理（proxy）对象，待额外逻辑跑完后再间接调用原函数。

深度理解

·核心概念：@deco 是 name = deco(name) 的语法糖；装饰器在 def 完成后、绑定名字前被调用一次。

·底层实现：def meth 生成函数对象 → 装饰器表达式求值并调用 → 结果赋给 meth。这要求装饰器在"定义时刻"完成，而非"调用时刻"。

·设计原因：复用"应用另一函数于函数"的操作，避免每个用法各写一遍样板代码——Python 3.9 还允许任意表达式做装饰器。

·实际问题：装饰器多层嵌套时的执行次序（从下往上应用，从上往下执行）必须清楚，才能分析组合效果。

·初学者误区：以为 @staticmethod 是魔法；它只是"显式一点、少写一行赋值"的语法糖。

用户自定义装饰器初探（A First Look at User-Defined Function Decorators）

英文原文

Although Python provides a handful of built-in functions that can be used as decorators, we can also write

custom decorators of our own. Because of their wide utility, we're going to devote an entire chapter to co ding decorators later. As a quick example, though, let 's look at a simple user-defined decorator at work. Re call that the call operator-overloading method imple ments a function-call interface for class instances. Ex ample 32-14 uses this to code a call proxy class that saves the decorated function in the instance and catc hes calls to the original name. Because this is a class, it also has state info—a counter. Example 32-14. trac er1.py

```
class tracer:
    def __init__(self, func):        #
        self.calls = 0
        self.func = func
    def __call__(self, *args):       #
        self.calls += 1
        print(f'call {self.calls} to {self.func.__name__}')
        return self.func(*args)

@tracer                              #  hack = tracer(hack)
def hack(a, b, c):                   #  hack
    return a + b + c

if __name__ == '__main__':
    print(hack(1, 2, 3))             #  tracer
    print(hack('a', 'b', 'c'))       #  __call__
```

Because the hack function is wrapped by the tracer d ecorator, when the original hack name is called, it act ually triggers the call method in the class. That meth od counts and logs the call, then dispatches it to the

original wrapped function. Note how the name argument syntax is used to pack and unpack the passed-in arguments; because of this, the decorator can be used with a function of any arbitrary number of positional arguments. The net effect, again, is to add a layer of logic to the original hack function. When run, the first output line comes from the tracer class, and the second gives the return value of the hack function:

```
$ python3 tracer1.py
call 1 to hack
6
call 2 to hack
abc
```

Trace through and note: this decorator works for any function that takes positional arguments, but it doesn't handle keyword arguments and cannot decorate class-level method functions (in short, for methods, its call would be passed a tracer instance only). Part VIII will show you more decorator patterns, some are better suited to methods. For example, using nested functions with enclosing scopes for state (instead of callable class instances) makes decorators broadly applicable to methods too. Example 32-15 provides a brief look at this closure-based model; it uses a function attribute for counter state, though variables and nonlocal also work. Example 32-15. tracer2.py

```
def tracer(func):                   #
    def oncall(*args):              #
        oncall.calls += 1
        print(f'call {oncall.calls} to {func.__name__}')
        return func(*args)
    oncall.calls = 0
    return oncall

class C:
    @tracer
    def hack(self, a, b, c): return a + b + c

if __name__ == '__main__':
    x = C()
    print(x.hack(1, 2, 3))
    print(x.hack('a', 'b', 'c'))
```

> The example's output is the same as its predecessor but reflects a decorated class method; more on this later.

中文翻译

> Python 提供了一小批内建函数可作为装饰器，但我们也能自编装饰器。鉴于它们用途之广，我们稍后会用一整章来写装饰器。这里先看一个简单的用户自定义装饰器工作中的样子。还记得吗：call 运算符重载方法为"类实例"实现了"函数调用"接口。Example 32-14 就用它写了一个"调用代理"类：把被装饰的函数存进实例、并截获对原名的调用。由于用的是类，它还能带状态信息——一个调用计数器。Example 32-14. tracer1.py

```
class tracer:
    def __init__(self, func):        #
        self.calls = 0
        self.func = func
    def __call__(self, *args):       #
        self.calls += 1
        print(f'call {self.calls} to {self.func.__name__}')
        return self.func(*args)

@tracer                              #  hack = tracer(hack)
def hack(a, b, c):                   #  tracer  hack
    return a + b + c

if __name__ == '__main__':
    print(hack(1, 2, 3))             #  tracer
    print(hack('a', 'b', 'c'))       #  __call__
```

因为 hack 函数经过 tracer 装饰器，所以当原名 hack 被调用时，实际触发的是类里的 call 方法。它记数并记录日志，然后把调用分派给被包裹的原函数。注意 args 参数语法用来打包、解包传入的参数；因此这个装饰器可以包裹"任意数量位置参数"的函数。净效果还是那句：给原 hack 函数加了一层逻辑。运行时第一行输出来自 tracer 类，第二行是 hack 函数自己的返回值：请亲手跑通该示例的代码，你会收获更多。就现在这样，它适用于任何"接收位置参数"的函数，但不处理关键字参数，也不能装饰类级方法函数（简要地说，对方法而言，传给 call 的只有一个 tracer 实例）——第八部分会介绍多种装饰器编写方案，其中有些比这份更适合方法。例如，用嵌套函数 + 闭合作用域携带状态来写装饰器，就能广泛适配类方法。Example 32-15 让你一窥这种闭包式写法；它用函数属性装计数状态，也可以改用变量 + nonlocal。

Example 32-15. tracer2.py

```
def tracer(func):                  #
    def oncall(*args):             #
        oncall.calls += 1
        print(f'call {oncall.calls} to {func.__name__}')
        return func(*args)
    oncall.calls = 0
    return oncall

class C:
    @tracer
    def hack(self, a, b, c): return a + b + c

if __name__ == '__main__':
    x = C()
    print(x.hack(1, 2, 3))
    print(x.hack('a', 'b', 'c'))
```

> 它与前例输出相同，但装饰的是一个类方法；具体细节我们后文再讲。

深度理解

·核心概念：装饰器的两种承载形态——可调用类对象（带属性状态）与闭包（带环境状态）。tracer1 是类式、tracer2 是闭包式。

·底层机制：oncall.calls = 0 在函数对象上挂属性（函数也是对象）；闭包里 func 由外层作用域捕获。两者都能保存"计数"状态。

·设计原因：类式便于开发者阅读/内省（状态就是属性），闭包式更轻、更易支持方法装饰——因为 oncall.get 的绑定由 descriptor 自动处理。

·实际问题：类式 tracer 装饰方法会收到 tracer 实例天然做 self，导致方法签名错乱；闭包式（此例）则仍正确。

·初学者误区：以为装饰器必须返回函数；返回"任何可调用对象（包括类的实例）"同样成立——tracer1 就是证明。

类装饰器与元类初探（A First Look at Class Decorators and Metaclasses）

英文原文

> Python later generalized decorators, allowing them to be applied to classes as well as functions. In short, class decorators are similar to function decorators, but they are run at the end of a class statement to rebind a class name to a callable. As such, they can be used to either manage classes just after they're created or insert a layer of wrapper logic to manage instances when later created. Symbolically:

```
def decorator(aClass): ...
@decorator
class C: ...
```

> is mapped to the following equivalent:

```
def decorator(aClass): ...
class C: ...
C = decorator(C)
```

> The class decorator is free to augment the class itself or return a proxy object that intercepts later instance construction calls. For example, in "Counting instance

s per class with class methods" we could use this hoo k to automatically augment classes with instance cou nters:

```
def count(aClass):
    aClass.numInstances = 0
    return aClass                   #

@count
class Hack: ...                     #  Hack = count(Hack)

@count
class Sub(Hack): ...                #  numInstances = 0
```

In fact, as coded, this decorator can be applied to cla sses or functions—it happily returns the object being defined in either context after initializing the attribut e:

```
@count
def hack(): pass                     #  hack = count(hack)
@count
class Hack: pass                     #  Hack = count(Hack)
hack.numInstances                    #  0
Hack.numInstances
```

As detailed later, class decorators can also manage a n object's entire interface by intercepting constructio n calls and wrapping the new instance for intercept — the multi-level technique we'll use for class attribu te privacy in Chapter 39.

```
def decorator(cls):                  #  @
    class Proxy:
        def __init__(self, *args):  #  cls
            self.wrapped = cls(*args)
        def __getattr__(self, name):  #
            return getattr(self.wrapped, name)
    return Proxy

@decorator
class C: ...
X = C()                              #  Proxy CX.attr
```

Finally, metaclasses are a similarly advanced class-ba sed tool whose roles often intersect with those of cla ss decorators. They route the creation of class object s to a subclass of the top-level type class, at the conc lusion of a class statement:

```
class Meta(type):
    def __new__(meta, classname, supers, classdict):
        #  +  type
        ...
class C(metaclass=Meta):
    ...
#  Meta
```

Python calls a class's metaclass to create the new clas s object, passing the data defined during the class st atement's run; if omitted, the metaclass defaults to t he type class. Abstractly, at the end of a class statem ent with an explicit metaclass:

```
classname = Meta(classname, superclasses, attributedict)
```

To manage the creation or initialization of a new clas s object, a metaclass generally redefines the new or i

nit method of the type class that intercepts this call by default. The net effect is to define code run automatically at class creation time.

> Both schemes — class decorators and metaclasses — are free to augment a class or return an arbitrary object to replace it. As you'll learn later, metaclasses may also define methods that process their instance classes rather than instances created from them — similar to class methods. Such mind-bending concepts require Chapter 40's conceptual groundwork (and — perhaps — sedation).

中文翻译

> Python 后来把装饰器推广到"类"上，允许它对类（同样对函数）进行包装。简言之，类装饰器与函数装饰器类似，只不过它们在 class 语句结尾运行，把类名重新绑定到一个可调用对象上。由此，可用来"管理刚创建出来的类"，或"插入一层管理逻辑来控制类后续创建的实例"。符号意义如下：

```
def decorator(aClass): ...
@decorator
class C: ...
```

等价于：

```
def decorator(aClass): ...
class C: ...
C = decorator(C)
```

> 类装饰器既能直接增强类自身，也可以返回一个"拦截随后实例构造调用"的代理对象。例如，在前文"用类方法按类计

数"的例子中，就能用这钩子自动给类装上计数器：

```
def count(aClass):
    aClass.numInstances = 0
    return aClass                   #

@count
class Hack: ...                     #  Hack = count(Hack)

@count
class Sub(Hack): ...                #  numInstances = 0
```

事实上，如上实现，这个装饰器既适用于函数也适用类——无论哪个上下文，它都初始化环境后把传入对象原样返回：

```
@count
def hack(): pass                    #  hack = count(hack)
@count
class Hack: pass                     #  Hack = count(Hack)
hack.numInstances                     #  0
Hack.numInstances
```

像后面要详述的那样，类装饰器还能管理一个对象的整个接口：拦截构造调用、用代理包装新实例、再用属性访问器拦截后续请求——这个用第 39 章实现类属性私有化的多级技巧：

```
def decorator(cls):                 #  @
    class Proxy:
        def __init__(self, *args):  #  cls
            self.wrapped = cls(*args)
        def __getattr__(self, name):  #
            return getattr(self.wrapped, name)
    return Proxy

@decorator
```

```
class C: ...
X = C()                              #  C  Proxy X.attr
```

最后，前面提到的元类（metaclasses）也是同为高级的"基于类"工具，其作用常与类装饰器交叉。它们把"类的创建"路由到顶级 type 类的一个子类，发生在 class 语句完成之时：

```
class Meta(type):
    def __new__(meta, classname, supers, classdict):
        ...                          #  +  type
class C(metaclass=Meta):
    ...
                                     #  Meta
```

Python 调用类的元类来创建新类对象，并传入 class 语句运行期间定义的数据；若省略 metaclass，缺省为 type 类。抽象地看，带显式元类的 class 语句结束时举行：

```
classname = Meta(classname, superclasses, classdict)
```

要管理新类对象的"创建/初始化"，元类一般重定义 type 类的 new 或 init 方法，从而默认拦截这个调用。这样定义出来的代码，会在类创建时刻自动运行。和类装饰器一样：两者都能增强类、或返回任意对象替换它——元类还可定义处理它们的"实例类"的方法，那与类方法异曲同工。这些烧脑概念，必须留到第 40 章的理念奠基（也许外加镇定剂）再展开。

深度理解

·核心概念：类装饰器（类语句末运行、可替换类）vs 元类（type 的子类接管类创建）。两者是"类定制"的两个入口。

·底层实现：@count 装饰的 class 语句，在类创建完成后调用 count 并重绑定；元类则把整个"建类调用"重定向到 Meta(...)。

·差异与重叠：类装饰器不需要元类基类、更轻量；元类能全程接管创建（如改 new）与名称、基类参数，能力更强。

·适用场景：路由/注册表/编译期检查用类装饰器已足够；ORM 或框架内部用元类实现"自动注册 + 自定义创建"。

·初学者误区：妄想在 init 之前拦截——要用元类的 new；两者产生的"类属性里凭空多字段"，也常让新手以为出 bug。

更增详情（For More Details)

英文原文

Naturally, there's more to the decorator and metacla ss story than shown here. Although they are a genera l mechanism whose usage may be required by some packages, coding new user-defined decorators and metaclasses is an advanced topic of interest primarily to tool writers, not application programmers. Becaus e of this, the book defers coverage until its final opti onal part: Chapter 38 covers data properties via funct ion decorators; Chapter 39 focuses on decorators wit h more comprehensive examples; Chapter 40 covers metaclasses and the class/instance management stor y. Although these chapters cover advanced topics, th

ey'll also give us a chance to see Python at work in substantial examples. For now, let's move on to our final class-related topic.

中文翻译

当然，装饰器与元类的故事远不止这些。虽然它们是一般机制、某些包可能必须用到，但"编写新的用户自定义装饰器与元类"仍属游戏高级主题，主要面向工具作家而非应用程序员。所以本书把它们的扩展覆盖放到最后可选部分：第 38 章细讲用函数装饰器写属性；第 39 章聚焦装饰器并给出更综合的例子；第 40 章覆盖元类和类/实例管理的故事。这些章节虽然覆盖高级主题，但也给了我们机会，在分量十足的例子里看到 Python 如何工作。眼下，就让我们移步本章最后一个与类相关的主题。

深度理解

·核心概念：装饰器/元类属于"元编程"，本默认给工具作者而非应用玩家；阅读者需识得、不需滥写。

·学习路径：第 38（属性）→ 39（装饰器）→ 40（元类）是它们的三大主场。

·设计原因：把高级专题留在书的最后与可选部分，主书不针对它们深入，才不会吓退初学者。

·实际问题：包内"请求注册、参数校验、接口代理"常见装饰器痕迹，读懂它们解开框架谜团。

·初学者误区：读不了几行就是乱给既有工具加装饰——工具错与理解错一样伤人。

32.10 super 函数的完整故事（The super Function）

英文原文

To close out this chapter, we turn to super: a built-in function that can be used to both reference superclass attributes implicitly without naming a superclass explicitly and route method calls coherently in multiple-inheritance trees. So far, super has been called out in the sidebar "The super Alternative," as well as multiple notes along the way, but has not appeared in code. This was by design: super comes with substantial complexities and downsides that make it difficult to recommend to learners. In short, it's an all-or-nothing tool with several arduous coding requirements and relies on special-case and wildly implicit semantics that run counter to Python norms. The super call also tends to be abused for "Java-fication" of Python code. Newcomers with backgrounds in Java often rush to use Python's super simply because of its similarity to a Java tool but are unaware of its much more subtle implications in Python's multiple inheritance—until adding superclasses breaks their programs. Nevertheless, this call has grown pervasive in Python code and merits further elaboration, especially for those opting to use it naively. Hence, this section both covers super usage and notes its pitfalls along the way. Ultimately, t

hough, the merit of this call, like everything else presented in this chapter and book, is ultimately yours to decide.

中文翻译

作为本章的收官，我们转向 super：这个内建函数既能"不显式点名超类"地隐式引用超类属性，也能在多重继承树中协调地路由方法调用。到目前为止，super 只在边栏"The super Alternative"和沿途的若干注释里被提到，从没在代码里出现过。这是刻意为之：super 带着实质性的复杂与短板，很难推荐给初学者。简言之，它是一个"全有或全无"（all-or-nothing）的工具，伴随好几条艰辛的编码要求，而且依赖"特例化 + 极度隐式"的语义——这与 Python 的惯例相悖。super 调用还常被滥用来把 Python 代码"Java 化"。有 Java 背景的新手往往只因它与 Java 里的某个工具相似，就急着用 Python 的 super，却不知道它在 Python 多重继承里的微妙含义——直到加入超类弄坏了他们的程序。尽管如此，这个调用在 Python 代码中已无处不在，值得进一步阐明，尤其对那些打算"天真地"使用它的人。因此，本节既覆盖 super 的用法，也沿途指出它的陷阱。但最终，这个调用的价值——就像本章与全书里呈现的其他一切事物一样——由你自己定夺。

深度理解

·核心概念：super 是个"隐式、魔法"的工具：不用点名超类、不用传 self，却知道该去哪找属性和方法。

·作者立场：作者反复提示它"全有或全无""隐藏语义""易被 Java 习惯带偏"——这是阅读全文的前提心态。

·底层实现：super() 无参形式依赖调用栈与 class 闭包变量，把"包含类 + self"配对成代理对象。

·实际问题：框架与库代码里 super 无处不在；读得懂它，是基本功；写不写它，是风格与风险权衡。

·初学者误区：把 super 当成"Java super"理解，忽略了它实际沿"调用者的 MRO"而非"静态继承链"查找。

super 基础（The super Basics）

英文原文

> First off, let's review the explicit alternative that's mo re closely in step with Python's own idioms. As we've seen so far, it's always possible to reference a supercl ass's attributes—whether method or data—by namin g their desired source class explicitly:

```
> >> class C:

...     def act(self):
...         print('hack')
> >> class D(C):

...     def act(self):
...         C.act(self)          #  self
...         print('code')
> >> I = D() >> I.act()

hack
code
```

> In single-inheritance trees like this one, the super alt ernative seems relatively straightforward at first glan

ce: its most common form automatically selects the calling class's superclass generically and implicitly when an attribute is later fetched. An explicit call like class.method(self) becomes an implicit super().method(), as in our example:

```
> >> class C:

...     def act(self):
...         print('hack')
> >> class D(C):

...     def act(self):
...         super().act()        #  self
...         print('code')
> >> I = D() >> I.act()

hack
code
```

This works as advertised and may minimize work—you don't need to list self in the call, don't need to update the call if D's superclass changes in the future, and don't need to code long superclass names or package-import paths.

中文翻译

首先，让我们回顾更贴合 Python 自身习惯的显式替代方案。如我们迄今所见，始终可以通过"点名期望的来源类"来引用超类的属性——无论是方法还是数据：

```
>>> class C:
...     def act(self):
...         print('hack')
>>> class D(C):
...     def act(self):
...         C.act(self)          #  self
...         print('code')
>>> I = D()
>>> I.act()
hack
code
```

在单继承树里，super 替代方案乍看相当直白：它最常见的形态，会在之后获取属性时，自动"通用且隐式地"选定"包含 super 调用的那个类"的超类。像 class.method(self) 这样的显式调用，在我们的例子里变成了隐式的 super().method()：

```
>>> class C:
...     def act(self):
...         print('hack')
>>> class D(C):
...     def act(self):
...         super().act()        #  self
...         print('code')
>>> I = D()
>>> I.act()
hack
code
```

它按宣传的那样工作，还能减少工作量——调用时不用写 self、将来 D 的超类变了也不用改调用、更不用敲长超类名或包导入路径。

深度理解

·核心概念：显式 C.act(self) 与隐式 super().act() 在单继承下行为等价，但后者隐藏了"去哪个类找"。

·底层实现：无参 super() 利用"当前方法所属类"（class 闭包变量）+ 第一个实参（self）构造代理。

·设计原因：少写代码、多写意图——"父类的方法"，而非"C 类的方法"；一旦改名/移包，显式版要全改。

·实际问题：单继承、明确需求（先父后己）时 super 是安全的减法；问题只出现在它被用于多重继承。

·初学者误区：以为 super().act() 是"调用 D 的直接父类 C 的 act"。在多重继承下，它可能调到 C 的兄弟类！

super 细节（The super Details）

英文原文

If you study the preceding code closely, you'll realize that there's something odd going on here. The super call somehow knows about the class, its superclass, and the self instance, even though none are present in its call. The backstory involves MROs, a proxy, and an algorithm. A "magic" proxy. To understand how super works, you need first to be fluent in the MRO algorithm covered in "Multiple Inheritance and the MRO"—and you should review that now if you gave it a pass. The MRO is both a firm prerequisite and nested component of super. Once you've mastered the MRO,

the simplest description of super is this: when used i
n a class method, super returns a proxy object that w
ill locate an attribute in a class following that of the c
ontaining class in the MRO of the self instance's class
. The net effect finds an attribute in a superclass or ot
her relative of the class containing the call. This work
s as expected in single-inheritance trees because the
superclass naturally follows the containing class on s
elf's MRO. Really, though, this relies on deep magic.
Apart from the MRO itself, super works by inspecting
: - The runtime call stack info for the calling method's
arguments - The class variable internally added to th
e closure of methods that call super The combo auto
matically locates both the self argument and the clas
s containing the super call and then pairs the two in
a special proxy object that routes later attribute fetch
es to a superclass's version of a name. In fact, the co
mmon no-argument super form is equivalent to man
ually passing in the class containing the call and the s
elf instance:

```
super()
super(class-containing-the-super-call, method-self-argument)
```

Both forms can be used; manual arguments may be h
andy in some roles. Because the second may be hard
er to code and maintain than explicit class-name refe
rences, though, Python roots out the class and instan
ce for you behind the scenes:

```
class D(C):
    def act(self):
        super(D, self).act()     #  super().act()
        print('code')            #  D  __class__
```

If that all sounds complicated and strange, it's becau se it is. Due to its unusual semantics, the no-argumen t super call form doesn't work at all outside a metho d:

```
> >> super

<class 'super'>
> >> super() #

RuntimeError: super(): no arguments
```

And where it does work, its implicit pairing of class a nd instance is nowhere to be found in your code:

```
> >> class E(C):

...     def method(self):
...         proxy = super()      # self  super ——
...         return proxy
> >> prx = E().method() #  >> prx

<super: <class 'E'>, <E object>>
> >> prx.act() #  MRO  E  act self

hack
```

To be sure, this call's semantics resemble nothing els e in Python—it's neither a bound nor unbound meth od and fills in a class and self even though you omit both in the call. This deviates from Python's explicit s elf policy, which holds true everywhere else. By hidin

g the instance, super violates this fundamental Python idiom for a single role.

中文翻译

若你细看前文代码，会发现这里有点怪：super 调用明明什么参数都没给，却"知道"那个类、它的超类、以及 self 实例。它的背后故事，涉及 MRO、一个代理（proxy）与一个算法。一个"魔法"代理。要理解 super 的工作原理，你首先得精通"多重继承与 MRO"一节里讲过的 MRO 算法——如果你当时跳过了，现在请回去复习。MRO 既是 super 的硬性前提，也是其嵌套组件。掌握 MRO 后，super 最简描述是这样的：在类方法中使用时，super 返回一个代理对象，它会在"self 实例的类的 MRO"中，于"包含 super 调用那个类"之后的类里查找属性。净效果就是在包含调用的那个类的超类（或其他"亲戚"）里找到属性。单继承树里这符合直觉，因为超类天然排在 self 的 MRO 里包含类的后面。但说真的，这依赖的是"深层魔法"：除了 MRO 本身，super 还靠检视——- 调用栈的运行时信息（调用方法的实参）- class 变量——它会被自动添加进"调用 super 的方法"的 closure（闭包单元）里这套组合自动定位"self 实参"与"包含 super 调用的类"，并把两者配成特殊代理对象，把后续属性获取路由到某个名字的超类版本。事实上，常用的无参 super 形式，等价于手动传入"包含调用的类 + self 实例"：

```
super()
super(class-containing-the-super-call, method-self-argument)
```

两种形式都可用；某些角色下手动参数或许很方便。但由于第二种可能比"显式类名引用"更难写、更难维护，Python

在幕后替你把类与实例挖了出来：

```
class D(C):
    def act(self):
        super(D, self).act()     #  super().act()
        print('code')            #  D  __class__
```

如果这一切听起来又复杂又奇怪——那正是因为它们确实如此。因为语义特殊，无参 super() 在"方法之外"根本不可用：

```
>>> super
<class 'super'>
>>> super()                      #
RuntimeError: super(): no arguments
```

而在能用的地方，它对"类与实例的隐式配对"在你的代码里也无迹可寻：

```
>>> class E(C):
...     def method(self):
...         proxy = super()      # self  super —
...         return proxy
>>> prx = E().method()           #
>>> prx
<super: <class 'E'>, <E object>>
>>> prx.act()                    #  MRO  E act self …
hack
```

坦白说，这个调用的语义在 Python 里再无第二家——它既非绑定方法也非未绑定方法，明明两个参数都被你省略，它却替你补上了类与 self。这偏离了 Python 处处贯彻的"显式 self"原则。super 为了这一个角色，就违反了 Python 的这一基本惯用式。

深度理解

·核心概念：super 的"魔法"= MRO + 调用栈 + class 闭包变量三合一，生成"越过包含类"的代理。

·底层实现：CPython 里 super() 通过 PyObjectFastCall 前的栈帧（frame->fcode 和 flocals）找到 self 与 class；super(type, obj) 的 getattribute 沿 obj.mro 从 type 之后开始搜。

·设计原因：省去"点名类"的重复劳动，把 MRO 交给统一算法，为"协作式分发"铺路。

·实际问题：在方法外部 super() 抛 RuntimeError；代理对象能被取出并延迟绑定——这种隐藏性既是便利也是坑。

·初学者误区：以为 super 是"对象"。它返回的是"代理"，是个临时视图，不是类也不是实例。

属性获取算法（Attribute-fetch algorithm）

英文原文

> In a single-inheritance tree like the preceding example, super is straightforward because there's just one obvious follower on the MRO. In fact, in this simple case, the immediate superclass can be had from an instance at class.bases without applying MROs at all:

```
>>> E.__mro__
(<class '__main__.E'>, <class '__main__.C'>, <class 'object'>)
>>> E().__class__.__bases__[0]
<class '__main__.C'>
```

> In the more complex class trees of multiple inheritance, though, you must understand super's full algorithm. The proxy object uses its saved instance and class to resolve attribute references as follows: 1. Fetch the MRO of the saved self instance's class, at self.class.mro. 2. Scan this MRO from left to right to find the saved containing class, and skip it. 3. Search the namespace dictionaries of each remaining class in the MRO from left to right until the requested attribute is found. 4. If the attribute was found and is a method, bind it with the saved self instance. This procedure is run for each attribute fetch and is wholly based on MRO ordering. It must start with self's MRO because the containing class's own MRO won't apply if it has been mixed with other classes. You can't ignore these underlying mechanics except in very simple class trees. Unlike in Java, mix-in classes make multiple inheritance from disjoint and independent superclasses common in realistic code. And once you add multiple superclasses, you've kicked super up to a whole new level.

中文翻译

在像上例那样的单继承树里，super 很直白——MRO 上只有一个显而易见的"后继者"。事实上，在这个简单情形里，直接从实例的 class.bases 就能拿到直接超类，根本用不上

MRO：

```
>>> E.__mro__
(<class '__main__.E'>, <class '__main__.C'>, <class 'object'>)
>>> E().__class__.__bases__[0]
<class '__main__.C'>
```

但在多重继承的复杂类树里，你必须理解 super 的完整算法。super 返回的代理对象，用保存的实例与"包含调用的类"按如下步骤解析属性引用：1. 取保存的 self 实例所属类的 MRO，即 self.class.mro。2. 从左到右扫描这个 MRO，找到保存的"包含类"并跳过它。3. 从左到右搜索 MRO 里其余每个类的命名空间字典，直到找到请求的属性。4. 若找到的属性是方法，就用保存的 self 实例绑定它。这个过程在每次属性获取时都会执行，且完全基于 MRO 的顺序。它必须从 self 的 MRO 出发，因为"包含类"如果被混进别的类，它自己的 MRO 就不再适用。除了非常简单的类树，你无法无视这些底层机制。与 Java 不同，Python 里 mix-in（混入）类让"从不相交、独立的超类多重继承"在真实代码里屡见不鲜。而一旦加上多个超类，super 就升入了一个全新的境界。

深度理解

·核心概念：super 的查找算法 = "在 self 的 MRO 上，越过包含类，往后找名字"。这与"找父类"并不总是同一个东西。

·底层实现：代理对象的 getattr/getattribute 实现正是第 2~4 步；MRO 由 type 创建时用 C3 线性化算好并缓存。

·设计原因：从 self 的 MRO（而非包含类的 MRO）出发，是"协作分发"能跨类协调的关键——否则混入场景立刻失效。

·实际问题：实例的 MRO 可能因更低层类混入而改变，super 的行为随树形浮动——"同一个 super 调用，不同实例可能走向不同方法"。

·初学者误区：以为 super().m() 与"我类的直接父类的 m"绑定。第四步说明它找到的是"MRO 上第一个含 m 的类"，未必是直系父类。

通用部署问题（Universal deployment）

英文原文

> Let's illustrate with code. Suppose you've written the following classes that happily deploy super in simple single-inheritance mode to implicitly invoke a method one level up from C:

```
> >> class A:

...     def act(self): print('A')
> >> class B:

...     def act(self): print('B')
> >> class C(B):

...     def act(self):
...         super().act()          # super
> >> C().act() #

B
```

> If such classes later grow to use more than one super class, though, super's effects might be surprising—it does not raise an exception when the same name ap pears in more than one superclass of a multiple inher itance tree but will naively pick just the leftmost supe rclass having the method being run (really, the first p er the class tree's flattened MRO). This may or may n ot be the class that you want, and is completely wron g if you want both:

```
> >> class C(B, A):

...     def act(self):
...         super().act()          #  A mix-in
> >> C().act() # ——

B
> >> class C(A, B):

...     def act(self):
...         super().act()          #  A B.act()
> >> C().act()

A
```

> This silently masks a source of OOP errors so commo n that it shows up again in this part's "Gotchas" ahea d. Explicit calls are one way to solve this dilemma. Wi th explicit class names, you can choose either the left class, the right class, or both, and with substantially l ess drama:

```
> >> class C(A, B):

...     def act(self):
...         A.act(self)            #
...         B.act(self)            #
> >> C().act()

A
B
```

> The real underlying issue with super here, though, is that it requires itself to be called in every class's met hod in order to propagate the call chain. Without this universal deployment, the call dies in the first class th at doesn't call super—as in our A and B. While we ca n add super to these classes, too, this is the first of th at handful of arduous super coding requirements.

中文翻译

> 我们用代码演示。假设你写了下面这些类，快乐地用 super 以单继承模式，隐式调用 C 上面一级的方法：

```
>>> class A:
...     def act(self): print('A')
>>> class B:
...     def act(self): print('B')
>>> class C(B):
...     def act(self):
...         super().act()          # super
>>> C().act()
B
```

但如果这些类后来长成"多个超类"，super 的效果可能出乎意料——当同名方法出现在多重继承树的好几个超类里时

，它不会报错，而是天真地只挑"最左的超类"（准确地讲，是按类树压平后的 MRO 顺序找到的第一个）。这可能不是你想要的那个类；若你想两个都跑，它更是完全错误的：

```
>>> class C(B, A):
...     def act(self):
...         super().act()          #  A mix-in
>>> C().act()                      # —
B
>>> class C(A, B):
...     def act(self):
...         super().act()          #  A B.act()
>>> C().act()
A
```

这静悄悄地掩盖了一类非常常见的 OOP 错误源——它还会在本部分的"Gotchas（陷阱）"一节再次现身。显式调用是解此困局的一条路：用显式类名，你可以挑左边的类、右边的类、或者两个都要，而且几乎不用上演什么戏剧：

```
>>> class C(A, B):
...     def act(self):
...         A.act(self)            #
...         B.act(self)
>>> C().act()
A
B
```

但 super 在这里真正的底层问题是：它要求每一个类的方法里都调用 super，调用链才能传播下去。没有这种"通用部署"（universal deployment），调用会在第一个不调用 super 的类那里戛然而止——就像我们的 A 与 B。当然，我们也可以给这些类都加上 super，但这正是那"几项艰苦的 super 编码要求"的第一条。

深度理解

·核心概念：super 在多重继承里"静默挑左"——不报错，却可能跑错方法；这与"显式 A.act(self)"的透明形成对照。

·底层实现：C3 线性化的 MRO 是扁平列表；"第一个含该名的类"就是算法结果，无关语义意图。

·设计原因：Python 选择"确定性优先"（MRO 决定一切），把"运行哪个方法"交给数学而非猜测。

·实际问题：这是 OOP 错误之源：你以为继承某方法，实际被 mix-in 顶替。第 32.12 节"Gotchas"还会重申。

·初学者误区：以为 super 会"调用所有父类"；不，它只调用 MRO 上的第一个匹配者，其余一概沉默。

调用链之锚（Call-chain anchors）

英文原文

> Since both A and B of the prior section's example are somewhere on C's MRO, we might be tempted to make both classes' methods run by propagating the call with added super calls in both. This scheme is called cooperative method dispatch: each class in a tree runs super to hand the call off to the next class on the MRO that cares about it.

This automatically routes calls through each calling class just once and avoids running a method in a diamond's common superclass more than once. It assume

s that the MRO's order makes sense for your method calls.

Armed with that info, here's the mod in our example:

```
> >> class A:

...     def act(self):
...         print('A')
...         super().act()          #  super
> >> class B:

...     def act(self):
...         print('B')
...         super().act()          #  super
> >> class C(B, A):

...     def act(self):
...         super().act()          #
> >> C().act()

B
A
AttributeError: 'super' object has no attribute 'act'
```

Immediately, though, we're in trouble here, and the r eason requires inspecting the MRO of an instance's cl ass:

```
> >> I = C() >> I.__class__.__mro__

(<class '__main__.C'>, <class '__main__.B'>, <class '__main__.A'>,…
```

Here's the subtle problem. The super in C will select a ct in B, the next on this MRO; the super in B will then select act in A, its follower on self's MRO; but then th ere are no more act definitions: the super in A looks f

or act in object and beyond, and of course fails. We s ay that there is no call-chain anchor—no end point f or the call propagation. Hence, the last super call in A dies with an exception.

In fact, you've just met a possible strike three for this call. While this code works if you omit the super in A, this policy won't help in general: classes like A and B are probably designed to be mixed into other classes , too, and it wouldn't make sense to specialize their c ode just for the C class's use case. To propagate supe r method calls, all classes must define super, too, and there must be an anchor to catch and end the chain s omewhere.

Although we can add an anchor in a pointless supercl ass that defines the method but does not call super a gain, this is another of those arduous coding require ments—and substantially more effort than simply run ning the explicit class-name calls alternative shown e arlier:

```
> >> class X:

...     def act(self):             #  super
...         print('anchor')
> >> class A:

...     ...                        #
> >> class B:

...     ...                        #
> >> class C(B, A, X):
```

```
...     def act(self):
...         super().act()          #  super
> >> C().act()

B
A
anchor
> >> [c.__name__ for c in C().__class__.__mro__]

['C', 'B', 'A', 'X', 'object']
```

This works because X precedes object on the MRO as shown, and hence stops the act call chain. Adding th e anchor class X as a common superclass to both A a nd B in a diamond would work, too, because the resu lting MRO is the same (the MRO removes all but the l ast [rightmost] appearance of a class from the DFLR order):

```
> >> class X:

...     ...                        #
> >> class A(X):

...     ...                        #
> >> class B(X):

...     ...                        #
> >> class C(B, A):

...     ...                        #
> >> C().act()

B
A
```

```
anchor
> >> [c.__name__ for c in C().__class__.__mro__]

['C', 'B', 'A', 'X', 'object']
```

> Again, though, if you have to code a special class just to appease super, why not just use explicit class nam es? Similarity to other languages isn't a very good re ason, especially in contexts that other languages don 't support.

> Also, keep in mind that the class selected by super fo r an attribute reference may not be a superclass at all and may vary per tree that a class is mixed into. For i nstance, the super in B in our example dispatches to A—the next on the MRO, but a sibling, not a supercla ss. If you must be sure that an immediate superclass' s method is run, you again must use explicit class na mes instead of super.

中文翻译

> 既然上例的 A 与 B 都在 C 的 MRO 上，我们或许会忍不住想"给两个类的方法都加上 super 来传播调用，让两个方法都运行"。这套方案叫协作式方法分发（cooperative meth od dispatch）：树里每个类都调 super，把调用接力给 M RO 上下一个关心它的类。它自动让调用途经每个调用类恰好一次，并避免菱形里公共超类的方法被执行两次。它的假设是：MRO 的顺序对你的方法调用是有意义的。带着这个信息，我们给例子做了如下改动：

```
>>> class A:
...     def act(self):
...         print('A')
...         super().act()          #  super
>>> class B:
...     def act(self):
...         print('B')
...         super().act()          #  super
>>> class C(B, A):
...     def act(self):
...         super().act()          #
>>> C().act()
B
A
AttributeError: 'super' object has no attribute 'act'
```

可我们立刻就栽了，原因得查看实例所属类的 MRO 才明白：

```
>>> I = C()
>>> I.__class__.__mro__
(<class '__main__.C'>, <class '__main__.B'>, <class '__main__.A'>,…
```

微妙的问题在这：C 里的 super 会选中 MRO 上下一位 B 的 act；B 里的 super 再选 self 的 MRO 上 A 的 act；可是再往后就没有 act 了——A 里的 super 去 object 及其后再找 act，当然失败。我们说这里没有"调用链之锚"（call-chain anchor）——没有能终结调用传播的端点。于是 A 里最后一个 super 调用带着异常死掉了。事实上，你刚见识了这项调用的"第三击"。如果把 A 里的 super 删掉，这段代码能跑——但这一招不能普遍生效：像 A、B 这样的类，大概还被设计去混进别的类；专门为了 C 这个类的用法而定制它们，不成道理。要传播 super 方法调用，所有类都得定义 super，而且树里必须有个锚来接住并终结这条链。虽然我们可以在一个"定义方法却不再调 super"的没意

义超类里加个锚，但这又是一条艰巨的编码要求——比直接跑前面那个显式类名调用方案费劲得多：

```
>>> class X:
...     def act(self):              #  super
...         print('anchor')
>>> class A:
...     ...                         #
>>> class B:
...     ...                         #
>>> class C(B, A, X):
...     def act(self):
...         super().act()           #  super
>>> C().act()
B
A
anchor
>>> [c.__name__ for c in C().__class__.__mro__]
['C', 'B', 'A', 'X', 'object']
```

它能跑，是因为 X 如上所示排在 MRO 的 object 之前，从而终结了 act 调用链。把锚类 X 作为 A、B 的公共超类织成菱形也一样可行——因为结果 MRO 相同（记住：MRO 会从 DFLR 顺序中删掉一个类的全部出现，只保留最右的那个）：

```
>>> class X:
...     ...                         #
>>> class A(X):
...     ...                         #
>>> class B(X):
...     ...                         #
>>> class C(B, A):
...     ...                         #
>>> C().act()
B
A
```

```
anchor
>>> [c.__name__ for c in C().__class__.__mro__]
['C', 'B', 'A', 'X', 'object']
```

> 不过话又说回来：如果必须专门写个类来安抚 super，为什么不直接用显式类名呢？"与别的语言相似"可不是好理由——尤其在别的语言根本不支持的场合。另外请记住：super 为某属性引用选中的类，可能根本不是超类，还可能随类被混入的树不同而变。比如本例中 B 里的 super 分发给了 A——MRO 上的下一位，却是兄弟而不是超类。若你必须确保"直接超类的方法一定运行"，那就还得用显式类名，而非 super。

深度理解

·核心概念：协作式分发（cooperative dispatch）的运作与前提：全员 super + 链尾有锚。

·底层实现：super().getattr 找下一个含名类；MRO 尾部的 object 不提供 act，链断裂即 AttributeError。

·设计原因：MRO 的 DFLR 简化规则（保留最右出现）让菱形两版本 MRO 相同——同一算法两种写法殊途同归。

·实际问题：真实框架里"链尾锚"常由混入的 object/Base 抽象类承担；写 mix-in 而不留锚，就是埋雷。

·初学者误区：以为"每个类都调用 super 就万事大吉"；链必须有个"终点"，否则末端的 super 必然抛错。

相同的参数列表（Same argument lists）

英文原文

> While super always demands a call-chain anchor, you may occasionally get one for free. As a special case, object defines a constructor that can be relied on to anchor some chains (recall that the init constructor method is a class attribute like any other, despite its odd name and automatic invocation):

```
> >> class A:

...     def __init__(self):
...         print('A')
...         super().__init__()     #
> >> class B:

...     def __init__(self):
...         print('B')
...         super().__init__()     #
> >> class C(B, A):

...     def __init__(self):
...         super().__init__()     #  object
> >> I = C()

B
A
```

> This propagates the constructor call through C, B, A, and object per the instance's MRO via cooperative method dispatch as before. This also fails, however, for constructors that take any arguments because that of object takes none except self:

```
> >> class A:

...     def __init__(self, name):
...         print('A')
...         super().__init__(name)
> >> class B:

...     def __init__(self, name):
...         print('B')
...         super().__init__(name)
> >> class C(B, A):

...     def __init__(self, name):
...         super().__init__(name)
> >> I = C('Pat')

B
A
TypeError: object.__init__() takes exactly one argument (the insta…
```

> And now you've run into another of those arduous c oding requirements: super generally assumes that all the methods in a call chain use the same argument li st because the MRO's ordering of method calls can v ary with class-tree shape: an arbitrary change in inher itance may change call order arbitrarily. This limits fle xibility inherently.

> While you may be able to ensure same arguments for classes used only in a single program, neither policy will apply to code meant to be reused in multiple con texts—which is really one of the main points behind Python programming.

中文翻译

> super 虽然总要求有"调用链之锚"，但你偶尔能白捡一个。特殊情况下，object 定义了一个构造器，可以可靠地给某些链当锚（回忆一下：init 构造器方法虽然名字怪、被自动调用，但它和其他属性一样，是类属性）：

```
>>> class A:
...     def __init__(self):
...         print('A')
...         super().__init__()     #
>>> class B:
...     def __init__(self):
...         print('B')
...         super().__init__()     #
>>> class C(B, A):
...     def __init__(self):
...         super().__init__()     #  object
>>> I = C()
B
A
```

> 这就如前述协作式分发，让构造器调用沿 I 实例的 MRO 依次穿过 C、B、A、object。可是对带参数的构造器，这招同样会失败——因为 object 的构造器除了 self 不接收任何参数：

```
>>> class A:
...     def __init__(self, name):
...         print('A')
...         super().__init__(name)
>>> class B:
...     def __init__(self, name):
...         print('B')
...         super().__init__(name)
>>> class C(B, A):
```

```
...     def __init__(self, name):
...         super().__init__(name)
>>> I = C('Pat')
B
A
TypeError: object.__init__() takes exactly one argument (the insta…
```

> 于是你撞上了又一条艰巨的编码要求：super 一般假定整条调用链里所有方法都使用相同的参数列表——因为 MRO 的方法调用顺序会随类树形状而变：继承上任意一次改动，都可能任意地改变调用顺序。这从根上限制了灵活性。对只在一个程序里使用的类，你或许能保证参数一致；但对"要拿到多个上下文里复用"的代码（这本就是 Python 编程的重要卖点之一），这两招都行不通。

深度理解

·核心概念：object.init 是免费的"链尾锚"（仅限无参构造器）；带参构造器会让链在 object 处崩断。

·底层实现：object.init 的 C 实现只接受一个实参；链上任何一处传参，最末端必然 TypeError。

·设计原因：统一参数列表（args 或零参数）是唯一能随"任意 MRO"运行的方案——这是"树形自由"与"参数一致"的永恒矛盾。

·实际问题：Django、SQLAlchemy 等框架要求 mix-in 的 init 必须兼容任意参数（args, kwargs），正是这个原因。

·初学者误区：只给链上部分类传参；链条一旦改变顺序或形状，你的"恰好配对"就会错位。

非调用与运算符重载（Noncalls and operator overloading）

英文原文

> To close, here are two more super oddities. First, keep in mind that super, like the MRO, is not just about methods. It can also fetch the class data attributes we met in Chapter 29 and the bound methods we met in Chapter 31:

```
> >> class C:

...     attr1 = 'hack'               # super
...     def attr2(self):             #
...         return 'code'
> >> class D(C):

...     def act(self):
...         return super().attr1, super().attr2
> >> I = D() >> I.act()

('hack', <bound method C.attr2 of <__main__.D object at 0x10399320…
> >> I.act()[1]()

'code'
```

> When you access a method like attr2 from a super proxy, the proxy binds it with the saved instance to produce a bound method. Fetching a method as a plain function, however, may require an explicit class name, like C.attr2.

Second, super also doesn't fully work in the presence of X operator-overloading methods. Explicit named calls to overloading methods in the superclass work normally, but using the super result in an expression fails to dispatch to the superclass's method:

```
> >> class C:

...     def __getitem__(self, ix):   #
...         print('C index')
> >> class D(C):

...     def __getitem__(self, ix):   #
...         print('D index')
...         C.__getitem__(self, ix)  #
...         super().__getitem__(ix)  #
...         super()[ix]              #  __getattribute__
> >> I = C() >> I[99]

C index
> >> I = D() >> I[99]

D index
C index
C index
TypeError: 'super' object is not subscriptable
```

This behavior is due to the same limitation described in the sidebar "Delegating Built-ins—or Not"—because the proxy object returned by super uses the getattribute method to catch and dispatch later attribute requests, it fails to intercept the automatic X method invocations run by built-in operations including expressions, as these begin their search in the class instead

of the instance.

> This may seem less severe than the other limitations, but operators should generally work the same as the equivalent method call, especially for a built-in like t his. Not supporting this adds another exception for s uper users to confront and remember. In Python, self is explicit, multiple-inheritance mix-ins and operator overloading are common, and superclass name chan ges are rare enough to pass as a red herring.

中文翻译

> 收尾之前，还有两件 super 的怪事。其一，请记住：与 M RO 一样，super 不只跟方法有关——它也能获取第 29 章的数据属性和第 31 章的绑定方法：

```
>>> class C:
...     attr1 = 'hack'               # super
...     def attr2(self):             #
...         return 'code'
>>> class D(C):
...     def act(self):
...         return super().attr1, super().attr2
>>> I = D()
>>> I.act()
('hack', <bound method C.attr2 of <__main__.D object at 0x10399320…
>>> I.act()[1]()
'code'
```

> 从 super 代理取 attr2 这类方法时，代理把它与保存的实例绑定，产出绑定方法；但要以"普通函数"形态取方法，可能就得用显式类名，比如 C.attr2。其二，在 X 运算符重载方法面前，super 也不能完全工作。对超类重载方法做显式

点名调用一切正常，但把 super 的结果放进表达式里，就无法分发到超类的方法：

```
>>> class C:
...     def __getitem__(self, ix):   #
...         print('C index')
>>> class D(C):
...     def __getitem__(self, ix):   #
...         print('D index')
...         C.__getitem__(self, ix)  #
...         super().__getitem__(ix)  #
...         super()[ix]              #  __getattribute__
>>> I = C()
>>> I[99]
C index
>>> I = D()
>>> I[99]
D index
C index
C index
TypeError: 'super' object is not subscriptable
```

这个行为与边栏"委托内建——或不要"描述的局限同源：super 返回的代理对象用 getattribute 方法捕获、分发后续属性请求，因此它拦不住"由内建操作（包括表达式）自动触发的 X 方法调用"——那些调用从类（而非实例）出发开始搜索。比起其他局限，这条看似较轻，但"运算符"与"等价的方法调用"理应行为一致，尤其对这样的内建；不支持它，等于给 super 用户又添一条必须面对和记住的例外。而在 Python 里：self 是显式的；多重继承 mix-in 与运算符重载很常见；超类改名这种事儿少得足以当作"红鲱鱼"（转移注意力的障眼法）。

深度理解

·核心概念：super 代理（1）能取数据属性/绑定方法；（2）却无法参与运算符表达式——super()[ix] 崩、super().getitem(ix) 活。

·底层实现：I[ix] 触发的是"类型级"协议查找（type(I).getitem），实例的 getattribute 压根不参与；而 super 代理只重写了 getattribute，于是对运算符"隐身"。

·设计原因：运算符重载从类出发（求快、防递归），与"实例级拦截"路线不同——这就是"委托内建失效"的普遍根因。

·实际问题：任何"包装/代理"类（super 代理、委托包装器）都要为每个运算符重载方法单独显式转发。

·初学者误区：以为"能用 super().name 拿到的，就能写进表达式"；getitem 类的运算符协议偏不买账。

super 收官小结（The super Wrap-Up）

英文原文

So there you have it: a brief tutorial on the super built-in. Hopefully, the coverage here has given you a balanced view of this tool's trade-offs while introducing its fundamentals. As we've just seen, in single-inheritance class trees, the super call may be used to refer to parent superclasses generically without naming them explicitly. In multiple-inheritance trees, this call can also be used to implement cooperative method dis

patch that propagates calls through a tree. The latter role may be especially useful in diamonds, as a confo rming method call chain visits each superclass just on ce. While these are clear upsides in some contexts, it' s important to know that super can also yield highly i mplicit behavior, which for some programs may not i nvoke superclasses as expected or required. To sum marize, the super method-dispatch technique genera lly imposes three main coding requirements: Anchors : the method called by super must exist—which requi res extra code and calls if no call-chain anchor is pres ent, and mix-in classes can't be specialized for a singl e tree's context. Arguments: the method called by su per must have the same argument signature across t he entire class tree—which can impair flexibility, espe cially for implementation-level methods like construc tors. Deployment: every appearance of the method c alled by super but the last must use super itself—whi ch can make it difficult to use existing code, change c all ordering, override methods, and code self-contain ed classes. In addition, super builds upon the already complex MRO, can mask problems when single-inher itance trees become multiple-inheritance trees, may s elect a class other than a superclass in multiple-inheri tance trees, and constitutes yet another special case f or attribute inheritance, which we'll revisit in Chapter 40. In the end, super is easy to use and relatively har mless in single-inheritance roles, but its unusual sem

antics, rigid requirements, and questionable net reward make it a mixed bag. Python programmers, especially those learning Python anew, might be better served by the more general and transparent coding paradigm of explicit class-name references. But you should judge all this for yourself in an OOP Python program near you.

中文翻译

就这样，你拿到了 super 内建的简短教程。希望这里的覆盖在介绍基础的同时，也给了你一幅关于此工具"利弊权衡"的均衡视图。如我们刚刚所见：在单继承类树里，super 调用可用来通用地指代父超类而无须点名；在多重继承树里，它还能实现"协作式方法分发"，让调用沿树传播——后一角色在菱形结构里尤其有用，因为合规的方法调用链会让每个超类恰好被访问一次。这些在某些情境下是明确的加分项；但重要的是你得知道，super 也可能带来高度隐式的行为——对某些程序而言，它未必按你期望或需要的方式调用超类。总结一下，super 的方法分发技巧一般强加三条主要编码要求：锚（Anchors）：被 super 调用的方法必须存在——没有调用链之锚时就要额外写代码、额外调用；而且 mix-in 类不能为了单个树的语境去特化。参数（Arguments）：被 super 调用的方法在整棵类树里必须保持相同参数签名——这会损害灵活性，尤其对构造器这类"实现级"方法。部署（Deployment）：被 super 调用的方法除了最后一次出现，其余每一处都必须使用 super 本身——这会让人难以复用既有代码、改调用顺序、覆盖方法、写自足类。另外，super 建立在已经够复杂的 MRO 之上；单继承树变多

重继承树时可能掩盖问题；在多重继承树里可能选中"并非超类"的类；并且它构成属性继承的又一个特例——我们在第 40 章还会旧话重提。说到底，单继承场景里 super 易用且相对无害，但它异常的语义、僵硬的要求、成疑的净收益，让它成为一锅"好坏参半"的大杂烩。Python 程序员——尤其是初学 Python 的人——或许更该受惠于"显式类名引用"这种更通用、更透明的编码范式。但这一切，还是请在你身边的某个 OOP Python 程序里自己裁决吧。

深度理解

·核心概念：三条"编码要求"（锚/参数/部署）是 super 的使用税；本文反复渲染"全有或全无"。

·底层实现：锚（链终点）、参数（同签名）、部署（全员 super）三者任一缺位，链就断或错。

·设计原因：super 从 MRO 求"确定性"，却不提供"意图性"——Python 宁可透明，也不愿魔法替你猜。

·实际问题：这是作者的教学立场：读得懂、用得对、敢于不。遇到代码里显式 Super.m(self)，不必觉得"老派"。

·初学者误区：把"能用 super"当作"该用 super"。权衡三税之后，很多场合显式类名更划算。

32.11 类的陷阱（Class Gotchas）

英文原文

We've reached the end of the primary OOP coverage in this book. After exceptions up next, we'll explore a

dditional class-related examples and topics in the las t part of the book, but that part mostly just gives exp anded coverage to concepts introduced here. As usu al, let's wrap up this part with the standard warnings about pitfalls to avoid. Most class issues can be boile d down to namespace issues—which makes sense, gi ven that classes are largely just namespaces with a ha ndful of extra tricks. Some of the items in this section are more like class usage pointers than problems, but even experienced class coders have been known to st umble on a few.

中文翻译

我们已抵达本书"主 OOP 覆盖"的终点。下一章（异常）之后，本书最后一部分还会探索更多与类相关的示例和主题，但那一部分大体只是把这里引入的概念讲得更广。照例，让我们用标准"避坑警告"来结束本部分。多数类问题都能归结为命名空间问题——这讲得通，因为类在很大程度上不过是"加了一小撮技巧的命名空间"。本节某些条目更像是"类用法指针"而非问题，但就连经验丰富的类编码者，也有几个人在这些地方栽过跟头。

深度理解

·核心概念：把"类问题"归约为"命名空间问题"是本节的统一视角：属性存放位置（类 or 实例）、查找顺序（左到右、MRO）、作用域可见性——全是命名空间的事。

·底层实现：类对象与实例对象的属性都存于字典/描述符；"在哪里赋值"决定了"谁能看见"。

·设计原因：作者希望读者以"命名空间思维"审视一切类谜题，而不是背一堆孤立规则。

·实际问题：调试 OOP 代码时先问三件事：这名字定义在哪？谁能继承到它？赋值发生在谁身上？

·初学者误区：以为"类是特殊的存在"；把类当作"带方法的字典"反而最容易理清。

修改类属性可能带来副作用（Changing Class Attributes Can Have Side Effects）

英文原文

> Theoretically speaking, classes (and class instances) are mutable objects. As with built-in lists and dictionaries, you can change them in place by assigning to their attributes—and as with lists and dictionaries, this means that changing a class or instance object may impact multiple references to it.

> That's usually what we want and is how objects change their state in general, but awareness of this issue becomes especially critical when changing class attributes. Because all instances generated from a class share the class's namespace, any changes at the class level are reflected in all instances unless they have their own versions of the changed class attributes.

In Python, we can normally change any attribute in any object to which we have a reference. Consider the following class. Inside the class body, the assignment

to the name a generates an attribute X.a, which lives i n the class object at runtime and will be inherited by all of X's instances:

```
> >> class X:

...     a = 1                   #
> >> I = X() >> I.a #

1
> >> X.a #

1
```

So far, so good—this is the normal case. But notice w hat happens when we change the class attribute dyn amically outside the class statement: it also changes t he attribute in every object that inherits from the clas s. Moreover, new instances created from the class du ring this session also get the dynamically set value, re gardless of what the class's source code says:

```
> >> X.a = 2 #  X >> I.a # I

2
> >> J = X() # J  X  >> J.a

2                               #  J.a  J  a X  I
```

Is this a useful feature or a dangerous trap? You be t he judge. As discussed in Chapter 27, you can actuall y get work done by changing class attributes without ever making a single instance—a technique that can simulate the use of "records" or "structs" in other lan

guages:

```
class X: pass                    #
class Y: pass
X.a = 1                          #
X.b = 2                          #
X.c = 3
Y.a = X.a + X.b + X.c
for X.i in range(Y.a): print(X.i)    #  0..5
```

Here, the classes X and Y work like "fileless" modules—namespaces for storing variables we don't want to clash. This is perfectly legal, but it's less appropriate when applied to classes written by others; you can't always be sure that class attributes you change aren't critical to the class's internal behavior. If you're out to simulate a C struct, you may be better off changing instances than classes, as that way only one object is affected:

```
class Record: pass
X = Record()
X.name = 'pat'
X.job = 'Pizza maker'
```

中文翻译

理论上讲，类（以及类实例）是可变（mutable）对象。与内置的列表、字典一样，你可以通过给它们的属性赋值来就地修改它们——同样，这意味着"修改类或实例对象"可能波及对它的多个引用。这通常正是我们想要的，也是对象普遍改变自身状态的方式；但修改类属性时，这一认识尤其关键。因为从一个类生成的所有实例共享类的命名空间，类层面的任何改动都会反映到所有实例上——除非这些实例自己

有被改属性的独立版本。在 Python 里，我们通常能修改"任何持有引用的对象"的任何属性。看这个类：类体内给名字 a 的赋值生成了属性 X.a，它运行时住在类对象里，会被 X 的所有实例继承：

```
>>> class X:
...     a = 1                    #
>>> I = X()
>>> I.a                         #
1
>>> X.a                         #
1
```

至此一切正常——这是常规情形。但注意：在 class 语句之外动态修改类属性时，它会同步改变每一个"继承自该类"的对象里的同名属性。而且本次会话里之后新建的实例，也会拿到这个动态设置的值——不管类的源码里写的是什么：

```
>>> X.a = 2                     #  X
>>> I.a                         # I
2
>>> J = X()                     # J  X
>>> J.a
2                               #  J.a  J  a X  I
```

这是有用功能还是危险陷阱？你来裁断。如第 27 章所述，你其实可以不建任何实例、仅靠修改类属性就完成工作——这招能模拟其他语言里"记录（records）"或"结构体（structs）"的用法：

```
class X: pass                    #
class Y: pass
X.a = 1                          #
X.b = 2                          #
X.c = 3
Y.a = X.a + X.b + X.c
for X.i in range(Y.a): print(X.i)    #  0..5
```

这里的 X 与 Y 两个类像是"无文件模块"——用来存放"不想让它冲突"的变量的命名空间。这完全合法，但用在"别人写的类"上就不那么合适了：你无法总保证"你修改的类属性"对这个类的内部行为无关紧要。若你想模拟 C 的 struct，也许改实例比改类更好——那样只会影响一个对象：

```
class Record: pass
X = Record()
X.name = 'pat'
X.job = 'Pizza maker'
```

深度理解

·核心概念："改类属性 = 改所有继承者"；"改实例属性 = 只改自己"。前者是共享，后者是遮蔽（shadowing）。

·底层实现：类属性存于类字典；实例查找沿"实例 → 类 → 超类"；实例一旦赋值同名属性，就在自己字典里"盖住"类版本。

·设计原因：Python 允许"类即命名空间"（无实例编程）——第 27 章把它当特性展示；但对第三方类这样做则危险。

·实际问题：运行时动态改类属性是"猴子补丁"（monkey patching）的原型；做之前要知道自己动的是全局共享数据。

·初学者误区：以为"给实例赋值"与"给类赋值"等价；事实上 J.a = 5 之后 X.a 依旧，I.a 也依旧。

修改可变类属性同样有副作用（Changing Mutable Class Attributes Can Have Side Effects, Too）

英文原文

> This gotcha is really an extension of the prior. Because class attributes are shared by all instances, if a class attribute references a mutable object, changing that object in place from any instance impacts all instances at once:

```
> >> class C:

...     shared = []             #
...     def __init__(self):
...         self.perobj = []    #
> >> x, y = C(), C() #  >> y.shared, y.perobj #

([], [])
> >> x.shared.append('hack') #  y  >> x.perobj.append('code…

(['hack'], ['code'])
> >> y.shared, y.perobj # y  x

(['hack'], [])
> >> C.shared #

['hack']
```

> This effect is no different than many we've seen in this book already: mutable objects are shared by simple variables, globals are shared by functions, module-le

vel objects are shared by multiple importers, and mutable function arguments are shared by the caller and the callee. All of these are cases of general behavior—multiple references to a mutable object—and all are impacted if the shared object is changed in place from any reference.

Here, this occurs in class attributes shared by all instances via inheritance, but it's the same phenomenon at work. It may be made more subtle by the different behavior of assignments to instance attributes themselves:

```
x.shared.append('hack')          #
x.shared = 'hack'                # / x
```

But again, this is not a problem, it's just something to be aware of; shared mutable class attributes can have many valid uses in Python programs.

中文翻译

这条陷阱其实是上一条的延伸。因为类属性被所有实例共享，若某个类属性引用的是可变对象，从任何实例对它做就地修改，会同时影响所有实例：

```
>>> class C:
...     shared = []             #
...     def __init__(self):
...         self.perobj = []    #
>>> x, y = C(), C()             #
>>> y.shared, y.perobj          #
([], [])
>>> x.shared.append('hack')     #  y
>>> x.perobj.append('code')     #  x
>>> x.shared, x.perobj
(['hack'], ['code'])
>>> y.shared, y.perobj          # y  x
(['hack'], [])
>>> C.shared                    #
['hack']
```

这个效应与本书里见过的大量例子并无二致：可变对象被简单变量共享、全局量被函数共享、模块级对象被多个导入方共享、可变函数实参被调用方与函数共享。它们全是同一种普遍行为的案例——"对同一个可变对象的多个引用"——只要从任一引用对共享对象就地修改，全都受影响。这里发生在"经继承被所有实例共享的类属性"上，但背后的现象完全相同。它可能被"给实例属性赋值"的不同行为弄得更微妙：

```
x.shared.append('hack')          #
x.shared = 'hack'                # / x
```

再说一遍：这不是 bug，只是需要意识到的事；共享的可变类属性在 Python 程序里有大量正当用途。

深度理解

·核心概念："就地修改（append）"与"整体重绑（=）"是两个完全不同的操作：前者动共享体，后者造私有影。

·底层实现：x.shared.append 走"查找→取到引用→调方法"；x.shared = ... 走"实例字典赋值（遮蔽）"。

·设计原因：这一"可变共享"正是 Python 对象模型一贯的语义（引用计数世界的常识）；类属性只是其中一个舞台。

·实际问题：GUI 里"所有窗口共享最近目录"，故意用共享类属性是特性；误用则是"一个实例改了全部实例"的惊吓。

·初学者误区：把"类里写 shared = []"和"init 里写 self.perobj = []"混为一谈；前者共享、后者独立。

多重继承：顺序很重要（Multiple Inheritance: Order Matters)

英文原文

> This may be obvious by now, but it's worth underscoring one last time: if you use multiple inheritance, the order in which superclasses are listed in the class statement header can be critical. Python always searches superclasses from left to right, according to their order in the header line. For instance, in the multiple inheritance example we studied in Chapter 31, imagine that the Super class implemented a str method, too:

```
class ListTree:
    def __str__(self): ...
class Super:
    def __str__(self): ...
class Sub(ListTree, Super):      #  ListTree  ListTree.__st…
    ...
x = Sub()                        #  ListTree  Super
```

Which class would we inherit it from—ListTree or Super? As inheritance searches generally proceed from left to right, we would get the method from whichever class is listed first (leftmost) in Sub's class header. Presumably, we would list ListTree first because its whole purpose is its custom str. But now, suppose Super and ListTree have their own versions of other same-named attributes, too. If we want one name from Super and another from ListTree, the order won't help—we will have to override inheritance by manually assigning to the attribute name in the Sub class:

```
class ListTree:
    def __str__(self): ...
    def other(self): ...
class Super:
    def __str__(self): ...
    def other(self): ...
class Sub(ListTree, Super):      #  ListTree.__str__
    other = Super.other          #  other  Super
    def __init__(self): ...
x = Sub()                        #  Sub  ListTree/Super
```

Here, the assignment to other within the Sub class creates Sub.other—a reference back to the Super.other object. Because it is lower in the tree, Sub.other effec

tively hides ListTree.other, the attribute that the inher itance search would normally find. Similarly, if we list ed Super first in the class header to pick up its other, we would need to select ListTree's method explicitly:

```
class Sub(Super, ListTree):      #  Super.other
    __str__ = ListTree.__str__   #  __str__  ListTree
```

For another example of the technique shown here, se e "Attribute Conflict Resolution." Ultimately, multiple inheritance is an advanced tool. Even if you understo od the last paragraph, it's still a good idea to use it s paringly and carefully. Otherwise, the meaning of a n ame may come to depend on the order in which class es are mixed in an arbitrarily far-removed subclass. A s a rule of thumb, multiple inheritance works best wh en your mix-in classes are as self-contained as possib le—because they may be used in a variety of context s, they should not make assumptions about names re lated to other classes in a tree. The pseudoprivate X a ttributes feature we studied in Chapter 31 can help b y localizing names that a class relies on owning and li miting the names that mix-in classes add to the mix. I n this example, for instance, if ListTree only means to export its custom str, it can name its other method ot her to avoid clashing with like-named classes in the t ree.

中文翻译

> 这点现在或许已不言自明，但仍值得最后强调一次：若使用多重继承，class 语句头部列出的超类顺序可能至关重要。Python 始终按头部行的顺序，从左到右搜索超类。例如，在第 31 章研究的多重继承例子里，假设 Super 类也实现了 str 方法：

```
class ListTree:
    def __str__(self): ...
class Super:
    def __str__(self): ...
class Sub(ListTree, Super):      #  ListTree  ListTree.__st…
    ...
x = Sub()                        #  ListTree  Super
```

> 我们会从哪个类继承它——ListTree 还是 Super？由于继承搜索一般从左到右进行，我们会拿到 Sub 类头里排在最左的那个类的方法。想必你会把 ListTree 排前面，因为它的全部意义就在于自定义 str。但现在假设 Super 与 ListTree 还有各自版本的其他同名属性。如果我们想从 Super 拿一个名字、从 ListTree 拿另一个名字，光靠顺序帮不上忙——必须在 Sub 类里手动给属性名赋值来覆盖继承：

```
class ListTree:
    def __str__(self): ...
    def other(self): ...
class Super:
    def __str__(self): ...
    def other(self): ...
class Sub(ListTree, Super):      #  ListTree.__str__
    other = Super.other          #  other  Super
    def __init__(self): ...
x = Sub()                        #  Sub  ListTree/Super
```

这里 Sub 类体内给 other 的赋值创建了 Sub.other——它引用回 Super.other 对象。因为它在树中更低，Sub.other 有效地隐藏了本会被继承搜索找到的 ListTree.other。同理，若我们把 Super 排在最左以取它的 other，就需要显式挑出 ListTree 的方法：

```
class Sub(Super, ListTree):      #  Super.other
    __str__ = ListTree.__str__   #  __str__  ListTree
```

此技巧的另一个实例见"属性冲突解决（Attribute Conflict Resolution）"。归根结底，多重继承是高级工具。即使你完全看懂上一段，也还是应该少用、慎用。否则，一个名字的含义可能会取决于"某个离你八丈远的子类里类的混合顺序"。一条经验法则：当你的 mix-in 类尽量自足时，多重继承效果最好——因为它们可能被用在各种环境里，不应假设树里其他类相关的名字。我们在第 31 章学的伪私有 X 属性特性有帮助：它把类赖以拥有的名字局部化，并限制 mix-in 类往混合物里加的名字。例如本例里，若 ListTree 只打算导出它的自定义 str，就可以把 other 方法命名为 other，避免与树里同名类冲突。

深度理解

·核心概念：多重继承 = 从左到右搜索 + 手动属性赋值覆盖 + X 名字局部化，三层策略齐备才安全。

·底层实现：MRO 由 C3 线性化（保持基类从左到右顺序）生成；Sub.other = Super.other 在更低层"遮蔽"查找结果。

·设计原因：把"冲突"当作必须显式处理的问题（Explicit is better），而不是悄悄规定某个赢家。

·实际问题：框架的 mix-in 类（日志、序列化、渲染）相互"同名撞车"是真实事故高发区；X 前缀是廉价保险。

·初学者误区：以为"调换顺序"能解决所有冲突；顺序只解决"整体偏好"，无法解决"各取其一"。

方法与类中的作用域（Scopes in Methods and Classes）

英文原文

> When working out the meaning of names in class-based code, it helps to remember that classes introduce local scopes, just as functions do, and methods are simply further nested functions. In the following example, the generate function returns an instance of the nested Hack class. Within its code, the class name Hack is assigned in the generate function's local scope and hence is visible to any further nested functions, including code inside method; it's in the E enclosing-function layer of the LEGB scope lookup rule:

```
def generate():
    class Hack:                   # Hack  generate
        count = 1
        def method(self):
            print(Hack.count)     #  LEGB E  generate
    return Hack()
generate().method()
```

This example works because the local scopes of all enclosing function defs are automatically visible to nested defs—including nested method defs, as in this exa

mple. Even so, keep in mind that method defs cannot see the local scope of the enclosing class; they can se e only the local scopes of enclosing defs. That's why methods must go through the self instance or the cla ss name to reference methods and other attributes d efined in the enclosing class statement. For example, code in the method must use self.count or Hack.coun t, not just count. To avoid nesting, we could restructu re this code such that the class Hack is defined at the top level of the module: the nested method function and the top-level generate will then both find Hack i n their global scopes:

```
def generate():
    return Hack()
class Hack:                         #
    count = 1
    def method(self):
        print(Hack.count)           #
generate().method()
```

Code tends to be simpler in general if you avoid nest ing classes and functions. On the other hand, class ne sting is useful in closure contexts, where the enclosin g function's scope retains state used by the class or it s methods. In the following, the nested method has a ccess to its own scope, the enclosing function's scop e (for label), the enclosing module's global scope, an ything saved in the self instance by the class, and the class itself via its nonlocal name:

```
> >> def generate(label): #

...     class Hack:
...         count = 1
...         def method(self):
...             print(f'{label}={Hack.count}')
...     return Hack
> >> aclass = generate('Gotchas') >> I = aclass() >> I.method()

Gotchas=1
```

中文翻译

> 在类代码里推敲名字的含义时，记住这一点很有用：类像函数一样引入局部作用域，而方法不过是更深一层的嵌套函数。在下面的例子里，generate 函数返回嵌套 Hack 类的一个实例。在它的代码中，类名 Hack 在 generate 函数的局部作用域里被赋值，因此对任何更深层的嵌套函数（包括 method 内部的代码）可见——它处在 LEGB 作用域查找规则的 E（闭包）层：

```
def generate():
    class Hack:                   # Hack  generate
        count = 1
        def method(self):
            print(Hack.count)     #  LEGB E  generate
    return Hack()
generate().method()
```

这个例子成立，是因为所有外层函数 def 的局部作用域对嵌套 def 自动可见——包括本例这种嵌套的方法 def。即便如此，请记住：方法 def 看不到外层 class 的局部作用域；它只能看到外层 def 的局部作用域。这就是为什么方法要引用"在外层 class 语句里定义的方法与属性"时，必须经

由 self 实例或类名——例如方法里的代码必须写 self.count 或 Hack.count，不能只写 count。若想避免嵌套，可以把 Hack 类挪到模块顶层：嵌套的 method 函数与顶层的 generate 就都能在自己的全局作用域里找到 Hack：

```
def generate():
    return Hack()
class Hack:                         #
    count = 1
    def method(self):
        print(Hack.count)           #
generate().method()
```

总体而言，代码若避免"类与函数互相嵌套"，往往更简洁。但另一方面，类嵌套在闭包情境里很有用——外层函数的作用域会保留类或其方法所用到的状态。下面这个例子里，嵌套的 method 能访问：它自己的作用域、外层函数的作用域（取 label）、外层模块的全局作用域、类存进 self 实例的一切，以及经 nonlocal 名字拿到的类本身：

```
>>> def generate(label):          #
...     class Hack:
...         count = 1
...         def method(self):
...             print(f'{label}={Hack.count}')
...     return Hack
>>> aclass = generate('Gotchas')
>>> I = aclass()
>>> I.method()
Gotchas=1
```

深度理解

·核心概念：作用域层级铁律——方法能看到"外层 def 的作用域"，看不到"外层 class 的作用域"。

·底层实现：函数作用域在"编译时"确定（LEGB 静态作用域）；class 语句体是"执行期命名空间"，不是方法查找的闭包环境。

·设计原因：类体是"构建类对象的代码"，运行一次即完成；方法在实例调用时才运行——两者生命周期不同，自然不能共享词法作用域。

·实际问题：类内想"引用隔壁类属性"而省略 self/类名，是新人高频报错点（NameError）。

·初学者误区：以为"类里定义的方法就像类里的函数一样自由引用类名"；必须显式走 self. 或 ClassName.。

杂项类的陷阱（Miscellaneous Class Gotchas）

英文原文

> Here's a handful of additional class-related warnings, mostly as review: Choose per-instance or class storag e wisely. Be careful when you decide whether an attri bute should be stored on a class or its instances: the former is shared by all instances, and the latter will di ffer per instance. In a GUI program, for instance, if yo u want information to be shared by all of the window class objects your application will create (e.g., the last directory used for a Save operation or an already ent ered password), it might be best stored as class-level data; if stored in the instance as self attributes, it will vary per window or be missing entirely when looked up by inheritance. You usually want to call superclass

constructors. Remember that Python runs only one in it constructor method when an instance is made—th e first it finds by inheritance. It does not automaticall y run the constructors of all superclasses higher up. B ecause constructors normally perform required startu p work, you'll usually need to run a superclass constr uctor from a subclass constructor—using either an ex plicit call through the superclass's name or super, pas sing along whatever arguments are required—unless you mean to replace the super's constructor altogeth er, or the superclass doesn't have or inherit a constru ctor at all. Stay tuned for a fix for getattr and built-in s. Another reminder: classes that use the getattr oper ator-overloading method to delegate attribute fetche s to wrapped objects may fail unless operator-overlo ading methods are redefined in the wrapper class. Th e names of operator-overloading methods implicitly fetched by built-in operations are not routed throug h generic attribute-interception methods. To work ar ound this, you must redefine such methods in wrapp er classes, either manually, with tools, or by definitio n in superclasses; you'll learn how in Chapter 39. "Ov erwrapping-itis" Finally, when used well, the code reu se features of OOP make it excel at cutting developm ent time. Sometimes, though, OOP's abstraction pote ntial can be abused to the point of making code diffi cult to understand. If classes are layered too deeply, code can become obscure; you may have to search t

hrough many classes to discover what an operation does. Imagine, for example, a framework with hundreds of classes and a dozen levels of inheritance. Deciphering method calls in such a complex system may be a monumental task: multiple classes might have to be consulted for even the most basic of operations. In fact, the logic of such a system can be so deeply wrapped that understanding a piece of code in some cases may require days of wading through related files. The most general rule of thumb of Python programming applies here, too: don't make things complicated unless they truly must be. Wrapping your code in multiple layers of classes to the point of incomprehensibility is always a bad idea. Abstraction is the basis of polymorphism and encapsulation, and it can be a very effective tool when used well. However, you'll simplify debugging and aid maintainability if you make your class interfaces intuitive, avoid making your code overly abstract, and keep your class hierarchies short and small unless there is a good reason to do otherwise. Remember: code you write is generally code that others must read.

中文翻译

这里还有一小撮与类相关的附加警告，大多算复习：明智地选择"每实例存储"还是"类存储"。决定一个属性该放类上还是实例上时要小心：前者被所有实例共享，后者每个实例各不同。比如在 GUI 程序里，如果你想让你应用创建的所有"

窗口类对象"共享某信息（例如"上次保存用的目录"或"已输入过的密码"），最好把它存成类级数据；若存成实例的 self 属性，它就会随窗口而异，或者在按继承查找时干脆找不到。你通常应该调用超类构造器。记住：Python 创建实例时只运行一个 init 构造器方法——即它按继承找到的第一个。它不会自动运行更上面所有超类的构造器。因为构造器通常负责必需的启动工作，你通常需要在子类构造器里调用超类构造器——要么用超类名的显式调用，要么用 super，把所需参数一并传下去——除非你想彻底替换超类构造器，或超类压根没有、也没继承到构造器。期待一个 getattr 与内建的修复吧。再提醒一次：用 getattr 运算符重载方法把属性获取委托给被包装对象的类，可能失效——除非在包装类里重新定义运算符重载方法。内建操作隐式获取的运算符重载方法名，不会经过通用属性拦截方法。绕开之法是在包装类里重定义这些方法：手工写、用工具生成、或在超类里定义——第 39 章你会学到怎么做。"过度包装症"（Overwrapping-itis）最后，用得好的话，OOP 的代码复用特性让它在削减开发时间上出类拔萃。但有时，OOP 的抽象潜力会被滥用，直到代码难以理解。若类分层太深，代码会变得晦涩：你可能要在许多类里翻找，才能搞清一个操作到底做了什么。例如，想象一个有着几百个类、十几层继承的框架。在这种复杂系统里厘清方法调用，可能是项庞大工程：哪怕最基本的操作，也可能要查阅好几个类。事实上，这样的系统逻辑可能被包裹得太深，以至于某些情况下，理解一段代码需要好几天的时间泡在相关文件里。Python 编程最通用的经验法则在这里同样适用：除非确实必须，否则别把事情搞复杂。把代码裹进多层类直到不可读，永远是个坏主意。抽象是多态与封装的基础，用好了是非常有效的

工具。但如果你让类接口直观、避免代码过度抽象、并让类层级短而小（除非有充分理由不这么做），你就能简化调试、助益维护。记住：你写的代码，通常也是别人要读的代码。

深度理解

·核心概念：三条复习警告（存储位置、超类构造器、委托与内建）+ 一条工程格言（过度包装）。

·底层实现：继承只运行一个 init（MRO 第一个）；运算符协议从类找、不走 getattr。

·设计原因：这些是"类即命名空间 + 协议分层"模型的必然推论——提醒读者把直觉建立在机制上。

·实际问题：GUI 共享状态、构造器链断链、委托包装器静默失败，是三类高频真实 bug。

·初学者误区：以为"多重构造器会被依次自动调用"；以为"包装类全自动转发一切"；以为"层级越多越优雅"。

32.12 章末小结（Chapter Summary）

英文原文

This chapter presented an assortment of class-related topics, including subclassing built-in types, the relationship of types and classes, slots, properties, static methods, decorators, and super. Most are optional extensions to the OOP toolbox in Python but may become more useful as you start writing larger object-orie

nted programs, and all are fair game if they appear in code you must understand. As noted earlier, some of these topics are continued in the final part of this book; be sure to look ahead for more info on properties, descriptors, decorators, and metaclasses. This is the end of the class part of this book, so you'll find the usual lab exercises at the end of the chapter: be sure to work through them to get some practice coding real classes. In the next chapter, we'll begin our look at our last core language topic, exceptions—Python's mechanism for communicating errors and other conditions to your code. This is a relatively lightweight topic but it was saved for last because new exceptions must be coded as classes. Before we tackle that final core subject, though, take a look at this chapter's quiz and the lab exercises.

中文翻译

本章呈现了一批与类相关的主题，包括子类化内置类型、类型与类的关系、slots、properties、静态方法、装饰器与 super。其中多数是 Python 工具箱里的可选扩展，在你开始编写大型面向对象程序时会日渐有用；而只要它们出现在"你必须弄懂的代码里"，就都属于"考试范围"。如前所述，其中一些主题会在本书最后一部分延续——记得向前翻，去看 properties、descriptors、decorators 与元类的更多信息。这是本书"类"这一部分的终点，照例章末有一批实验练习（lab exercises）：务必把它们做一遍，好在编码真实代码时得到锻炼。下一章我们开始看最后一个核心主题

——异常（exceptions）——Python 用来把错误与其它状况传达给你的代码的机制。这是个相对轻量的主题，却被留到最后，因为新异常必须以类来编写。在攻克那最后的核心主题之前，先看看本章的测验（quiz）与实验室练习吧。

深度理解

·核心概念：章末小结把整章内容浓缩成"子类化/SMKH"等一行行引索，并预告三大延拓（属性、装饰器、元类）。

·设计原因：本书的结构"类→异常"是为了让读者先掌握"类"，因为异常本身就是类。

·后续：第 33 章开始异常；第 38–40 章是本章高级主题的深化。

·初学者误区：把"读完简单小结"当作"已完成全章"；小结只是地图，练习才是路。

Test Your Knowledge: Quiz（测验）

英文原文

1. Name two ways to extend a built-in object type. 2. What are function and class decorators used for? 3. How are normal and static methods different? 4. Are tools like slots and super valid to use in your code?

中文翻译

> 1. 说出两种扩展内置对象类型的方法。2. 函数装饰器与类装饰器各有何用途？3. 普通方法与静态方法有何不同？4. slots 与 super 这类工具用在你的代码里合法有效吗？

深度理解

四个问题分别对位本章四大主题：子类化（类型扩展）、装饰器、静态方法、以及"全有或全无"的工具权衡。检验的不是死记，而是"知道该用哪种武器、何时用"。

Test Your Knowledge: Answers（测验答案）

英文原文

1. You can embed a built-in object in a wrapper class, or subclass the built-in type directly. The latter approach tends to be simpler, as most original behavior is automatically inherited. This works because types are classes, though the way you create an instance from a type/class determines its functionality. 2. Function decorators are generally used to manage a function or method, or to add to it a layer of logic that is run each time the function or method is called. They can be used to log or count calls to a function, check its argument types, and so on. They are also used to "declare" static methods (simple functions in a class that

are not passed an instance, however they are called), as well as class methods and properties. Class decora tors are similar but manage whole objects and their i nterfaces instead of a function call. 3. Normal (instan ce) methods receive a self argument (the implied inst ance), but static methods do not. Static methods are simple functions nested in class objects. To make a m ethod static, it must either be run through a special b uilt-in function or be decorated with decorator synta x. Python also allows simple functions in a class to be called through the class without this step, but calls th rough instances still require static-method declaratio n. 4. Of course, but you shouldn't use advanced tools automatically without considering their implications. Slots, for example, can break code; super can mask la ter problems when used for single inheritance, and in multiple inheritance brings with it substantial comple xity for an isolated use case; and both require univers al deployment to be most useful. Evaluating new or a dvanced tools is a primary task of any engineer, and t his is the reason we look for trade-offs in thinking. T his book's goal is not to tell you which tools to use b ut to underscore the importance of objectively analyz ing them—a task often given too low a priority in the software field. In engineering, as in life in general, we must not let other people make choices for us.

中文翻译

> 1. 你可以把内置对象嵌入一个包装类（wrapper class），也可以直接子类化内置类型。后者往往更简单——大多数原生行为会自动继承。这之所以成立，是因为"类型就是类"；不过你用类型/类创建实例的方式，决定了它的功能。2. 函数装饰器一般用来管理某个函数/方法，或给它加上"每次调用都会运行的逻辑层"。它们可以给函数记录或计数调用、检查参数类型等。它们也被用来"声明"静态方法（类中不管怎么调用都不会被传实例的简单函数）、类方法与属性。类装饰器类似，但管理的是整个对象（及其接口），而非一次函数调用。3. 普通（实例）方法接收 self 参数（隐含的实例），静态方法则不然。静态方法是嵌套在类对象里的简单函数；要让方法成为静态，要么经特殊内建函数、要么用装饰器语法。Python 也允许类里的简单函数不经处理就通过"类"调用，但"经实例调用"仍要求静态方法声明。4. 当然有效。但你不能"不经权衡"就用高级工具。例如 slots 可能弄坏代码；super 用于单继承可能掩盖日后的问题、用于多重继承则带来一大堆复杂度，只为换来一个孤立用例；两者都要"通用部署"才能发挥最大效用。评估新工具/高级工具，是每个工程师的首要任务——这正是我们在这章里一路权衡的原因。本书的目标不是告诉你"该用哪个工具"，而是强调"要客观地分析它们"——这项任务在软件行业里常被低估。在工程里，就像在生活中一样，我们不应该让别人的选择替我们做决定。

深度理解

·核心概念：答案第 4 题实际上是全文的方法论总结："评

估（evaluate）而非盲从（follow）"。

·应考提示：这三题都要用"机制 + 权衡"两层作答：先说清楚原理，再说优缺点。

·工程师素养：作者把"评估工具"升格为工程师首要任务——学习 Python 的过程也应培养这种"敢于质疑工具"的定力。

·初学者误区：背诵"工具 X 好不好"的结论，而不是掌握"Why/Whether"的判据。

Test Your Knowledge: Part VI Exercises（第六部分练习）

英文原文

These exercises ask you to write a few classes and experiment with some existing code. Of course, the problem with existing code is that it must be existing. To work with the set class in exercise 5, either copy/paste Example 32-1 from emedia, find it in this book's examples package (see the Preface), or type it up by hand (mildly tedious but a great way to make syntax more concrete). These programs are growing sophisticated, so be sure to check the solutions at the end of the book. You'll find them in Appendix B, under "Part VI, Classes and OOP". 1. Inheritance. Write a class called Adder that exports a method add(self, x, y), which prints a "Not Implemented" message. Then, define tw

o subclasses of Adder that implement the add metho d: ListAdder (with an add method that returns the co ncatenation of its two list arguments) and DictAdder (with an add method that returns a new dictionary co ntaining the items in both its two dictionary argumen ts; any definition of dictionary addition will do—see dictionary union in Chapter 8). Experiment by making instances of all three of your classes interactively and calling their add methods. Now, extend your Adder s uperclass to save an object in the instance with a con structor (e.g., assign self.data a list or a dictionary), a nd overload the + operator with an add method to a utomatically dispatch to your add methods (e.g., X + Y triggers X.add(X.data, Y)). Where is the best place t o put the constructors and operator-overloading met hods (i.e., in which classes)? What sorts of objects ca n you add to your class instances? In practice, you mi ght find it easier to code your add methods to accept just one real argument (e.g., add(self, y)) and add tha t one argument to the instance's current data (e.g., s elf.data + y). Does this make more sense than passin g two arguments to add? Would you say this makes y our classes more "object-oriented"? 2. Operator overl oading. Write a class called MyList that shadows ("wr aps") a Python list: it should overload most list opera tors and operations, including +, indexing, iteration, slicing, and list methods such as append and sort. Se e the Python reference manual for a list of all possibl

e methods to support. Provide a constructor for your class that takes an existing list (or a MyList instance) and copies its components into an instance attribute. Experiment with your class interactively. Things to ex plore: - Why is copying the initial value important her e? - Can you use an empty slice (e.g., start[:]) to copy the initial value if it's a MyList instance? - Is there a g eneral way to route list method calls to the wrapped l ist? - Can you add a MyList and a regular list? How a bout a list and a MyList instance? - What type of obje ct should operations like + and slicing return? What about indexing operations? - You may implement thi s sort of wrapper class by embedding a real list in a s tandalone class or by subclassing the built-in list. Wh ich is easier, and why? 3. Subclassing. Make a subclas s of MyList from exercise 2 called MyListSub, which e xtends MyList to print a message to stdout before ea ch call to the + overloaded operation and counts the number of such calls. MyListSub should inherit basic method behavior from MyList. Adding a sequence to a MyListSub should print a message, increment the c ounter for + calls, and perform the superclass's meth od. Also, introduce a new method that prints the ope ration counters to stdout and experiment with your c lass interactively. Do your counters count calls per ins tance or per class (for all instances of the class)? How would you program the other option? (Hint: it depen ds on which object the count members are assigned t

o: class members are shared by instances, but self members are per-instance data.) 4. Attribute methods. Write a class called Attrs with methods that intercept every attribute qualification (both fetches and assignments), and print messages listing their arguments to stdout. Create an Attrs instance and experiment. What happens when you try to use the instance in expressions? Try adding, indexing, and slicing the instance. (Note: a fully generic approach based on getattr requires Chapter 39's workarounds for reasons noted in Chapter 28.) 5. Set objects. Experiment with the set class of Example 32-1. Run commands to: - Create two sets of integers, and compute their intersection and union by using & and | operator expressions. - Create a set from a string, and experiment with indexing your set. Which methods in the class are called? - Try iterating through the items in your string set using a for loop. Which methods run this time? - Try computing the intersection and union of your string set and a simple Python string. Does it work? Now, extend your set by subclassing to handle arbitrarily many operands using the args argument form. Compute intersections and unions of multiple operands with your set subclass. How can you intersect three or more sets, given that & has only two sides? How would you go about emulating other list operations in the set class? (Hint: add can catch concatenation, and getattr can pass most named list method calls like append to the wra

pped list.) 6. Class tree links. In "Namespaces: The Conclusion" and "Multiple Inheritance and the MRO", we learned that classes have a bases attribute that returns a tuple of their superclass objects. Use bases to extend any of the listing mix-in classes we wrote in Chapter 31 so that they print the names of the immediate superclasses of the instance's class. Modding Example 31-10 first may be easiest. When you're done, the first line of the string representation should look like this (your hex addresses will vary):

```
<Instance of Sub(Super, Lister), address 0x...:
```

7. Composition. Simulate a fast-food ordering scenario by defining four classes: Lunch (a container and controller class), Customer (the actor who buys food), Employee (the actor from whom a customer orders), and Food (what the customer buys). The order simulation should work as follows: - The Lunch class's constructor should make and embed an instance of Customer and an instance of Employee, and export a method called order. When called, this order method should ask the Customer to place an order by calling its placeOrder method. The Customer's placeOrder method should ask the Employee object for a new Food object by calling Employee's takeOrder method. - Food objects should store a food name string (e.g., "burritos"), passed down from Lunch.order, to Customer.placeOrder, to Employee.takeOrder, and finally to Food's constructor. - The top-level Lunch class should also

export a method called result, which asks the custom er to print the name of the food it received from the Employee via the order (this can be used to test your simulation). Note that Lunch needs to pass either the Employee or itself to the Customer to allow the Cust omer to call Employee methods. Experiment with you r classes interactively; the Customer is the active age nt—how would your classes change if Employee were the initiator instead? 8. Zoo animal hierarchy. Consid er the class tree shown in Figure 32-1 (a zoo hierarch y composed of classes linked into an inheritance tree). Code a set of six class statements to model this tax onomy with Python inheritance. Then, add a speak m ethod to each of your classes that prints a unique me ssage and a reply method in your top-level Animal su perclass that simply calls self.speak to invoke the cat egory-specific message printer in a subclass below. Fi nally, remove the speak method from your Hacker cla ss so that it picks up the default above it. When you'r e finished, your classes should work this way:

```
$ python3
> >> from zoo import Cat, Hacker >> spot = Cat() >> spot.reply() #…

meow
> >> data = Hacker() # Animal.reply  Primate.speak >> data.reply…

Hello world!
```

中文翻译

这些练习要求你写几个类，并用一些既有代码做实验。当然，"既有代码"的问题就是它必须"已经有"。要处理练习 5 里的集合类，可从 emedia 复制/粘贴 Example 32-1、在本书示例包（见序言）里找，或亲手打出来（略繁琐，但能让语法更具体）。这些程序已变得相当复杂，务必查看书末的答案——在附录 B 的"Part VI, Classes and OOP"下。1. 继承。写一个类 Adder，导出一个方法 add(self, x, y)，打印"Not Implemented"消息。再定义 Adder 的两个子类来实现 add 方法：ListAdder 的 add 返回两个列表参数的拼接；DictAdder 的 add 返回"同时包含两个字典参数所有项"的新字典（字典加法怎么定义都行——参见第 8 章的字典并集）。交互地创建三个类的实例并调用它们的 add 方法。现在扩展你的 Adder 超类：用构造器把对象存进实例（例如把 self.data 赋成列表或字典），并用 add 方法重载 + 运算符，使其自动分发给 add 方法（如 X + Y 触发 X.add(X.data, Y)）。构造器与运算符重载方法最适合放在哪里（哪个类）？能给类实例加哪些种类的对象？实践中你可能会发现：把 add 方法写成只收一个真实参数（如 add(self, y)），配合实例当前数据（如 self.data + y）更简单。这比"传两个参数给 add"更有道理吗？这会让你的类更"面向对象"吗？2. 运算符重载。写一个类 MyList，影子化（"包装"）一个 Python 列表：重载 +、索引、迭代、切片以及 append、sort 等列表方法。查看 Python 参考手册了解所有可支持方法。给它提供构造器：接收一个现有列表（或一个 MyList 实例）并把其成分复制进实例属性。交互实验，探索这些点：- 为何复制初始值在此很重要？- 若初始

值是 MyList 实例，能用空切片（如 start[:]）复制吗？- 有没有通用办法把列表方法调用路由给被包装的列表？- 能把 MyList 和普通列表相加吗？反过来（列表 + MyList 实例）呢？- +、切片应返回什么类型的对象？索引操作呢？- 你可以"嵌入一个真实列表到独立类"或用"子类化内置 list"的方式实现包装类。哪种更简单？为什么？3. 子类化。为练习 2 的 MyList 写子类 MyListSub，扩展 MyList：在每次 + 重载操作调用前打印一条消息，并统计此类调用的次数。MyListSub 应继承 MyList 的基本方法行为。给 MyListSub 追加一个序列时，应打印消息、把 + 调用计数 +1、并执行超类的方法。再引入一个打印操作计数器到 stdout 的新方法，交互实验。你的计数器是"每实例"还是"每类"（所有实例共享）？另一选项怎么实现？（提示：取决于 count 成员赋给了谁——类成员被实例共享，而 self 成员是每实例数据。）4. 属性方法。写类 Attrs，其方法拦截每一次属性限定（读取与赋值都算），并把参数以消息形式打印到 stdout。创建实例实验。把它放进表达式会怎样？试试对它做加、索引、切片。（注意：完全通用的 getattr 方案需要第 39 章的工作区，原因在第 28 章已说明。）5. 集合对象。拿 Example 32-1 的 set 类做实验：- 建两个整数集合，用 & 与 | 运算符计算交并集。- 由一个字符串创建集合，试索引。会调用类里哪些方法？- 用 for 循环遍历字符串集合的项，这次哪些方法跑起来？- 用字符串集合与普通 Python 字符串求交集并集，能行吗？再用 args 子类化扩展你的集合，支持任意多操作数。如何用 & 求三个及以上集合的交——& 只有两个边啊？如何在集合类里模拟其他列表操作？（提示：add 可捕获拼接，getattr 可把 append 之类的具名列表方法传给被包装列表。）6. 类树链接。在"命

名空间：结论"与"多继承与 MRO"中我们学过：类有 bases 属性返回其超类对象元组。用 bases 扩展第 31 章写过的任一 "列表 mix-in 类"，让它的输出里打印实例所属类的直系超类名。先改 Example 31-10 最省事。完成后，字符串表示的首行应长这样（十六进制地址随你环境而定）： <Instance of Sub(Super, Lister), address 0x...: 7. 组合。用四个类模拟一个快餐点餐场景：Lunch（容器与控制器）、Customer（买食物的行动者）、Employee（顾客向其下单的行动者）、Food（顾客买的东西）。点餐模拟按如下流程运转：Lunch 的构造器创建并嵌入一个 Customer 实例与一个 Employee 实例，导出方法 order；调用 order 时先要求 Customer 调 placeOrder 下单，而 Customer.placeOrder 又调 Employee.takeOrder 取一份新 Food。Food 对象存一个食物名字符串（如"burritos"），从 Lunch.order 一路传下去。顶层 Lunch 还导出 result 方法，请顾客打印它从 Employee 处拿到的食物的名字（可用于测试）。注意 Lunch 需把 Employee（或它自己）传给 Customer，否则 Customer 无法调 Employee 方法。交互测试你的类：顾客是"主动代理"，若换成 Employee 发起交互，你的类要怎么改？8. 动物园类层级。参照图 32-1（由"链接进继承树的类"构成的动物园层级），用 Python 继承写六个 class 语句建模这一分类体系。给每个类加 speak 方法打印独特消息；顶层 Animal 超类加 reply 方法，简单调用 self.speak 以触发下层子类的分类专属消息打印（这会从 self 发起一次独立的继承搜索）。最后把 Hacker 类的 speak 方法删掉，让它改用其上方的默认。完成后类应按如下工作：

```
$ python3
>>> from zoo import Cat, Hacker
>>> spot = Cat()
>>> spot.reply()                # Animal.reply  Cat.speak
meow
>>> data = Hacker()             # Animal.reply  Primate.speak
>>> data.reply()
Hello world!
```

深度理解

·练习 1 要点：Adder 用"打印 NotImplemented 的占位方法"演练多态；+ 运算符重载分发到 add 即"运算符 → 方法"的转运。

·练习 3 的计数器揭示"类成员 vs 实例成员"的安放问题——实践版"共享 vs 遮蔽"。

·练习 5 的 for 循环追问"哪些方法运行"：调用的是 getitem 与 iter（依次、按优先级）。

·练习 6 用 bases 串联出 Animal(Lion, Tiger) 式的类树展示。

·练习 8 的 spot.reply() 体现"模板方法"：reply 定义在基类、speak 多态于子类。

WHY YOU WILL CARE: OOP BY THE MASTERS（奇君•大师论 OOP）

英文原文

Almost invariably, when teaching live Python classes,

about halfway through the OOP section, people who have used OOP in the past are following along intens ely, while people who have not are beginning to glaz e over (or nod off completely). The point behind the t echnology just isn't apparent. A book like this has th e luxury of slowly presenting material like the overvie w in Chapter 26, and the gradual tutorial of Chapter 28—in fact, you should probably review those sectio ns again if you're starting to feel like OOP is just som e computer science mumbo-jumbo. Though OOP add s more structure than the generators we met earlier, i t also relies on some magic (an inheritance search an d a special first argument) that beginners can find dif ficult to rationalize. In real classes, however, to help g et the newcomers on board (and keep them awake), i t often helps to stop and ask the experts in the audie nce why they use OOP. Here, then, with only a few e mbellishments, are the most common reasons to use OOP, as cited by students over the years: Code reuse — This one's easy and is the main reason for using O OP. By supporting inheritance, classes make it natural to program by customization instead of starting each project from scratch. Encapsulation — Wrapping up i mplementation details behind object interfaces insul ates users of a class from code changes. Structure — Classes provide new local scopes, which minimize na me clashes. They also provide a natural place to write and look for implementation code and to manage ob

ject state. Maintenance — Classes naturally promote code factoring, which allows us to minimize redundancy. Due to both the structure and code reuse support of classes, usually only one copy of the code needs to be changed. Consistency — Classes and inheritance allow you to implement common interfaces and hence create a common look and feel in your code; this eases debugging, comprehension, and maintenance. Polymorphism — This is more a property of OOP than a reason for using it, but by supporting code generality, polymorphism makes code more flexible and widely applicable and hence more reusable. And, of course, the number one reason students gave for using OOP: it looks good on a résumé!

中文翻译

几乎无例外地，当我面对面上 Python 课讲到 OOP 部分中间时，用过 OOP 的人正全神贯注，没用过的人开始两眼无神（或彻底昏昏欲睡）——这门技术背后的意义，并不那么显而易见。一本书可以悠悠地按第 26/28 章那样的节奏呈现素材——事实上，如果你开始觉得"OOP 不过一堆 CS 黑话"，建议你回去重读那几节。OOP 比前面遇到的生成器多了不少结构，但同样依赖一些"魔法"（继承搜索与特殊的第一参数），初学者难以理性化这些。但在真实的课堂里，为了让新人上车（并让他们保持清醒），常常有效的一招是"停下来请教在场的高手：你为什么要用 OOP？"。这里，稍加润色地，历年学生给出的使用 OOP 的最常见理由如下：代码复用（Code reuse）——这一条最简单，也是用 O

OP 的主因：通过继承，类让"用定制的方式编程"变得自然，而不是每个项目从零开始。封装（Encapsulation）——把实现细节裹在对象接口后面，隔离了类使用者免受代码变更的影响。结构（Structure）——类提供新的局部作用域，减少名字冲突；也提供了书写与查找实现代码、管理对象状态的自然场所。维护（Maintenance）——类天然促进代码"因子化"（factoring），从而让我们最小化冗余。凭借结构与代码复用，常常只需要改动代码的一份拷贝。一致性（Consistency）——类与继承让你实现统一接口，从而在代码里创造统一的"观感"，这让调试、理解与维护都更轻松。多态（Polymorphism）——与其说是用 OOP 的理由，不如说是它的属性：通过支持代码的通用性，多态让代码更灵活、更普适、因而更可复用。其他（Other）——当然，还有个学生给出的头号理由：它写进简历很好看！

深度理解

·核心概念：OOP 六大价值（复用/封装/结构/维护/一致/多态）+ 一句行业实话（简历好看）。

·作者意图：用"为何要用"的问题收束章末，重申"读者只有实践才懂 OOP 的好处——去写大项目、做练习"。

·学习建议：把六大价值当作"看完第六部分后的心智检查清单"：你写的类是否占齐这几样？

·初学者误区：以为 OOP 是"最潮的规则方法"；作者明确说，它"不是必须、有时甚至过度"，有理由地不用也是智慧。

本章总结

本章是第六部分"类与 OOP"的收官章，主题像一桌"高级杂烩"：Python 对象模型（type/object 的循环关系与两棵继承树）、slots、properties、描述符预览、静态方法与类方法、装饰器与元类、super 的完整故事，以及一整套类的陷阱与练习。

技术拓展（Technical Expansion）

实际项目中的应用场景

主题	应用场景
slots	高并发缓存对象、游戏实体、传感器数据包等"实例海量 + 属性固定"的内存敏感程序
property	数据校验（age 范围、email 格式）、计算属性（面积、全名）、把内部 x 平滑升级为公共 x
静态/类方法	实例计数器、工厂方法（@classmethod 返回 cls(...)）、类级注册表、配置读取
装饰器	日志/计时/重试、参数校验、路由注册（Flask/Django）、属性私有化（第 39 章）
元类	ORM 模型注册（SQLAlchemy 的 Declarative）、单例、自动接口生
super	框架基类的"协作式构造器"、钻石继承链的初始化

与其他语言的区别（Java/C++ 视角）

机制	Python	Java/C++

多重继承	支持 + MRO（C3 线性化），super 沿"MRO 尾部"查找	Java 接口多实现；C++ 虚继承手动处理菱形
属性访问控制	无 private 关键字；约定 x / 伪私有 x	private/protected/public 编译期强制
方法类型	instance/static/class 三种，@staticmethod 等装饰	static 修饰符；无类方法（可模拟）
元编程	装饰器 + 元类（type 子类）	注解/反射（运行时），无类创建期拦截
运算符重载	双下划线协议（add 等）全量	仅 C++ 支持（operator 函数）；Java 不支
类即对象	类是 type 的实例，可被替换/动态修改	类是编译期产物，运行时不可替换

Python 发展历史背景

·新式类（new-style，2.2 起）统一了类型与类，本章全部机制（MRO、super、描述符）都建立其上；Python 3 只保留新式类。

· slots 随新式类引入（2.2），本意是模拟 C++ 固定布局的效率。

·装饰器在 Python 2.4 引入（PEP 318），最初动机正是简化 staticmethod/classmethod 声明；@ 任意表达式支持到 3.9。

·类装饰器随 Python 3.0 引入（PEP 3129）。

· super() 无参形式自 Python 3.0 起可用（PEP 3135），依赖 class 闭包单元。

·作者观测：slots 在 3.x 时代（CPython 3.12 实测）已无速度优势，仅剩内存收益。

高级开发者需要掌握的相关知识

·描述符协议（get/set/delete）——slots、property、绑定方法、staticmethod/classmethod 的共同底座（第 38 章）。

· C3 线性化与 MRO——super 的行为根基，菱形继承下的协作分发（第 31、40 章）。

· getattribute 与属性解析顺序——数据描述符优先于实例字典，运算符协议绕行。

·元类 new/init 与类装饰器——"类创建时刻"的两大拦截入口（第 40 章）。

·工具类对 slots 的策略——dir + getattr、沿 MRO 扫描、hasattr 探测（mapattrs.py 模式）。

学习建议（Learning Advice）

·重要程度：★★★★☆（4/5）。本章主题"高级且可选"，但type/class 对象模型、@staticmethod/@classmethod/@property 装饰语法、super 的语义是阅读任何成熟 Python 框架的必备装备；slots 与元类可按需学习。

·掌握程度：

·必须会：说出 type/object/class 三者的关系；区分静态方法与类方法；看懂 @property 与 @staticmethod 的写法；理解 super 在单继承与多重继承下的行为差异（锚/参数/部署三要求）。

·应该会：用 slots 时知道它的六条规则与"全有或全无"属

性；能写简单的类式与闭包式装饰器；能说出类装饰器与元类的区别与重叠。

·可以不会：手写元类、描述符完整实现——属于工具作者范畴（第 38–40 章再学）。

·后续内容：

·第 33–35 章：异常（例外如何作为类工作、用户自定义异常）。

·第 36–37 章：模块与包的系统化（命名空间思想的另一面）。

·第 38 章：property/descriptor 深入；第 39 章：装饰器专题；第 40 章：元类专题——本章埋下的伏笔都在那里引爆。

·学习建议：做完本章 8 道实验练习（尤其 Adder、MyList、zoo），再带着"两棵继承树、三个方法协议、一条 MRO"这三句话去读任何框架的基类代码——你会发现它们出奇地好懂。